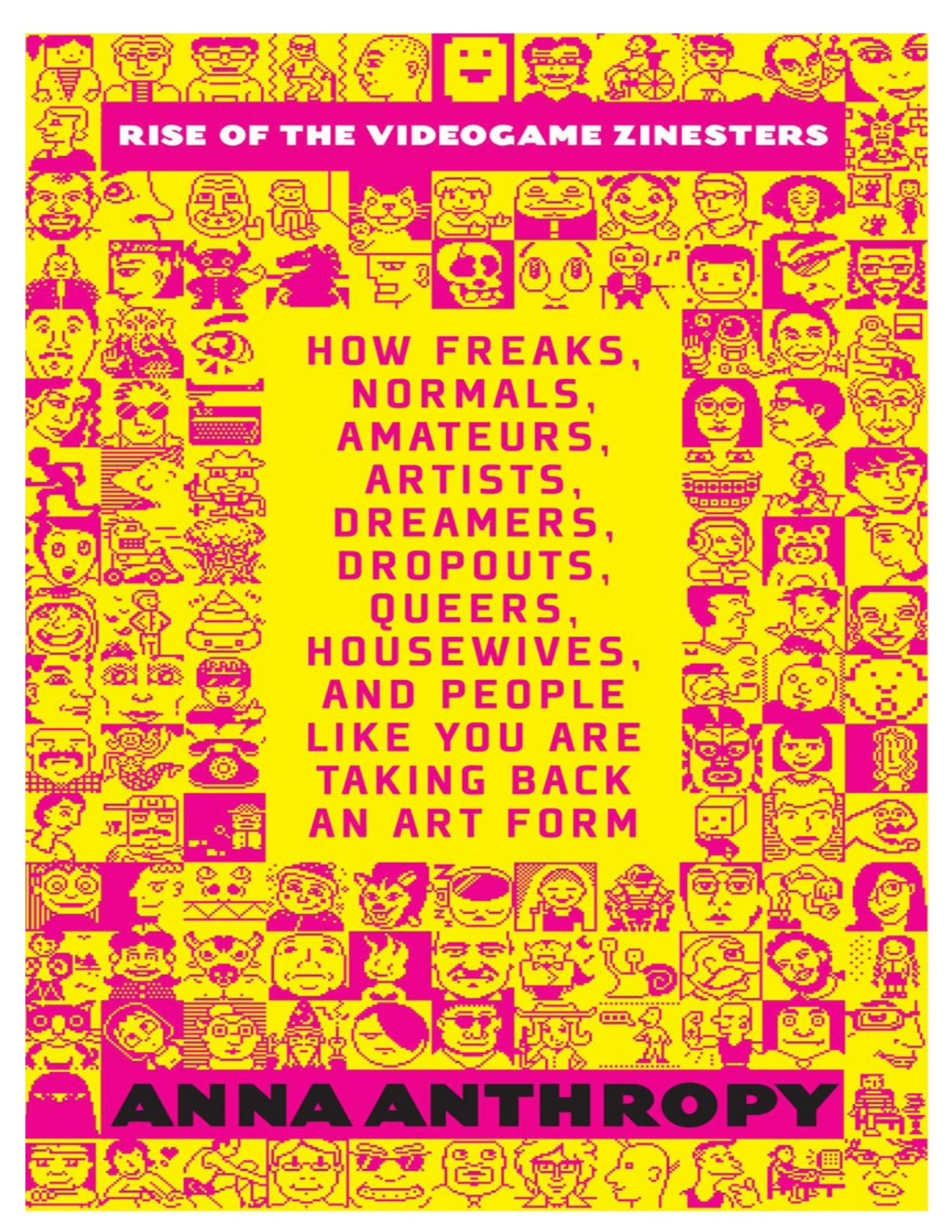




RISE OF THE VIDEOGAME ZINESTERS

HOW FREAKS,
NORMALS,
AMATEURS,
ARTISTS,
DREAMERS,
DROPOUTS,
QUEERS,
HOUSEWIVES,
AND PEOPLE
LIKE YOU ARE
TAKING BACK
AN ART FORM

ANNA ANTHROPY

RISE OF THE VIDEOGAME ZINESTERS

**HOW FREAKS,
NORMALS,
AMATEURS, ARTISTS,
DREAMERS, DROPOUTS,
QUEERS,
HOUSEWIVES, AND
PEOPLE LIKE YOU
ARE TAKING BACK
AN ARTFORM**

**ANNA
ANTHROPY**

**Seven Stories Press
New York**

Copyright © 2012 by Anna Anthropy

A Seven Stories Press First Edition

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, including mechanical, electronic, photocopying, recording, or otherwise, without the prior written permission of the publisher.

Seven Stories Press
140 Watts Street
New York, NY 10013
www.sevenstories.com

College professors may order examination copies of Seven Stories Press titles for a free six-month trial period. To order, visit <http://www.sevenstories.com/textbook> or send a fax on school letterhead to (212) 226-1411.

Book design by Elizabeth DeLong

Library of Congress Cataloging-in-Publication Data

Anthropy, Anna.

Rise of the videogame zinesters : how freaks, normals, amateurs, artists, dreamers, drop-outs, queers, housewives, and people like you are taking back an art form / Anna Anthropy.

p. cm.

Includes bibliographical references and index.

ISBN 978-1-60980-372-8 (pbk.)

Ebook ISBN: 978-1-60980-373-5

1. Video games. 2. Video games--Social aspects. I. Title.

GV1469.3.A53 2012

794.8--dc23

2011045409

Printed in the United States

9 8 7 6 5 4 3 2 1

*For the child I was,
the book no one could write for me.*

Contents

[Chapter One: The Problem with Videogames](#)

[Chapter Two: The History of Magic](#)

[Chapter Three: What Is It Good For?](#)

[Chapter Four: Changing the Game](#)

[Chapter Five: The New Videogame](#)

[Chapter Six: Making the Games](#)

[Chapter Seven: By Your Bootstraps](#)

[Chapter Eight: Growing Up](#)

[Appendix A: What to Use](#)

[Appendix B: Zinester Games](#)

[About the Author](#)

Chapter One

The Problem with Videogames

I have a problem with videogames.¹

Plenty of people seem to have problems with videogames these days. Newscasters are fond of reporting that videogames are dangerous to children, either because they teach children how to steal cars and kill cops or because they actually connect children electronically to the game-playing predators who are waiting to snatch them away. Religious leaders have wasted no time condemning videogames as a trap for children's souls, and armchair psychologists accuse them of turning kids into antisocial hermits.

People condemn videogames because videogames are pervasive in popular culture. They're on our computers and our cell phones, in our homes and purses and pockets. Even if you yourself don't play games, you have a hard time escaping their marketing. When the television isn't telling you to be afraid of videogames, it's telling you to buy them, and to drink *World of Warcraft*-flavored Mountain Dew while you play.

These are some problems people have with videogames. What's *my* problem with videogames?

As a queer transgendered woman in 2012, in a culture pervaded by videogames—a culture in which, typing on my computer, I am seconds away from a digital game, even if I have not taken the time to buy or install a single game on my computer—I have to strain to find any game that's about a queer woman, to find any game that resembles my own experience.

This is in spite of the fact that videogames in America and elsewhere are an industry and an institution. I've already brought up *World of Warcraft*, a game about performing repetitive tasks until numbers increase. So, now that we're in the land of numbers, here are some numbers. The ESA—that's the Entertainment Software Association, who spend half their time assuring the population that videogames aren't worth being mad at, and the other half pursuing litigation against anyone who distributes games that their shareholders have long since stopped distributing or profiting

from—claims that, as of 2009, 68 percent of American households play digital games.² In 2008 alone, people bought 269,100,000 games (the ESA word is *units*).³

So digital games, by the numbers, are here, and they take up a lot of space. And almost none of these games are about me, or anyone like me.

What are videogames about?



Mostly, videogames are about men shooting men in the face. Sometimes they are about women shooting men in the face. Sometimes the men who are shot in the face are orcs, zombies, or monsters. Most of the other games the ESA is talking about when it mentions “units” are abstract games: the story of a blue square who waits for a player to place him in a line with two other blue squares, so he can disappear forever. The few commercial games that involve a woman protagonist in a role other than slaughterer put her in a role of servitude: waiting tables at a diner (or a dress shop, a pet shop, a wedding party). This is not to say that games about head shots are without value, but if one looked solely at videogames, one would think the whole of human experience is shooting men and taking their dinner orders. Surely an artistic form that has as much weight in popular culture as the videogame does now has more to offer than such a narrow view of what it is to be human.

And yes, from here on out I'll be talking about videogames as an art form. What I mean by this is that games, digital and otherwise, transmit ideas and culture. This is something they share with poems, novels, music albums, films, sculptures, and paintings. A painting conveys what it's like to experience the subject as an image; a game conveys what it's like to experience the subject as a system of rules. If videogames are compared unfavorably to other art forms such as novels and songs and films—and they are compared unfavorably with these forms, or else this paragraph defending videogames as art wouldn't be necessary—it is likely a result of how limited a perspective videogames have offered up to this point. Imagine a world in which art forms are assigned value by the number of dykes that populate them. This is the world I inhabit; this is the value games have for me. And why not? The number of stories from marginalized cultures—from people who are othered by the mainstream—that a form contains tells us something about that form's maturity. If a form has attracted so many authors, so many voices, that several of them come from experiences outside the social norm and bring those experiences and voices to bear when working in that form, can't that form be said to have reached cultural maturity?

It should go without saying that novels and films have plenty of dykes in them, as do the media of writing and filmmaking. American comics have been around since 1896—that's over one hundred years—yet comics are still involved in a debate, as videogames are, about their cultural and artistic value. But I can think of many comics about queer women. More important, I can think of plenty of queer women who make comics: to name a few, Diane DiMassa, Alison Bechdel, Jennifer Camper, Kris Dresen, and Colleen Coover, in order of how disappointed I was when they came out in defense of the Michigan Womyn's Music Festival.⁴ And those are just print comics, in a world where the majority of comics are published on the Internet.

In Alison Bechdel's *Dykes to Watch Out For*, Mo (a dyke to watch out for) explains a metric she uses to decide whether she'll watch a movie. This criteria has become known as the Bechdel Test: the movie has to (1) contain at least two women who (2) talk to each other about (3) something

other than a man. So why do videogames fail my variant of the Bechdel Test? Why are there no dykes in videogames?

I know at least one of you has been itching, for several pages, to point out games like *Fear Effect 2: Retro Helix* and *Mass Effect*, both of which include scenes in which women smooch women, both on and off camera. In *Fear Effect 2*, women make out for the benefit of the male audience the game's creators expect to buy the game. (The first scene, in fact, is of the protagonist stripping as seen through a hidden camera, which tells us something about her relationship to the player.) And the lady-sex in *Mass Effect* is just one of many branches on a tree of awkward dialogue, offering nothing resembling the actual lust, desire, and flirtation that women feel for each other. But, aesthetic failures aside, the most important distinction here is that these are stories about queer women that are generally written by white, college-educated men. These are not cases of queer women presenting their own experiences.

Why are digital games so sparse in the dykes making art department? Why are the experiences that games present, the stories they tell, the voices in which they speak, so limited?

The limitations of games aren't just thematic. When I criticize games for being mostly about shooting people in the head, that's a design criticism as well. Most games are copies of existing successful games. They play like other games, resemble their contemporaries in shape and structure, have the same buttons that interact with the world in the same way (mouse to aim, left click to shoot), and have the same shortcomings. If there's a vast pool of experiences that contemporary videogames are failing to tap, then there's just as large a pool of aesthetic and design possibilities that are being ignored. I don't believe these are separate issues, either. To tell different stories, we need different ways of interacting with games. Why are games so similar in terms of both content and design?

The problem with videogames is that they're created by a small, insular group of people. Digital games largely come from within a single culture. When computers were first installed in college campuses and laboratories, only engineers had the access to the machines, the comparative leisure time, and the technical knowledge to teach those computers to play games. It is not surprising that the games they made looked like their own experiences: physics simulations, space adventures drawn from the

science fiction they enjoyed, the *Dungeons & Dragons* tabletop role-playing games they played with their friends. As computers made their way out of labs and into homes, the games that programmers were hacking together became a salable product—and salespeople showed up to profit off of them. And so as businessmen and marketers guided videogames into becoming a billion-dollar industry, publishers installed themselves as the gatekeepers of game creation.

Commercial games have become expensive: according to a presentation at the High Performance Graphics 2009 conference, *Gears of War 2*—an industry leader in the “men shooting things” genre—had a “development budget” of 12 million dollars.⁵ (“Development” refers just to the cost of creating the game—it doesn’t include all the bucks that were spent marketing, manufacturing, and shipping the game.) If the game cost that much to produce, you can imagine what it would have to earn in sales in order to make any money. Hint: more than 12 million dollars. With that much money at stake, publishers and shareholders are not going to permit a game that is experimental either in terms of its content or in terms of its design. The publisher will do the minimum amount it can get away with in order to differentiate its game from all other games that follow its previously established model and that are being sold to its previously established audience.

Now we have a dangerous cycle: publishers permit only games that follow a previously established model to be marketed to previously established audiences, and only to those audiences. The audiences in question are mostly young adults, and mostly male. And it’s these dudes, already entrenched in the existing culture of games, who are eventually driven to enter the videogame industry and to take part in the creation of games. The population who creates games becomes more and more insular and homogeneous: it’s the same small group of people who are creating the same games for themselves.

Videogames as they’re commonly conceived today both come from and contain exactly one perspective. It should be terrifying that an entire art form can be dominated by a single perspective, that a small and privileged group has a monopoly on the creation of art. Before the adoption of the printing press, the church was responsible for the creation of books, and

the books that monks hand-lettered in Latin in monasteries were largely the Bible or books that agreed with the Bible. Not to knock the Bible, but that a single institution can hold power over what works are allowed to exist within any art form should demonstrate the power that institution has over that art form, and therefore over that culture. And so the printing press, which allowed people to print their own versions of the Bible in their own languages—and eventually to print books that had nothing to do with the Bible—had a role to play in the decentralization of religious authority in Europe.

The printing press is a piece of technology. If digital games, a form that is often (and not entirely correctly) described as being “technology driven,” can be compared to books, where then is the printing press for videogames?

What Videogames Need

There’s a videogame about a dyke who convinces her girlfriend to stop drinking. Mainstream gamer culture by and large does not know about this game. I know about this game because I made it.

I created *Calamity Annie* in 2008. I made it by myself: I wrote the dialogue, composed the music, designed the rules, scripted the game, and drew all the characters. It was made in a couple of months. The development costs were the cost of the food that went into my belly. I made the game in a program called Game Maker, which, at the time, cost fifteen dollars.

I am nowhere close to the only person who has used Game Maker, nowhere close to the only person who makes digital games outside of the games industry’s publisher model. There are hundreds, if not thousands, of such creators. A few of them have achieved some mainstream recognition, like Jonathan Blow and Jason Rohrer, who was profiled in *Esquire* magazine. But these rich white dudes were professional programmers before they came to videogames, and so they don’t represent the new dynamic that I’m excited about: hobbyists and non-programmers making their first games. There are lots of tools that allow people to make and distribute games without ever having written a line of code and without

having to pass through publishers' gates. In years to come, there will be a lot more tools. I hope that there will also be a lot more people.

I once heard the criticism that the phrase "what videogames need" can usually be more honestly rephrased as "what I want from videogames." In that case, what I want from videogames is a plurality of voices. I want games to come from a wider set of experiences and present a wider range of perspectives. I can imagine—you are invited to imagine with me—a world in which digital games are not manufactured by publishers for the same small audience, but one in which games are authored by you and me for the benefit of our peers.

This is something the videogame industry, by its nature, cannot give us. I like to think about zines—self-published, self-distributed magazines and books. Send me a dollar and a self-addressed envelope; I'll send you a stapled book of some stories from my life, or some pictures I took of out-of-the-way nooks of my city, or researched accounts of historical murders, or some jokes about sea life. (What does the merman's waiter bring? He brings the MERMANATEE.⁶) I like the idea of games as zines: as transmissions of ideas and culture from person to person, as personal artifacts instead of impersonal creations by teams of forty-five artists and fifteen programmers, in the case of *Gears of War 2*.

The Internet in particular has made self-publishing and distributing games both possible and easy. Authors are able not only to put their works online, but to find audiences for them. Publishers want to be gatekeepers to the creation of videogames, but the Internet has opened those gates.

Currently, the only real barrier to game creation is the technical ability to design and create games—and that, too, is a problem that is in the process of being solved.

Digital game creation was once limited to those who knew how to speak with computers: engineers and programmers, people who could code. In the games industry of today, coders are an inescapable fixture of the hierarchy of production, since games that we play with machines need creators capable of negotiating with machines. Game creation is daunting for someone who doesn't code professionally. But more and more game-making tools are being designed with people who aren't professional coders in mind. (I describe several of these tools, and what each is good

for, in the appendix.) It's now possible for people with no programming experience—hobbyists, independent game designers, zinesters—to make their own games and to distribute them online.

What I want from videogames is for creation to be open to everyone, not just to publishers and programmers. I want games to be personal and meaningful, not just pulp for an established audience. I want game creation to be decentralized. I want open access to the creative act for everyone. I want games as zines.

It's a tall order, maybe, but the ladder's being built as you read these words.

Is What You Want Really What Games Need?

Why transform videogames, though? What do I get out of it? What, for that matter, do videogames get out of it?

In 2005, movie critic Roger Ebert infamously remarked that he does not think games can ever be considered as art. (By whom? By him, apparently.) He argues, mostly by assertion, that he doesn't feel game designers can exercise enough authorial control over the experience of a game. Ebert has gone on to make no attempt to justify or defend his remark or engage in any kind of debate, other than to allow, five years after the original remark, that he should have kept his opinion to himself.⁷

As I've made clear above, Ebert is wrong about videogames as a form. But frankly, I don't care whether Ebert is wrong or not. Achieving "artistic legitimacy" is not a good reason to transform videogames. Who legitimizes art? To cede the right to decide the value of games to an authority that has nothing to do with games—or to concede the right to decide what is and is not art to any authority outside of the artist—is a dangerous trap. Creation is art. It doesn't need validation beyond that.

What it needs is to be free. That an art form exists should be justification enough for people to be able to contribute to it, to work in it. We finally have the means to allow more than just programmers and big game publishers to create games—and the vast majority of people in the world aren't computer engineers, or designers employed by Epic Games. What do we gain from giving so many people the means to create games? We gain a lot more games that explore much wider ground, in terms of both

design and subject matter. Many of these games will be mediocre, of course; the majority of work in any form is mediocre. But we'll see many more interesting ideas just by the sheer mathematical virtue of so many people producing so many games without the commercial obligations industry games are beholden to. Remember, I'm talking about hobbyists, people who make games in their spare time with the tools they have on hand. And even if a game isn't original, it's personal, in the way a game designed to appeal to target demographics can't be. And that's a cultural artifact our world is a little bit richer for having.

To visualize this new world of games, think about network television versus YouTube. The former spends a lot of money and time creating content designed to appeal to the lowest common denominator. Because network shows need to justify themselves monetarily—they need to catch enough viewers to earn advertising dollars—they can rarely afford to be brilliant, daring, or bizarre. (Sometimes a director has enough force of will, and fights the network hard enough, to create a show that is all of these things. But it's certainly not the norm.)

YouTube: millions of videos from millions of authors. Most of them are mediocre: boring, familiar, or unwatchable. That's to be expected in an arena where everyone is allowed to contribute. But others are sublime, brilliant, valuable: Grishno's "Transgender in New York" videos,⁸ wendyvainity's surreal computer animations and music,⁹ or shaneduarte's *Simpsons* remixes.¹⁰ As long as there's some sort of infrastructure, valuable works—those by both dabbling amateurs and dedicated artists—can reach their audiences. YouTube has its own infrastructure of user ratings and featured videos, but people are just as likely to share the addresses of specific videos with the friends they think those videos will appeal to. And there's far more value in the collective content of YouTube—even given that there are more piles of trash than treasure—than in the collective content of a television network, simply as a function of the number of people contributing and the overwhelming volume of their contributions. YouTube's content is far more diverse, too, since involvement in the television industry isn't a requirement for entry. Network television shows are all made by professionals working in the field, a far smaller set of people than the set of people who own webcams.

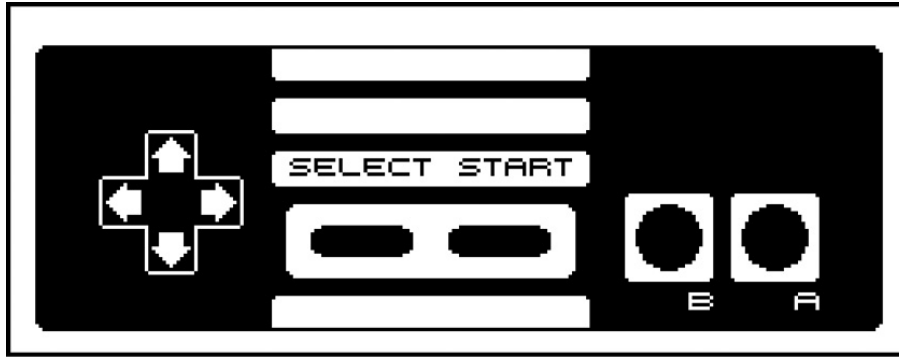
YouTube's content is made much more quickly and cheaply because it's not (usually) designed with a commercial agenda: videos can be recorded and broadcast, and their value assessed later.

YouTube also gives people the means to make videos of themselves, their friends, their babies, and their puppies—video snapshots—not for the world at large, but for their social circles and themselves. YouTube is a means of transmitting a video directly from the author to an audience—one that can be as small and specific as the author desires. Videos become more specialized, and hence more personalized. A medium that was formerly accessible only to those with money and training can now be used by anyone for personal ends. If Internet television is in the process of reinventing television, imagine how game design tools for nonprogrammers and the free distribution of games online might reinvent videogames.

The Culture of Alienation

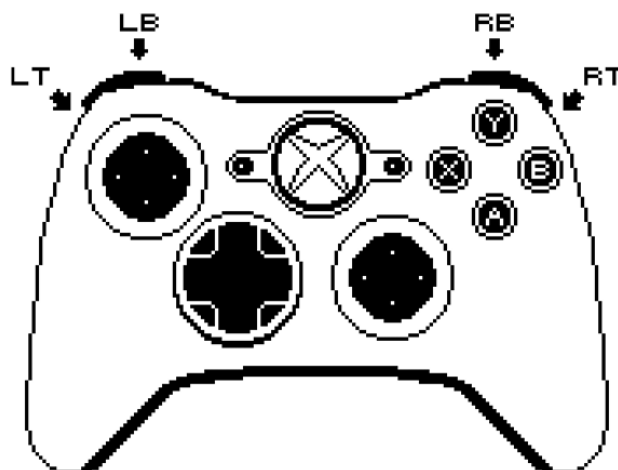
Limiting the creation of games to a small, exclusive group leads not only to creative stagnation, but also to the alienation of anyone outside that group. I've described the round-the-drain cycle the games industry is in: games are designed by a small, male-dominated culture and marketed to a small, male-dominated audience, which in turn produces the next small, male-dominated generation of game designers. It's a bubble, and it largely produces work that has no meaning to those outside that bubble, those not already entrenched in the culture of games.

There are mechanical consequences as well. Look at how game controllers have changed as their audiences changed. The home game machines of the 1970s and '80s, which marketed themselves to large, general family audiences, had the simplest control pads. The Atari Video Computer System (or the Atari 2600) is a simple joystick with a single button. And here is the design of the Nintendo Entertainment System (NES) controller, released in the United States in 1985:



The NES controller has a four-way compass rose and two prominent red buttons. (There are also two buttons in the center for secondary functions like pausing the game, but the design of the controller clearly communicates that they're peripheral.) You use the compass to navigate your character or cursor. You use the buttons to perform actions.

After over thirty years of catering to an audience that is continuously playing and learning games—an audience that hence requires more and more complicated games to interest it—games and the controllers with which players interact with them have become more and more complex. This is not to say *different*: layers of complexity have simply been added to the same few models of games and the same few models of controllers. Here's the controller for the Xbox 360, released in the United States in 2005:



The Xbox 360 controller is the same model as the NES controller: held between two hands, with navigational functions assigned to the left hand and manipulation verbs to the right. But instead of a single navigational pad on the left, two verb buttons on the right, and two option buttons in the center, the Xbox pad has a navigational pad plus a stick on the left, four verb buttons plus another stick on the right, four “shoulder” buttons on the top of the controller (two to each side), plus three option buttons in the center. (Additionally, some games call for the player to “click” either of the sticks in like a button, adding two more verbs.)

The means players use to interact with games guides the design of those games. A game for the NES might have a button for jump and a button for shoot, and the compass rose directional pad for moving a character left and right. You can imagine the kinds of games that are designed for eight buttons and four sticks. Imagine introducing someone who had never seen a movie before to Matthew Barney’s *Cremaster* films. The amount of both manual dexterity and game-playing experience required to operate a game designed for the Xbox 360 makes play inaccessible to those who aren’t already grounded in the technique of playing games. And to attain that level of familiarity with games requires a huge and continuous investment of time (and money—keeping up with new games costs bucks). This means that older people—people with families and obligations, people trying to raise kids, or any people with a lack of free time to invest—have a harder time gaining access to games. At the same time, as a side effect of this unnatural selection, commercial games become longer and longer, with game covers advertising dozens and occasionally hundreds of hours of gameplay. (*Shin Megami Tensei: Persona 3*, a PlayStation 2 game from 2008, advertises a “70+ hour game” on the back of its box.) Who has that much time to invest in playing a videogame? Answer: the target audience of most of the industry’s games, a mostly young and mostly male audience that has few obligations and plenty of disposable income.

The culture that this audience creates and exists within is one of in-jokes and brand worship, rituals to establish whether the participants are in or out of the tribe. It’s an exclusive culture, an alienating environment that speaks only to itself. Its interactions with the outside world are decidedly hostile.

Destructoid, one of the most popular sources for videogame news on the Internet, employs a writer named Jim Sterling who once called my girl a “feminazi slut” on Twitter. This isn’t some rogue nerd; this is a “journalist” whom *Destructoid* employs to write on such topics as whether the penis is more powerful than the vagina because it can rape,¹¹ or on whether female *Mortal Kombat* characters have secret cocks.¹² And lest you think that such a character couldn’t possibly be taken seriously, hundreds of his readers responded publicly to my open letter to *Destructoid* complaining about Sterling’s behavior in an attempt to bully and shame us.¹³ How is a woman, a trans person, or any rational individual expected to feel safe enough to participate in such a community?

What I want from videogames is for videogames to speak to more than just the handful of people already engaged in producing and consuming them. To de-monopolize game creation is to de-monopolize access to games.

Beyond Consumer

In an era when the Internet makes it easy to transmit and disseminate media, there’s no reason for people to accept that their only contribution to the growth of an art form is as a consumer, supporting “elite” creators with money.

I’ve wanted to make videogames since I played Fukio Mitsuji’s NES game *Bubble Bobble* as a kid. I drew characters on construction paper, cut them out, and laid out obstacle courses for them to navigate—*Bubble Bobble* stages on hardwood floors. But the technical leap to digitize my designs was beyond my reach. Programming was something mystical and arcane. I came into contact with code sometimes: the most basic BASIC examples. But something as simple as making a picture of a character move across a screen required a working knowledge of control loops, writing to video memory buffers, and advanced bit-shifting math—all of which was so inaccessible to me as a kid that I sublimated my childhood desire to make games until well into my adulthood.

It’s not like it was then. There’s a commercial product in videogame stores right now—*Warioware: D.I.Y.*, from Nintendo—that allows players to create their own small games.¹⁴ What *Warioware: D.I.Y.* does is to

introduce its players to the concept of designing rules, of using art and sound to communicate the state of the game to the player, of scripting the events of a game and of working cleverly within limitations. For kids today, digital game creation doesn't have to be the mystical process it was when I was little.

Kids today also have tools like Stencyl,¹⁵ a free tool for making games and distributing them online. A website collects kits and resources contributed by the entire community, which are all made available to an individual creator for use in her game. The rules are put together in Scratch,¹⁶ a system designed by programmers at MIT for young children to use. It involves snapping simple instructions together like LEGOs.

But before things like Stencyl and *Warioware* existed, I made games and digital stories however I could: an old DOS shooting-game creation program that I can no longer remember the name of, the track editor in Nintendo's *Excitebike*, an editor for creating worlds made out of text called *ZZT*. People with something to say will always manage to find ways to say it, and there's a history of clever people using whatever means they can find to modify and subvert digital games and to create new ones—to engage with games in a role beyond consumer. Today, this process is easier than ever.

The Big Crunch

This same false sense that the knowledge needed to create videogames is unattainable without special institutional training is the same carrot the Big Games Industry uses to entice wannabe game artists into taking jobs within their system—and putting up with insane hours and ridiculous working conditions. There exists within the games industry a phenomenon called “crunch mode”: working sixteen-hour days, staying at work until the game you're being paid to make is finished. This isn't something you're asked to do—it's expected, a standard condition of the job. And it's likely the reason most people in the games industry, their physical and mental health fizzled, burn out and quit within a few years, forced to retrain and find a new career. According to the International Game Developers Association (IGDA), the closest thing the industry has to an advocacy group for employees, 34 percent of game developers expect to

leave the industry within five years, and 51 percent—half of them!—expect not to last a decade.¹⁷ That's lunacy.

The industry gets away with this because it's convinced its employees that these jobs are the only gateway to videogame creation. "We've graciously allowed you to fulfill your childhood dream of making games. We're even paying you for it! And what's more, we're the only way you'll ever be able to do that." Mike Capps, a former member of the board of directors at the IGDA and the president of Epic Games said that Epic expected employees to work more than sixty hours a week and in fact only hired people they expected to be willing to do so.¹⁸ The IGDA has no official stance on the hours of unpaid overtime the people it claims to represent are obliged to do by their employers.

Since the industry sees itself as ubiquitous—as the only possible means of creating games—it feels no need to change itself for the benefit of either its employees or its art. Which is another reason why carving new paths to game creation and distribution is valuable. By undermining the industry's claim to being the only route to game creation—especially to making a living from game creation—we force the industry to reconsider its totalitarian attitude toward the people it employs. Publishers need creative people to make games for them. We have one foot in an era when creative people will no longer need publishers to distribute their games.

Creating more and better games will also challenge the industry creatively. Spending millions of dollars to remake the same seventy-hour-long games for the same small audience is no longer feasible when so many people want different experiences out of games and have the means to find them elsewhere. Games from hobbyists have the potential to change the dominant format of the videogame: instead of seventy-hour multimillion dollar games that sell for sixty bucks apiece, digital games can be short and self-contained—less than an hour, short enough to fit comfortably into an adult player's day. The focus of games could shift from features, the ways in which a game is differentiated from similar games—*thirty hours of play, twelve unique weapons, advanced four-dimensional graphics acceleration*—to ideas. Take Tarn Adams' *WWI Medic*¹⁹ for example: a game not about chain-gunning enemy soldiers but about trying to patch them up as the bullets cut them down. Saving even a

single soul—climbing out of the trench, grabbing a fallen body and lugging it back to safety under a senseless hail of bullets—is incredibly difficult. The game takes minutes to play, and communicates an idea about war that may perhaps be more valuable than space marines frotteurizing each other with chainsaws.

Smaller games with smaller budgets and smaller audiences have the luxury of being more experimental or bizarre or interesting than 12 million dollar games that need to play it as safely as possible to ensure a return on investment. Imagine what a videogames industry that wasn't fixated on hits—that wasn't required to make hits—would create.

What Are Games Good For?

But even given all of this, why worry about the accessibility of digital game creation at all when other forms—like the short story or novel—are already established and available for non-professionals to work in?

Answer: because different forms are suited to different kinds of expression, and some are more effective at communicating in certain ways than others. Broadly, films and photographs are best suited for communicating action and physical detail. Novels are best suited for communicating internal monologue and ambiguity.

What are games best suited for? Since games are composed of rules, they're uniquely suited to exploring systems and dynamics. Games are especially good at communicating relationships; digital games are most immediately about the direct relationship between the player's actions or choices and their consequences. Games are a kind of theater in which the audience is an actor and takes on a role—and experiences the circumstances and consequences of that role. It's hard to imagine a more effective way to characterize someone than to allow a player to experience life as that person.

Take, for example, a game called *We the Giants*.²⁰ Most people who connect to this game's website in order to play it—taking the role of a squat, block-like cyclops—will be unable to reach the game's goal, a star high in the sky. Rather, most players are given the responsibility of voluntarily dying in a position that will allow future players to use their solidified bodies as steps in a staircase leading skyward. Each player

guides her cyclops to the position of its sacrifice, presses a button, types a single message to future players of the game, and watches the cyclops's eye close forever. Thereafter, the player is never allowed to play the game again; logging on to the website, she can only watch the ongoing progress of the staircase of which her body is a part.

That's a pretty compelling way to explore themes of sacrifice in a work: to ask players actually to make a sacrifice, and to show them the meaning of that sacrifice over the course of generations. This is something games are almost uniquely capable of doing, and we haven't even begun to explore the possibilities of this kind of expression.

It's also the sort of experience—a minutes-long game in which the player is asked to commit voluntary suicide and never allowed to play again afterward—that is unlikely to come out of a commercial publishing system that needs its creations to sell millions in order to justify their having been made. The author of *We the Giants*, Peter Groeneweg, is a student and created the game as part of a monthly “experimental gameplay” challenge.²¹

The ability to work in any art form with the digital game's unique capabilities for expression shouldn't be restricted to a privileged (and profit-oriented) few. If everyone is given the means to work in an art form, then we'll invariably see a much more diverse, experimental, and ultimately rich body of work. In a speech at the 2007 Game Developers Conference, Greg Costikyan—a board and videogame designer and critic of the games industry—said: “I want you to imagine a 21st century in which games are the predominant art form of the age, as film was of the 20th, and the novel of the 19th.”²²

This is what I want from videogames, and this is what I'm trying to help you imagine. Throughout the rest of this book, I hope to help you imagine how this transformation of games—and the role games will play in the art and culture of the twenty-first century—is not only necessary, but inevitable.

Footnotes

¹ I'll use this term interchangeably with “digital games” and “electronic games” throughout this book.

- 2 ESA, *2009 Sales, Demographic and Usage Data: Essential Facts about the Computer and Video Game Industry*, p. 2, http://www.theesa.com/facts/pdfs/ESA_EF_2009.pdf.
- 3 Ibid, p. 10.
- 4 For insight into the Michfest controversy, see <http://www.auntiepixelante.com/?p=1247>. Also, I don't mean to imply that all of these artists have taken a position on Michfest. I'm really just disappointed in Diane Dimassa.
- 5 Tim Sweeney, "End of the GPU Roadmap" (keynote address, High Performance Graphics 2009, New Orleans, August 3, 2009), http://www.highperformancegraphics.org/previous/www_2009/presentations/TimHPG2009.pdf.
- 6 Sea life joke courtesy Emily Alden Foster.
- 7 "I was a fool for mentioning video games in the first place. I would never express an opinion on a movie I hadn't seen. Yet I declared as an axiom that video games can never be Art. I still believe this, but I should never have said so." *Roger Ebert's Journal*, "Okay, kids, play on my lawn," July 1, 2010, http://blogs.suntimes.com/ebert/2010/07/okay_kids_play_on_my_lawn.html.
- 8 <http://www.youtube.com/user/grishno>.
- 9 <http://www.youtube.com/user/wendyvainity>.
- 10 <http://www.youtube.com/user/shaneduarte>.
- 11 Jim Sterling, "How Aliens are blatantly better than Predators," *Destructoid*, February 9, 2010, <http://www.destructoid.com/how-aliens-are-blatantly-better-than-predators-162754.phtml>.
- 12 Jim Sterling, "Why Penis Why?: MK vs. DC's Kitana looks . . . lumpy," *Destructoid*, August 5, 2008, <http://www.destructoid.com/why-penis-why-mk-vs-dc-s-kitana-looks-lumpy-98300.phtml>.
- 13 Anna Anthropy, "An Open Letter to Destructoid on Jim Sterling's Misogyny," *Auntie Pixelante*, <http://www.auntiepixelante.com/?p=912>.
- 14 Because Nintendo controls the infrastructure by which five-second *Warioware: D.I.Y.* games are distributed, or even allowed to exist, transmission of those games from creator to creator is fairly restricted. What most creators do, in fact, is publish videos of themselves playing their games on YouTube.
- 15 <http://www.stencyl.com>.
- 16 <http://scratch.mit.edu>.
- 17 Casey O'Donnell, "Quality of Life in a Global Game Industry," *International Game Developers Association*, http://www.igda.org/articles/codonell_global.
- 18 Greg Costikyan, "Mothers, Don't Let Your Children Grow Up to Be Game Developers," *Play This Thing!*, April 3, 2009, <http://playthisthing.com/mothers-dont-let-your-children-grow-be-game-developers>.
- 19 <http://www.bay12games.com/ww1medic>.
- 20 Playable online at <http://wethegiants.thegiftedintrovert.com>.
- 21 "Best of the Net: Art Game," *Experimental Gameplay Project*, January 1, 2010, <http://experimentalgameplay.com/blog/2010/01/best-of-the-net-art-game>.
- 22 Greg Costikyan, "Maverick Award Speech," *Man!festo Games*, March 14, 2007, <http://www.manifestogames.com/node/3413>.

Chapter Two

The History of Magic

Since digital games have existed, their creation has been dominated by a small part of the population: generally white male engineers. In the 1960s and '70s, universities like MIT and Southern Illinois University contained computers and computer networks that were available for student use. Most of these games existed on the school network and were played and contributed to by only those people on the network. Often they were disguised as other programs, because systems administrators tended to delete games as a waste of time.

It's beside the point to try to identify the first videogame—as with most inventions, a number of people were working along the same lines simultaneously. But whatever the first game was, it had to have been inspired by something—so what came before it? Answer: an entire history of human civilization in which folk games—Go, Chess, Hide and Seek, Stickball—were important cultural experiences, that's what. But the most immediate predecessors of digital games were carnival games (throwing a ball at a stack of bottles from a set distance), mechanical games (a shooting gallery with moving targets), and pinball machines. Coincidentally, these are the games that typify the shift in the history of games from folk to designed games, or games with identifiable authors. When videogames were first monetized, it's this model that the people making money used: pay-to-play games of skill in public spaces designated for game-playing. But that's getting ahead of ourselves.

So, to create digital games in the sixties and seventies, one first needed access to a computer. The “home computer,” like the Apple Macintosh—a computer designed specifically for non-engineers—wasn't popularized until the eighties. To have access to a computer, then, generally required being connected to an engineering school. But being able to make contact with the computer was only the first barrier: in order to teach computers to play games, one needs to know how to talk to computers.

At the time, neither computers nor the tools people used to communicate with computers were designed with non-engineers in mind. Most programs

were written in the super-technical language Assembly. Here's a sample of game code written in Assembly, from the 1979 Atari 800 game *3-D Tic-Tac-Toe*.²³

```
stx CASINI,Y
rti
.byte $02,$80,$00,$00,$00,$00,$00,$00
.byte $00,$00,$00,$00,$00,$00,$03,$00
.byte $07,$14,$07
jmp L0714
and POKMSK,Y
.byte $00
L001D:      lda #$46
sta COLOR2
L0022:      bne L0022
ldy #$00
lda #$6B
sta (SAVMSC),Y
jsr L07D9
bcs L001D
jsr L07C4
lda L087A
ora L0876
bne L001D
lda LOMEM
sta L02E0
lda LOMEM+1
sta RUNAD+1
lda #$00
sta INITAD
sta INITAD+1
```

Completely illegible! By 2010, we have coding languages like Ruby and scripting languages like Lua that are designed to be readable by human beings, and we have tools like Scratch and Twine that minimize, if not obviate, the need for coding entirely. But in 1975, there was no way to

make a game on a computer without understanding the computer inside and out.

The Affairs of Wizards

What digital games were being made in the 1970s? And who was creating them?

The college engineer who programmed games in the mid-seventies had most likely been exposed to the role-playing game *Dungeons & Dragons* (*D&D*), published in 1974 by TSR, and possibly to the wargames that preceded it. *Dungeons & Dragons* is storytelling with rules—a human player, the “Dungeon Master,” presents story situations to which the other players must respond. The Dungeon Master keeps the rules and facilitates the adventure of the other players, each of whom plays a role within the game world. It borrows from wargames a complex set of rules and tables for resolving situations, mostly those related to combat: whether a sword hits an opponent, whether it does any damage to that opponent, how much damage it does. And it borrows the fantasy world—the wizards and dragons, orcs and elves—of J. R. R. Tolkien’s *Lord of the Rings* books, popular among student engineers at that time.

That *Dungeons & Dragons* was hugely influential on digital game creators of the seventies can be seen in network games like *dnd*—short for “Dungeons & Dragons,” naturally—created by Gary Whisenhunt and Ray Wood at Southern Illinois University in 1974, the year of *D&D*’s release.²⁴ Like TSR’s game, *dnd* involves descending into a dungeon, fighting monsters (dragons included), and collecting treasure. In this version of the game, it’s the computer that keeps the rules, taking on the “Dungeon Master” role that would formerly have been given to a human participant.

The Tolkienesque fantasy setting of *Dungeons & Dragons* is all but ubiquitous in digital games of the time, but what’s really interesting is the way designers transformed that setting by transposing it into a digital world. In 1977, at MIT, Tim Anderson, Marc Blank, Bruce Daniels, and Dave Lebling began working on a game initially called *Dungeon*, later renamed *Zork*. *Zork* is a text adventure game: the player is presented with a paragraph of descriptive text, types a sentence explaining the action she wishes to take, and is presented with further text by the game in response.

In this way, it resembles both prose fiction and the refereed experience of a game of round-the-table *Dungeons & Dragons*.

Anderson, Blank, Daniels, and Lebling, along with others, founded Infocom in 1979. The fantasy world that they created in *Zork* grew into many more games, such as *Enchanter* and *Spellbreaker*. But what's interesting to me is the particular way magic is treated within the fantasy world these MIT engineers built. As Jeff Howard writes for the blog *The Gameshelf*:

Infocom's *Spellbreaker* trilogy, consisting of *Enchanter*, *Sorcerer*, and *Spellbreaker*, entails a magical grammar, in which spells are verbs that take direct objects, allowing players to type "frotz stone" to make a stone glow or "blorb chest" to open this locked container. Frotz and Blorb are names for, respectively, an interactive fiction interpreter and a wrapper for multimedia elements. While amusing, this application of the names of in-game spells to the programming and technology outside of and supporting the game also suggests a powerful relationship between programming and the verbal grammars of magic. Simply put, programmers and magicians both master a grammar in order to make things happen. Both hackers and wizards achieve this alteration of reality, whether simulated or real, through an arcane set of words and phrases known as programs or spells. When properly configured, a program causes amazing events to occur (calculates our taxes, launches an anti-missile defense system, summons a longed-for package from Amazon.com to our doorstep), just as magicians can throw fireballs and (when very powerful) grant wishes. However, when the programmer makes the slightest error in the placement of a semicolon or case sensitivity, the program won't compile, much as a spell fizzles. ²⁵

It's not surprising at all that in a fantasy world constructed by programmers, the power to create and change the world would be indistinguishable from programming. For these engineers, technology was their means of making magic happen.

This theme of magic as technology (or technology as magic) doesn't occur only in Infocom games. The *Ultima* games were created by role-playing games enthusiast Richard Garriott, who named some of the people in his fantasy world after characters he role-played. The first *Ultima* game progresses from slaying monsters for Lord British and Shamino to flying a spaceship; the nemesis of the third *Ultima* game (*Ultima III: Exodus*) is ultimately revealed to be a computer that the player must reprogram. We can see the trend persisting into the MUDs (multi-user dungeon games, the descendents of games like *Dungeon*) and MOOs (MUD object oriented games), online games in which game administrators are called "Wizards"

and have the ability to ban players from the game by “toading” them (turning a player into a toad).

This early in the history of digital game creation, we can still see that games, as with all works of art, contain the values of the people who make them. Which is precisely why more than a single group of people should have access to the means of creating them.

Digital Barkers

The founders of Atari, NAMCO, and SEGA, three early videogame publishers that still exist (in some form) today, were all involved in either carnival barking or the distribution of pinball and other mechanical games. When these people set out to make digital games that earned them money, they used the system they were already familiar with: installing games in public spaces and calling people over to play them.

The arcade cabinets that housed these new games were designed to act as their own barkers: they were stylized pieces of wood decorated with artwork, containing video screens that flashed invitations to players and demonstrated their games. For a quarter (sometimes more), a player bought one play of the game. As in pinball, the player’s skill extended the game: the better you were, the longer you could play. Arcade cabinets were initially placed in settings like bars and, later—when videogames became popular among kids—pizzerias and malls. Eventually, they came to inhabit dedicated spaces—arcades—that existed solely to house digital games. The arcade cabinet was the way most of mainstream culture first encountered videogames.

How did the arcade game become more ubiquitous than the carnival and pinball games whose sales model it borrowed? Carnival and mechanical games were huge and required human supervision. Pinball machines, because they contained lots of moving parts that constantly collided with each other, broke frequently and were expensive to maintain. The first digital arcade games were cheaper to maintain than the pinball machine because the parts were all standard pieces of electronics, far easier to replace than a piece of track molded specifically for a single pinball machine. The arcade game was also more compact and self-regulating. Since it didn’t take up a lot of space or require constant maintenance by

the owner, it could inhabit spaces like the aforementioned bars and pizzerias, spaces not exclusively dedicated to electronics, and thus, spaces not exclusively populated by the engineers and programmers who, up to that point, had been almost exclusively the audience for digital games.

But naturally, the manufacture and distribution of these arcade cabinets required capital, and here businesspeople gained their foothold (soon to be a stranglehold) on videogames. Engineers, sometimes with the assistance of artists, still designed the games and the hardware that made them possible. Businesspeople handled distribution to bars, malls, and arcades. Venture capitalists were brought in to fund the costs of production and expansion. A need for marketers began to appear, although this was not as important as it would become later, when games weren't sold only to arcade operators but directly to players. But what's important to note is that it was the business folks, not the engineers or artists, who controlled the capital. As long as game creators were hardware manufacturers, this was the case.

The Invasion of Home

It wasn't long before the people who manufactured machines for the arcade hit upon the idea of manufacturing machines for the home. This would allow them to market their games not to the middleman arcade operator, but to the players themselves. Atari was among the earliest publishers to have great success in the arcade, and in 1977 it began to publish the Atari Video Computer System—later retroactively renamed the Atari 2600, after the Atari 5200 and 7800 went to market—selling home versions of its most popular arcade cabinets to players. Games for the Video Computer System were distributed on cartridges that plugged into a base machine, rather than on miniature arcade cabinets, which meant that after the initial purchase of the hardware, the actual game software became much cheaper to produce and distribute. The market exploded.

Arcade cabinets were more expensive to play than home game cartridges, and now that the player could play digital games in her own home, arcades became less of an attraction. Because home game hardware was fixed and arcade cabinet hardware was not—the home game player

buys a single piece of hardware, while most arcade cabinets have hardware specialized to the game that inhabits them—arcade cabinets still managed to offer unique experiences. Arcade games became more and more specialized over time, distributing games that were implausible in the home, either because of the technology or the context. The games that predominate in modern arcades are large ride-on vehicles, dance platforms, or drum sets that make less sense in the home than in a commercial space. Most arcades didn't survive this shift in the market: there aren't many arcades left these days, at least in America.

The shift in the way people discovered and played games also led to a shift in game design trends. Arcade games, because they earn money on each play, are designed to be as succinct as possible, and to teach new players how to play quickly. They are also often designed to be hard, because a player, once she loses, will either have to pay again to continue her game or relinquish the machine to a new player. Home games, which players pay for one time in exchange for infinite plays, require publishers to set the price of the game higher than the traditional quarter. Thus home games became longer and longer in an attempt to appear more valuable to potential players. They could have much longer learning curves and be much gentler to play. But this longer game requires more content, and hence bigger teams to design and create that content. Marketing, now that the games were sold directly to the player, became a powerful force, and began to make many of the creative decisions.

Take, for example, this account of a conflict between the marketers and game programmers at Mattel Electronics, publishers of the Intellivision and its software, related by former Mattel Electronics staff:

On December 6, 1982, all of the programmers and graphic artists were herded into a conference room and shown a series of TV commercials—the new Kool-Aid ad campaign. It was announced that Marketing had made a tie-in deal to release Intellivision and M Network Atari 2600 *Kool-Aid Man* cartridges. The games were scheduled to be ready in about six months, which meant that programming had to begin immediately. Worse, they wanted game-screen mockups to appear in the 1983 Mattel Electronics catalog at the Consumer Electronics Show—one month away. A two-week contest to come up with the best game concept was announced. Separate ideas were developed for Intellivision and Atari 2600.

This led to a confrontation with Marketing. The programmers' viewpoint was that the features of a game should be tailored to the system it would be played on, to take full advantage of the system's strengths. Marketing, on the other hand, wanted games

designed for multiple systems, with the features being the same on each system. If a game *couldn't* be ported to other systems, it shouldn't be done on *any* system.

The programmers argued that this meant all games would have to be designed for the lowest common denominator—the Atari 2600. Marketing argued that keeping the features the same would make games easier to advertise and make word-of-mouth among customers more favorable.²⁶

On contemporary home game consoles, most games come from companies other than the manufacturer of the hardware. The hardware manufacturer generally enforces an approval process for games commonly called “lot check,” or “technical requirements,” which contains a list of requirements the game must meet before it can be printed and distributed. For example: The game must display a message when a game controller is unplugged from the machine. It must support a variety of novelty controllers that have a limited run. There's a lot of room for error, and applying for the process isn't cheap. Rejection means that the fee will have to be paid again after the asked-for changes are made to the game, and there can be many rejections before a game is approved, putting distribution of digital games to early home consoles still well out of the financial reach of almost anyone outside of the growing industry, despite the cheaper manufacturing costs.

Game consoles weren't the only home invaders. In 1984, Apple released the Macintosh computer. Microsoft began distributing its Windows operating system with computers the following year. These machines were conceived and marketed as “personal computers,” designed for home use by non-engineers and marketed to the public. Their use of a mouse for navigating between different files and programs visually made these new computers far more approachable to non-engineers than the traditional text prompt, where users typed from a list of hidden commands.

Publishing a game for the home computer was similarly expensive to publishing one for a home game console. While game consoles have identical components (every individual PlayStation 3 has the same pieces and the same capabilities, with a few small deviations), computers aren't homogeneous. Certifying that a game will run on a wide variety of contemporary computers, with hundreds of potential variations in operating system, installed programs, hardware, and input devices, is an extensive and expensive process. Manufacturing the game, getting it on

store shelves, providing on-call technical support to players, and marketing it to those players all costs money.

The Games Publishing Industry Today

Given the expenses of distributing a game—lot check, compatibility testing, printing, marketing—how does anyone afford to make games?

The contemporary games industry uses a developer-publisher model. The developer actually designs, programs, and animates the game at the behest of the publisher, who pays the expenses of distributing it. The developer may pitch the game to the publisher, or the publisher may bring the game concept to the developer. The publisher might just own the developer: bigger publishers like EA (formerly Electronic Arts) and Ubisoft have purchased many development studios.

A developer may start a project with her own resources before attempting to find a publisher for the game. Because the publisher controls the distribution of the game, it has control over the content of the game. The publisher's agents will periodically check the progress of the game and demand changes from the developer. Often these changes are for the sake of marketing the game: a publisher will always do what it can to make a game more salable, or what it perceives as being more salable. A publisher may shape a game to better resemble trends in the widely selling games of the day.

Within a development company, employees are typically divided into three roles: designer, artist, and engineer or programmer. All of these roles have a technical (knowledge) barrier to entry. An artist doesn't just need to be able to draw; she needs to be proficient in the 3-D modeling software the developer prefers. She needs to know how to prepare images in a way that the engineers can use. A designer needs to be familiar with the "engine" the game is being developed in, and to be fluent in the scripting language that engine uses in order to create events and interactive elements within the level she designs.

A game is made by at least one team of each of these groups: a team of engineers under a lead engineer, a team of artists under a lead artist, and a team of designers under a lead designer (or "game designer"). The engineers/artists/designers receive their instructions from the leads. The

leads report to a director. The director reports to a producer, who in turn represents the publisher. Within this system, which exists to coordinate teams of increasingly unmanageable numbers of people (numbers needed to produce the huge amount of content Hit Games demand), you can see that the people who exercise the most creative power over the project are the people who are farthest from its creation.

The expenses of hiring and coordinating all these people mean that a game has to be a hit in the market in order to make a profit. And so the publisher, with its final authority on the content of a game, will almost always make a conservative decision about that content in order to make the game more marketable. If it wants to make a profit, the publisher is obligated to.

Publishers have installed themselves as gatekeepers to videogames publishing. To distribute and sell a game in the contemporary market requires their consent. But for as long as people have had access to computers, there's been a history of game creators who've sought alternative solutions to the problem of game distribution.

Rethinking Distribution: Share? Where?

The personal computer appeared in homes in the eighties. Personal computers are not just for consumption; they are also tools for creation. Anyone with the technical knowledge and the tools can make a game on a computer. And any game I make on my Windows (or Mac or Linux) computer, you can play on your Windows (or Mac or Linux) computer. It's just a matter of getting the game from my computer to yours. Distribution—whether it's intended to make a profit or not—has been the major problem of most small game creators.

“Shareware” was a popular concept in small game distribution throughout the eighties and nineties. Shareware relies on the players themselves to distribute a game. If I encounter a game I like, I might duplicate it and give a copy to a friend, who in turn makes more copies. Copying games initially meant floppy disks: the cost of producing digital media containing the game was deferred to the audience. Some authors might include their address in their games and ask for a tip: a donation of any amount, a postcard from somewhere interesting. Some authors, for the

cost of a disk and some compensation, might offer an expanded version of the game, a second episode or a sequel.

This is how Tim Sweeney of Epic MegaGames (now Epic Games, mentioned earlier) and Scott Miller of Apogee Software got their start. They reinvested the money their games earned into creating distribution networks, hiring developers to create more games, marketing their games at first through catalogs and eventually on store shelves. They went the path of the publisher, which unfortunately remains the only viable method for widespread physical distribution.

But the rise of online networks gave hobbyists and small game developers a new method—and critically, a wholly digital method—for distributing their games. The Bulletin Board System, or BBS, was a public online space that proliferated from the 1970s to the '90s. A home computer user with a modem could dial in to a BBS through the phone line, and would then have access to all the files available on that system: shareware games, for example, that could be downloaded to the user's computer. More important, that user, and countless other hackers, hobbyists, and coders, could upload games to that BBS for other users to download. Games could be passed from computer to computer this way. And they could be passed around without the need for physical copies and the associated costs. This means that non-professionals and non-publishers were able to transmit all sorts of games to players—and in fact, there's a swath of weird, personal, and experimental shareware games around that could never have come from the hit-driven games mainstream.

For example, I discovered a game when I was young called *Evolve! Lite*.²⁷ This game simulated life by allowing the player to program a species of digital creature with a set of different reactions to different stimuli (for example: when in the presence of two or more predators, the creature turns and runs in the opposite direction). Individuals of the species who mate pass on these tables of behaviors—this virtual DNA—but not all of it! Some of the behaviors will randomly mutate, as in real sexual reproduction, and individuals with beneficial mutations will survive long enough to pass on their mutated DNA. This shareware game, then, provides a working model of evolution!

The game was made in 1993 by Matt Bace and Mike Wall, who published under the label “FunTek.” This is all I know about them. The game is called *Evolve! Lite* because there supposedly exists an expanded version of the game called *Evolve!*, one that allows for a world that’s four times larger and populated with twelve competing species, rather than the two of *Evolve! Lite*. A registration form included with the game offers copies of *Evolve!* for \$19.95 plus shipping. It also encourages me to register on CompuServe (an online network of the time), and it contains an advertisement and phone number for JAB BBS: “We have one of the largest collection of PD [Public Domain] & Shareware.”

I actually discovered *Evolve! Lite* on a CD I bought in a store—a shareware CD containing the noncommercial versions of hundreds of shareware games. This was another solution to the problem of distributing shareware games: a small publisher would offer to distribute shareware authors’ games in stores, and the publisher would sell the CD to buyers, promising hundreds of games on a single disc. Many creators were able to infiltrate store shelves this way.

But it was digital distribution that offered the most potential for the distribution of small games. Side-stepping the cost of printing media entirely, digital distribution promised to ship a game directly from computer to computer, from author to player. The BBS allowed for digital distribution, but was hampered by its bandwidth—stuffing data through phone lines, a BBS could only allow for small, slow downloads—and the isolation of BBS networks. One BBS wasn’t connected to another, and a user plugged in to one BBS would only have access to what was available on that BBS. This made widespread distribution more difficult and slow.

And so, for a long time, the digital distribution of games was scattershot. But eventually a network would coalesce that would resolve these problems.

I’m Referring to the Internet

Today the Internet is linked by cables, not phone lines. The Internet of BitTorrent gives us a model for file sharing that’s fast and decentralized. The Internet of 2012 is different from the BBS systems and early online networks of the eighties and nineties in a few important ways: there’s the

speed, yes, but more important is the access. The infrastructure of the Internet is different: a user doesn't dial in to an isolated part of it, but rather always has access to any part of it (government censorship aside). Which is to say that if I make a game, I can post it in one location (say, my website), and anyone connected to the Internet can visit that site and download that game.

Whether they can run that game is another question. But there's been a progression toward infrastructure not only in playing games but in running them. Take Flash, for example, an Internet plug-in originally designed, by Macromedia, to allow animators to insert movies into web pages so that visitors could watch them inside their web browsers. Almost immediately creators began to co-opt Flash in order to put playable games into web browsers—obviating the need to download a game before playing it. Look at Newgrounds.com, a Flash “portal” whose current slogan is, “Everything, by everyone.” *Newgrounds* (which began as a zine distributed by the thirteen-year-old Tom Fulp²⁸) began accepting visitor submissions in 1999. A decade later, *Newgrounds* hosts 170,000 Flash movies and games created by over 2.2 million registered users.²⁹ Plenty of those are cartoons about Super Mario, but consider how many creators have found audiences for their creations. *Newgrounds* has even found ways to earn money for its creators, by selling ads to interested companies and giving creators the option of including those ads in their movies and games. There was a time when I made my living almost exclusively by creating Flash games for *Newgrounds*.³⁰

So you can get an impression of how much potential digital distribution has to allow games to proliferate outside the industry. To physically publish games has always been difficult for authors without access to capital: that accounts for the rise of publishers. But the speed and interconnectedness of the contemporary Internet provide authors with a means to distribute their games to players without having to deal with the costs of physical publishing and the marketing these costs engender.

Publishers, incidentally, are aware of the Internet as well. Corporations like Valve, Apple, and Microsoft have set up online infrastructures (“Steam,” the “App Store,” and “XBox Live,” respectively) to sell games. Users buy games with a credit card, allowing them to digitally download

games to their computers. Small game creators have been able to ride the coattails of these online marketplaces, using them to sell and distribute their own creations. The danger is that these markets are maintained and regulated exclusively by the corporations who built them, corporations who will of course police them according to their own interests. Take for example, February 2010, when Apple deleted over 5,000 iPhone games from its digital store overnight for being, in Apple's judgment, too sexual.³¹

But digital distribution potentially means the most to the creators of free games—hobbyist game creators. There can be hobbyist game creators because distributing games no longer requires capital. An author can produce a game in her spare time, upload it to the Internet, and watch as an audience finds, downloads, and experiences it.

But what does she use to produce her game?

New Tools for Artisans

The first digital games were created by engineers in university computer labs. They alone had access to computers, and they alone had access to the technical information required to teach those machines to play games. But now personal computers inhabit homes—and, consequently, new game-creating tools have come into being for people who aren't engineers with technical knowledge.

I'm going to discuss many of these tools, and what each is good for, later in this book. But for now, I think a sample of source code might illustrate how far the tools of today have come from the Assembly code at the opening of the chapter. This is a sample of Inform 7 code. Inform is a tool for creating interactive fiction: text adventures. The newest version was created by Graham Nelson to allow authors to write "natural language" code—that is, lines of code that look like English sentences. Natural language code isn't necessarily the most efficient or effective way to write a game, but Inform 7 was made with the idea that an interactive story should be as easy to write as a prose story, and that if it was, more people would create games. The following code gives the player a bag of four candies, one of which is poisoned.³²

The plural of piece of candy is pieces of candy. A piece of candy is a kind of thing.

A piece of candy is always edible. Four pieces of candy are in the Halloween bag.

Toxicity is a kind of value. The toxicities are safe and poisonous. A piece of candy has a toxicity. A piece of candy is usually safe.

The Porch is a room. The player carries the Halloween bag.

After eating a poisonous piece of candy:

say “Oh, that didn’t taste right at all. Oh well!”

When play begins:

now a random piece of candy is poisonous.

Maybe you don’t follow the example totally, but it looks very different from the 1979 Assembly code. If the two biggest barriers to free game creation—and by *free*, here, I mean creation that’s universally accessible—have been the technical knowledge required to teach game logic to computers and the high cost of publishing physical copies of games, then at the time of this writing, both of those barriers have been breached.

Right now, we can imagine a future where creating a game is as easy as writing a story or drawing a picture. We can imagine videogames that are written, like *Newgrounds* suggests, “by everyone” for everyone, rather than by corporations for consumers or by technical wizards for stunned onlookers. This is our time, and games are ours to create.

So what are games good for?

Footnotes

²³ Contributed by Steve Harvey.

²⁴ See “Pixel Journeys: The Magic of dnd5,” *GameSetWatch*, December 19, 2008, http://www.gamesetwatch.com/2008/12/column_pixel_journeys_dnd.php.

- 25 Jeff Howard, "Magick Systems in Theory and Practice, Installment 2: Word and Gesture as Input Methods in Gaming History," *The Gameshelf*, June 30, 2010, <http://gameshelf.jmac.org/2010/06/magick-systems-in-theory-and-p.html>.
- 26 "1983 Releases," *Intellivision Lives*, http://www.intellivisionlives.com/bluesky/games/credits/1983c.html#kool_aid.
- 27 I host a download at <http://blog.dessgeega.com/?p=51>.
- 28 "Newgrounds Wiki: History," *Newgrounds*, <http://www.newgrounds.com/wiki/history>.
- 29 I emailed Tom Fulp for the 2.2 million figure.
- 30 Having done so, by the way, I can assure you that Flash is not an ideal tool for creating games. Since the program was designed as an animation player with interactivity included as an afterthought, it's an unwieldy and confusing game-making tool.
- 31 Craig Grannell, "Apple's Stance on 'Adult' Apps Is Indefensible," *TechRadar*, February 23, 2010, <http://www.techradar.com/news/phone-and-communications/mobile-phones/apple-stance-on-adult-apps-is-indefensible-672642>.
- 32 From *The Inform Recipe Book*, <http://inform7.com/learn/man/Rex62.html>.

Chapter Three

What is it Good For?

So, for the first time in the history of the videogame form, people who aren't programmers or corporations can easily make and distribute games.

But why would they want to? Why make a game—especially when there already exist the means to write stories, play songs, film yourself for YouTube? What can we do with games that we can't do with those forms?

To begin, let's define what a game is.

You've played games and you have assumptions about what they are. Maybe when you read *game* you imagine a videogame; maybe when you imagine a videogame you imagine a big-budget run-jump-shoot game. Maybe you imagine Tetris. Since I'm more interested in games, digital and otherwise, that don't resemble games that already exist, I think a fresh definition is in order. I also think it's worthwhile to have a definition that isn't specific to digital games, because I'm interested in the commonalities between digital and non-digital games, and in connecting videogames to that much older tradition.

So here's my definition:

A game is an experience created by rules.

That's pretty broad, huh? I'm interested in as inclusive a definition as possible, though you might argue that mine is too broad: for example, you can use it to describe getting stuck in a traffic jam or paying your taxes. A tax form is nothing but a series of rules you follow to produce a final number, after all. But is it useful to think about your taxes as a game?³³ Not really. Do the rules on a tax form really create a strong experience, or are they just a method for producing a number?

A game is an experience, and that experience has a certain character. Maybe a game is a story, or maybe it's the experience of control giving

way to panic giving way to relief. Maybe it's about taking something and making it grow bigger and bigger and bigger, or maybe it's about two rivals, equally matched, each trying to out-guess the other's plans. The experience that we identify as a game has character, and we can talk about what that experience is.

And if we're discussing an experience, then that implies someone is there to have that experience, someone we refer to as a *player*. We can't talk about a game without talking about the experience of the player playing that game, even if the playing experience we're talking about is often our own.

The experience we call a game is created by the interaction between different rules, but the rules themselves aren't the game, the interaction is! A game can't exist without a player or players: someone needs to be engaging with the rules for the experience to happen.

How does that work? Consider a game of Tag. Rules: One player is IT, and must tag as many of the other players as possible with a touch. Each of those other players is SAFE when she touches this gnarled-up oak tree. You can see the way the interaction between those two rules creates an interesting (and volatile) dynamic. The players who aren't IT want to reach the tree, but the player who is IT wants to stop them.

You can imagine a situation where the IT player is standing between two other players—one to her left, one to her right—and the SAFETY of the tree. Maybe one of them will make a break for the tree, maybe IT will be forced to pick one of the two to chase while the other gets to make a run at the tree, maybe a fourth player will take advantage of IT's distraction to make a run at the tree from behind. When we talk about a game of Tag, we're talking about this experience. But this situation (and it's a good, tense one) isn't explicitly defined anywhere in the rules. However, notice how these rules guide the creation of that situation. The rules set the players in opposition to each other, give most of the players a goal, and give the other player a reason to intervene, creating a tense dynamic.

What if we were to take either of these rules away: the SAFE location or the player who's IT? Without a SAFE location, players have no reason to stay nearby and interact with the other players, especially the IT player. The ideal strategy to avoid IT would be to go as far away as possible, and that breaks the tension and hence the experience of the game. What if

there was no IT player? Then it'd just be people running around, and while a bunch of people running around has value, it doesn't have the character or dynamic of a game.

But there's certainly room to change the details of the rules. Tag, being a folk game, has been played by many people in many places with many, many different versions of the rules. In one version, a player might be done once she's tagged the SAFE tree. As more and more players tag the tree and leave the game, the players who are less fast become greater and greater targets because the IT player can focus less on monitoring the tree and more on pursuing them.

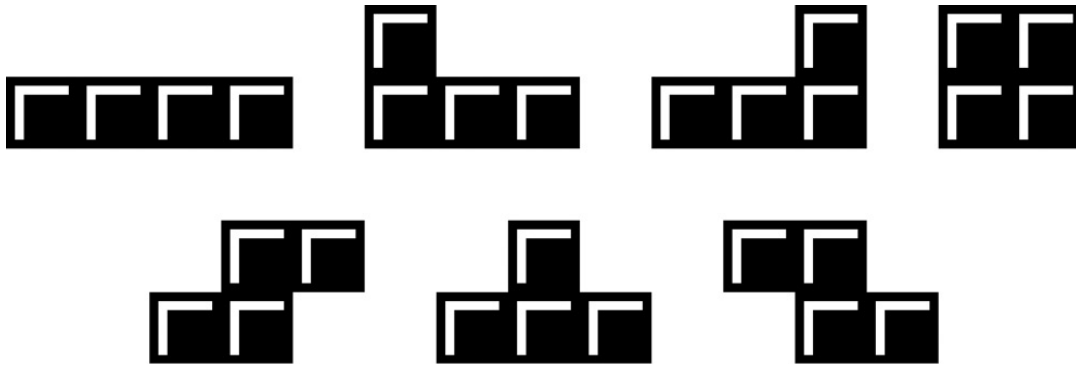
Alternately, what if a player who touches the tree isn't permanently safe—what if players are only allowed to be in contact with the tree for five minutes at a time? That keeps players vulnerable to IT and keeps the game from stagnating. Maybe a player who leaves the tree has temporary immunity to allow her to get safely out of IT's sight, or maybe it becomes a stand-off, where the escaping player has to wait for another player to distract IT's attention before she can make a break for it.

What about freeze tag? In this case, a player who's tagged by IT is “frozen” and has to wait for another player to come and “rescue” her before she can move again. This variation has much more direct interaction between the non-IT players. Instead of just depending on one another as decoys, they have to actively put themselves at risk to aid other players, which only adds to the tension of the game. And it creates a new dynamic between the non-IT players: I rescued you this time, but if I get tagged you're going to have to leave the tree and rescue me.

And that's what games are good at: exploring dynamics, relationships, and systems.

The Story of Tetris

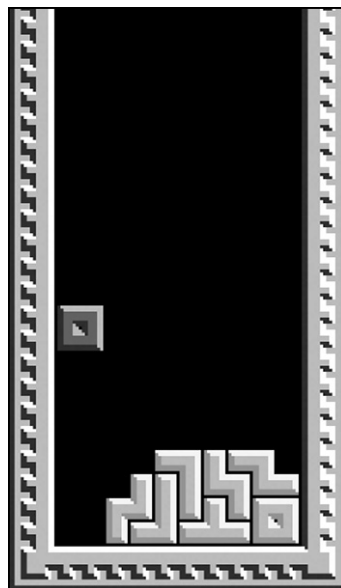
A “system” is what we'll call the interaction (or ongoing interactions) between a set of rules. Let's talk about Tetris now.

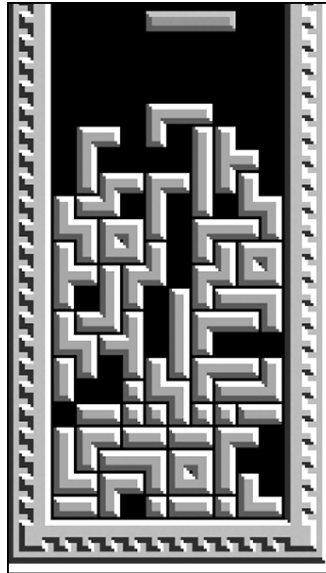


What are the rules of Tetris, essentially? The basic rules that drive Tetris are:

The game is played with *pieces*, comprised of every possible combination of four squares. (See the image above.)

- Pieces fall continuously into a well of a certain volume. The player can guide the pieces' fall to the left and right of the well, and also rotate the pieces both clockwise and counterclockwise.
- Pieces are removed from the well when and only when the player organizes them into complete rows.
- If there is no room left in the well for a new piece to fall, the player loses.^{[34](#)}





You can see how these rules create a system where the player's mistakes compound on one another to cause further mistakes: Only full rows are eliminated, so incomplete rows stick around and take up space in the well. Clutter in the well then makes it more difficult to position other pieces and to create rows. As the row fills with mistakes, it eventually becomes impossible to fit more pieces, and the game ends.

These rules function in tandem to give the game a momentum and shape: the player makes errors that cause further errors, until eventually the player is overcome. (And consider how well a commonly added rule, "the pieces fall faster every time ten lines are made," works with these basic rules to help the game escalate.) We could consider this a system.

All games aren't necessarily simulations of existing systems: it would be difficult to imagine a situation in the world that actually resembled Tetris. But it's easy to imagine simulations that model systems of rules that are far less abstract: urban planning, politics, oil drilling. And there are games whose rules mimic such systems. Will Wright's *SimCity* is a game in which the player plans a city, Jim Gasperini's *Hidden Agenda* is a game in which the player governs a post-revolutionary South American nation. Arch D. Robison's *Seismic Duck*³⁵ models the way drillers use aimed sound waves and seismogram to find oil reservoirs.

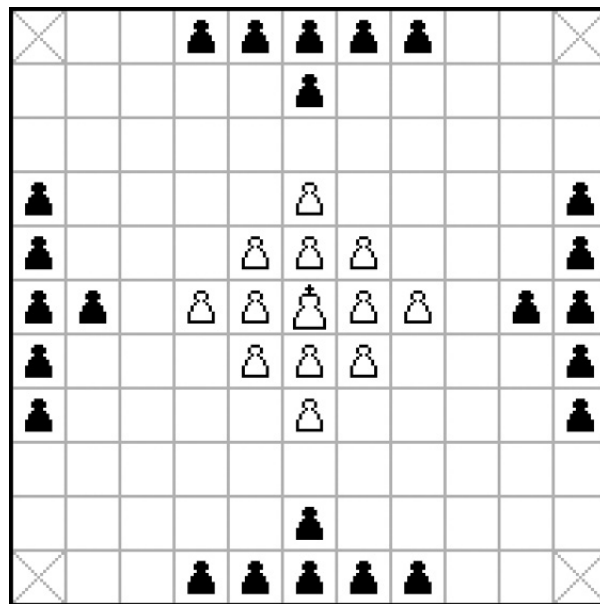
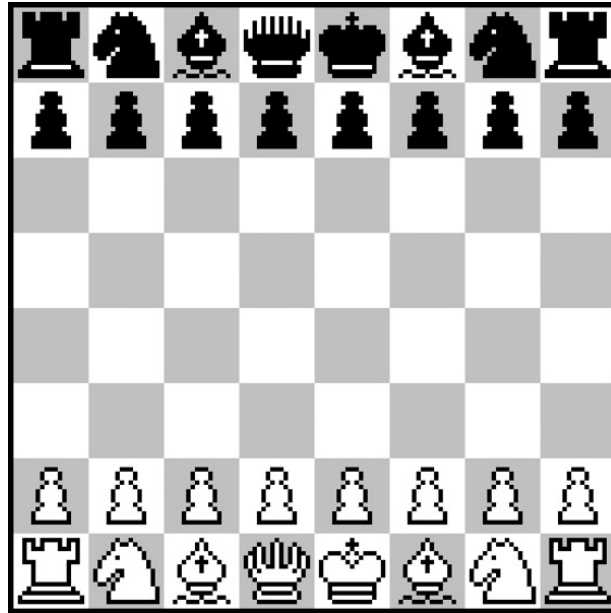
You can begin to see how systems can be translated into game rules: a commercial zone in *SimCity*, for example, needs people to act both as a work force and as consumers. That means the people need homes to live

in, transportation to get them around the city, power to make sure the lights are on. The system teaches concepts about the interdependency of urban forces. To again cite Greg Costikyan's "Maverick Award Speech": "I want you to imagine a world in which the common person is no longer ignorant of economics, physics and the functioning of the environment—things which are themselves interactive systems—because they have interacted with them in the form of games."

Every game of Tetris has the same shape—errors compound on errors until the well is filled and the player is overcome—because the system of rules we've discussed guides the experience in that direction. But the player places all the pieces herself. Every player will place the pieces differently, will play a different game, but experience a similar result. The same holds true for any system of rules, as simple as Tag or Tetris or as complicated as *SimCity*. Games have a lot of potential for examining the relationships between things—or, rather, for allowing the player to examine the relationships between things, because the player does not merely observe the interactions; she herself engages with the game's systems.

The Rise of the Designer

Tag is an example of a folk game, along with Go, Chess, Poker, Stickball, Hide and Seek, and most of the world's oldest games. Games have been around as long as civilization has; the game is by no means a new form or a recent invention. What is relatively recent is the shift from folk to authored games. Folk games, like folk songs and folk texts such as the Bible, have no single credited author, but rather many untraceable authors over many years. They're artifacts shaped by entire cultures, and generally they can tell us a lot about those cultures.



For example, compare Chess, a continental European board game of warfare, with Hnefatafl, a Viking board game of warfare. Chess is a game of combat between kings with equal resources. Each player has the same pieces and starts in the same position on opposite sides of the game board. Each player's goal is to capture the other player's king. In Hnefatafl, one player represents a king and his defenders, who start in the center of the game board. The other player represents the attackers, who surround the king's forces on all sides of the board. The king player's goal is to get the

king through the attacking hordes to safety, while the other player's goal is to surround and capture the king. The differences between these games' interpretations of combat tell us a lot about the differences between strategic thought between European vassal kings and Viking warrior bands: their priorities, the nature of their battles, and whether they approach warfare as a platonic war between equals. And the games themselves, in turn, shape the strategic thought of those who play them.

Our history is full of folk board games. Authored board games—games created by a single person or small group, and whose authors can be identified—are a more recent phenomenon. For example, I can tell you that the board game *Cosmic Encounter* was designed in 1977 by Bill Eberle, Bill Norton, Jack Kittredge, and Peter Olotka of Eon Games. (We can date *Cribbage*, by Sir John Suckling, to the 1630s.) These are games as texts of specific rules, rather than as patterns of rules that are subject to change through mimicry. A game of Tag will always have a chasing player and a safe position, but the actual rules will change from play to play. The majority of contemporary board games are designed by a single author or team, and the same is true of digital games.

Can there be folk videogames? Videogames retain credits better than board, card, and physical games. I think that there are digital games, though, that exist as patterns of similar rules, perpetuated through duplication with small mutations. There are a thousand different versions of Tetris, for example, each coded by one of a thousand different authors, and each version with a slightly different set of rules, a slightly different set of numbers, and often (to avoid litigation) a different name. There's a digital game that's commonly known as "the snake game," which began as an arcade game called *Nibbler* by Joseph Ulowetz and John Jaugilas. In this game, the player directs a snake to gobble pieces of food. The snake dies whenever it crashes into either a wall or its own body by coiling around itself. Each piece of food causes the snake's tail to grow longer, making it take up more space and making it more difficult for the player to avoid collisions with her own body. So many different authors have remade this game on so many different machines that all of its forms and variants are usually just referred to as "the snake game." Is this how authored games become folk games?

But what can authored games tell us that's different from folk games? Folk games tell us about the culture that created them; authored games tell us about the author that created them. Authored games have the potential to be more personal, and thus more specific and diverse, than folk games. Two plays of an authored game are likely to be more similar than two plays of a folk game, because the authored game retains the rules set created by its original designer. It's the fact that folk games change with each player that makes them so long-lived, that makes them adapt to suit the culture that adopts them. But in this book, it's authored games, and the diverse set of voices they embody, that I want to focus on.

What's Video Good For?

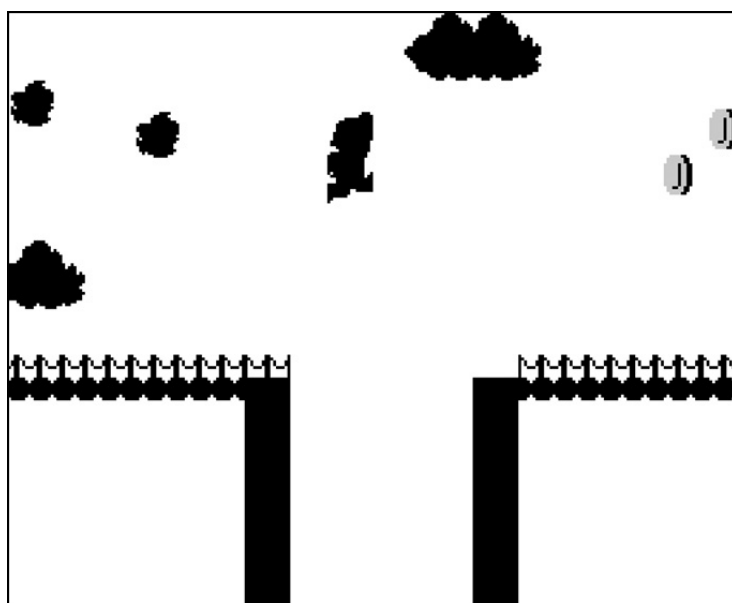
In a board game, players have to track how much money is left in the bank, which pieces are in play, how high the water level rises. A deck of cards can keep players from knowing in what order pieces will come into play, dice can generate random outcomes to situations, and players have hands of cards that represent information they keep from the other players, but beyond these basic devices, little information can be hidden from the players, because the players must make sure the rules are being observed by tracking most of the information themselves.

In digital games, the computer keeps the rules. The computer tracks all the numbers. Digital games therefore have much greater control over what information the players have access to, making videogames capable of much greater ambiguity than board or card games.

What's ambiguity good for? Telling stories! Digital games have great potential for storytelling. The author has a lot of control over the pace at which information is revealed; therefore the author can pace the telling of a story. This is not to say that videogame stories are being told as well as they could be. But the format of a videogame—which lets rules be changed and introduced over the course of the experience, and which lets the author hide the causes for events and show only the effects—lends itself more easily to an overt, sustained narrative than any physical game format.

Because the rules are kept by the machine, the rules in digital games tend to be more numerous and more subtle. Think of a game like Shigeru

Miyamoto and Takashi Tezuka's *Super Mario Bros.* Unless you've studied the game in great detail on a technical level, you probably don't know exactly how high Mario can jump relative to the height of the screen, or how fast he accelerates horizontally when he runs. The interactions between these hidden rules in videogames can result in very complex systems without necessarily complicating the game, because the player isn't required to track and compare all the numbers. For example, imagine the designer creating a situation where there's a tiny platform with a long pit on either side. Mario has to run to build up the momentum to clear the pit and land on the platform, but instead of stopping there he needs to immediately jump again in order to make the second pit without losing the momentum that will let him cross it. This is a problem that wouldn't be obvious to someone who had just approached the game.



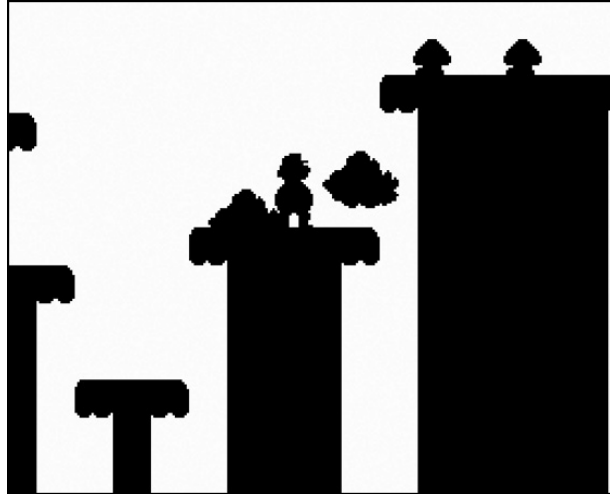
Through playing the game, the player develops a sense of the limits and subtleties of these hidden rules. This interaction between the player and the game, dependent on the game's hiding information, gives digital games their special capacity for subtlety and nuance. You could compare it to the use of "English" in a physical sport: the difference between hitting a ball and hitting it with a particular force, and in a particular direction.

Because of this capacity, videogames are often performative: they allow the player room to interact with rich and complex systems with grace and

finesse. We usually refer to this as “skill.” A system may persist through an entire game, but the game may start very permissive of less graceful playing and require the player to play with more and more finesse as the game goes on. The game gets HARDER, asking that the player become more skillful, but allowing her to learn the game’s systems over the course of navigating increasingly difficult situations.

The systems that the player manipulates in *Super Mario Bros.* are introduced very early in the game, with the only added rules coming with the periodic introduction of new enemy characters or hazards. But the situations that Mario has to navigate start fairly relaxed and demand more and more skillful playing as the game progresses. In the first stage of the game, obstacles are low enough that a simple jump from a standing position will allow Mario to clear them. In later stages, the height of obstacles will require Mario to run and build momentum before jumping, in order to jump higher. In this way the designer teaches the player the subtleties of the game’s complex system through careful use of machine-controlled variables. Digital games are thus good at teaching, and at communicating a sense of the player’s progress, which often parallels the progress of the protagonist and the development of a story.

What else is handy for telling a story? The ability to generate or play video and audio, either as accompaniments or as central vehicles for information. Digital games can incorporate a variety of media when telling their stories. Consider how the music in *Super Mario Bros.* speeds up when there’s only a hundred ticks left on the time limit to complete a stage, creating a sense of urgency, or how the sound played when Mario jumps on an enemy gets higher and higher pitched, indicating that a reward—in this case, an extra life—will come if the player keeps doing what she’s doing. Consider how the player’s journey takes her through a changing visual landscape, from a sunlit field to a black-and-blue underground, to treetops, to the mushroom forest, and to Bowser’s castle, and the way each of these sights—withheld from the player until her skill develops to give her access to later areas—provides a sense of progression through the Mushroom Kingdom.



I don't mean to imply that non-digital games are incapable of the things I've described, or that digital games are in some absolute sense better or more worthy of interest. There are many different kinds of games, all of them suited to different things. Digital games, because of their ability to withhold and pace the player's access to information, because of the strict narrative control the author is able to have over the player's experience (because the machine enforces the rules), and because of their capacity for generating a wide variety of sights and sounds to enhance or even define the playing-out of the rules, are particularly well suited for the telling of stories. And the telling of stories—games becoming more personal—is what especially interests me about games as a form.

Role-Playing Games

Digital games have certain strengths for telling stories, but the nature of games in general—even without the advantages the machine provides—makes them good for storytelling. And I do think, in the way that film and photography have generally changed the focus of novels and visual art, the mechanical rules keeper that videogames provide has caused similar focus changes in other aspects of games. In the last chapter, I couldn't discuss the earliest mainframe computer games without mentioning role-playing games like *Dungeons & Dragons*. Role-playing games came out of the tradition of "miniatures wargaming," a set of rules for moving armies of dolls around a tabletop battlefield and pitting them against one another in combat. Later role-playing games kept the rules for combat and situation

resolution, gave players the responsibility for a single combatant rather than a larger number of soldiers, and largely got rid of the dolls.³⁶ But the important thing that games like *Dungeons & Dragons* introduced was the concept of a “Dungeon Master,” or “Game Master.” This is a player who manages the game for the other players, laying out the scenario and directing the world’s responses to the players’ actions. The Game Master essentially inhabits the role of storyteller, preparing and guiding the players through a story in which they make decisions.

This aspect of role-playing games—an overseer who negotiates the player’s choices using a set of rules—was eagerly adopted by many of the first digital game authors. Early digital games like *Rogue*, a graphical game of maze exploration and combat, lifted its rules and probability calculations directly from *Dungeons & Dragons*. And text adventure games like *Zork* took the idea of a narrator who relates the world of the game to the player using a consistent voice. Many early digital games are, conceptually, role-playing games in which the computer takes on the role of Game Master.

The computer’s adoption of the responsibilities of rules keeping and number counting has shifted the focus of many tabletop games away from their original focus—providing players with an extremely finely grained simulation of combat and other adventuring situations. Why spend thirty minutes rolling dice and looking up random treasure and critical hit information from lengthy tables when you can play a computer game that resolves everything, with far greater mathematical complexity, in seconds and in color? Instead, tabletop role-playing games have been able to move away from a mathematically dense combat situation and toward collaborative storytelling and improvisation.

Take, for example, Paul Czege’s *My Life with Master*, in which the players invent a Victor Frankenstein-style mad scientist under whom they will serve as Igors. The Game Master, as the titular Master, assigns the servants duties that enable the players to act out the conflict between their duty to obey and their desire to reclaim their humanity. The dice rolls aren’t to test the player’s ability to penetrate leather armor with her sword, but rather to test the player’s “love” versus her “self-loathing”: if a player “fails” her roll, her character must either perform the duty assigned her or

refuse to perform it, regardless of which outcome the player might prefer, a simulation of the state of being in emotional servitude. In the end, one of the servants will rise up and destroy the Master, a result that the rules of the game make inevitable: the destruction of the Master is the climax of the story, which each player finishes in turn by creating an epilogue for her character.

What interests me about *My Life with Master* isn't just its use of rules as a unique device for telling a story of personal, internal conflict, rather than as a means of resolving physics simulations in a fantasy world. I also find Czege's distribution method interesting. *My Life with Master* is sold on Czege's website as both a book, sent in the mail, and as a downloadable PDF file that the buyer may print herself if she's interested. Digital distribution! In the past, the rules for tabletop role-playing games were so elaborate that they required hardbound books, distributed through traditional bookstores and novelty stores, which is still the method used for distributing recent editions of *Dungeons & Dragons*. But authors of small, more experimental role-playing games like *My Life with Master* are avoiding or mitigating the costs of publishing and distributing by selling their rules online as digital downloads, or in some cases simply posting them for free on the Internet.

Role-playing zinesters! And ones who, through their change of focus from complicated and expensive rule books full of encounter tables to simple rules that create conflicts and guide the players in creating a story, offer useful lessons that the designers of digital games could stand to learn.

Grown-Up Games

Games are useful, I wrote earlier, for exploring and teaching about dynamics and relationships. *Gang Rape* is a role-playing game made by Tobias Wrigstad in 2007. Outraged by how many rape cases the courts in his native Sweden dismiss without charges, Wrigstad wrote a game that he hoped would allow players to explore and talk about the experience, and the horror, of rape.

Most games are designed to be pleasing and stimulating to play in an immediately rewarding way: they're intended to be fun. This game is not

like those games. In fact, the rules—which are only available by directly and personally requesting them from the author³⁷—open with the sentence: “A scenario about gang rape is not meant to be fun to play.” The game is intended to be harrowing; its goal is to give players some respect for the severity of its subject.

A game isn’t defined by being fun just as comics aren’t defined by being funny. A game is defined as an experience created by rules. Wrigstad’s *Gang Rape* is like any other game in this regard.

In the traditional role-playing games we’ve discussed, one player takes the role of Game Master. That player then guides the other players’ experiences by telling another player, for example, what things her character “sees.” *Gang Rape* has no Game Master—one player is a victim, the others are her attackers—but it gives each player limited, Game Master-like control over the characters the other players are playing. Specifically, the rapists can tell the victim how her body reacts to their actions, but not how she feels about those actions. The victim can tell the rapists how they feel while they perform those actions. Rape is about control: these rules are designed to give the players an impression of the power a rapist has over the person being raped. There are additional rules that allow the rapist players to dictate each other’s behavior, and to allow the players to explore the role of peer pressure and “egging on” in the dynamic of a gang rape. All the rules are clearly crafted to create a sense of the dynamics at work in a situation where two or more people have power over another, and to give the players the liberty to explore and better understand those dynamics. (Though the scenario the author had in mind is clearly a woman being raped by two or more men, he admits that the characters involved—and their players—can be of any gender, and that the rules can apply to a scenario like bullying and mobbing instead of sexual assault.)

This game (which, again, is distributed exclusively digitally, and with the special limitation that anyone wishing to play it must identify herself directly to the author) is an example of using the capabilities of games—experiences created by rules—not to indulge an escapist fantasy but rather the direct opposite: to try to educate players about the dynamics at work within a horrible real-life experience, and how those dynamics might

come to be as a product of individual choices and responses. The players narrate, through their characters, the events prior to the assault, the events of the assault, and the aftermath.

Games can be topical; they can be relevant to our lives as human beings. They can be relevant without having to be about rape. But *Gang Rape* is one example of what we gain when people other than commercial publishers author, publish, and distribute games that commercial publishers could never touch.

The World's a Stage and We are Players

Often, games—particularly digital games, with their use of video and audio—are compared to film, probably because the videogame publishing industry strongly resembles the Hollywood studio system. But I don't think this comparison is particularly constructive, in that it gives us little insight into what the game, as a form, is capable of. Film tells a static story; what's exciting about the game is that it allows the audience to interact with a set of rules. This doesn't mean the game can't tell a story: in the role-playing genre, the players aren't merely watching a story but playing the roles of the characters within the story.

A better comparison than film is theater, which is where a lot of our game vocabulary (“the player,” “stages,” “set pieces,” “scripting”) comes from. A play defines the roles, events, and scenes of a story. An individual performance of those roles and scenes will always be different: different actors will perform the same role in different ways. Every performance and interpretation of a particular play is different—sometimes in minute ways, sometimes in radical ways—but we consider the play itself and the scene itself to be the same.

Compare this to a game story, particularly a videogame story. Every player will perform the story called *Super Mario Bros.* differently (and the same player will perform the story differently each play), but the role of Mario and the actions Mario is capable of taking remain the same. There is always a scene called “World 1-2,” although each performance of “World 1-2” will be different. In a more contemporary videogame such as *Half-Life 2*, a very clearly cinema-inspired game, each player will always pass through the events the designers have scripted in the order in which

they are presented, but each player's (and each play's) performance of Gordon Freeman, the game's protagonist, will be at least subtly different. The player will always get chased across the rooftops by cops, but in one performance she might hesitate, unsure of where to go, in one she might head straight for the escape route, in one she might panic, almost getting Gordon Freeman killed, and in another she might walk a little too close to the edge of the roof, fall, and have to start the scene over.

As game storytellers, we are not directing static stories take-by-take but rather arranging the scenes that will comprise the shape of our story. We can begin to think of the player as someone performing a role we've written rather than as an audience who experiences our story without any input as to its outcome. We allow room for improvisation, room for the player to make a role her own. The audience of a game can be more usefully compared to the audience for a play than the audience in the movie theater. In videogames, the audience is there, live, with the actors—or as the actors—experiencing a single performance that is unique, despite the story having been performed and continuing to be performed many times.

Some players record videos of their performances, either for documentation or for the purpose of recording a specific achievement, such as reaching the game's conclusion as quickly as possible—what is usually called a “speed run” (YouTube has given lots of these videos a means of reaching an audience). That there's an incentive to capture individual performances of a game testifies to the amount of variance there is within a game depending on who's playing it.

Games and Chance

The board and card game traditions have also given a lot to digital games. What I think digital games have taken the most from board games and card games is the way they manage chance. Both contemporary designed games and older folk games have invented many systems for managing chance. The six-sided die, for example, allows for the random selection of six equally likely outcomes (and can then be further used to access other percentages and ratios; for example, three outcomes, each represented by

two sides of the die, or eleven outcomes with different likelihoods represented by two rolls of the die, and so on.)

Card games themselves are designed as a system for managing chance and gradually revealing information. When all cards are in the deck, every card in the game has (as far as the player knows) an equal chance of being in any position. Once a card has come into play and been seen by the players, though, the players then know where it is and can use that information to make guesses about the remaining cards. Cards also allow players to manage the pace at which they reveal information: a player might have a hand of seven cards hidden from the other players, who don't know whether those cards have come into the game yet or not. Poker is a classic game of using limited knowledge of the cards in play to predict the positions of cards not yet in play. This is what makes Poker an elaborate game of bluffing. One player tries to see through the other's "Poker face" because the decisions she'll make are based on what she can predict about the information the other player is concealing. Contemporary game designers have contrived even more rules to control the revelation of information.

Aside from hiding information, chance is frequently used to break symmetry. Having different starting conditions between players prevents both players from having the same set of ideal moves, and thus having the game become a stalemate. Having different, randomly selected values between one play and another, or having different game events happen at different, impossible-to-predict times (or not at all), means that each game will demand a different strategy, keeping play from becoming stagnant.

Franz-Benno Delonge's and Thomas Ewert's board game *Container*, for example—a game where players trade and transport commodities—uses chance to ensure that all players do not value the commodities identically. At the start of the game, a number of cards are shuffled and randomly distributed, one to a player. These cards describe how valuable the different commodities are to the players who hold them, and each card values the commodities differently. The cards are also kept hidden until the end of the game, each card seen only by the player that holds it. Because each player is aware of the entire possible set of values on the cards—she knows which cards are in the game, and which card she, and therefore not the others, possesses—she can watch the other players'

decisions and make deductions about which players have which cards, and therefore which commodities are valuable to which players.

Computers have an innate capacity for manipulating chance. Though true randomness doesn't exist, computers handle numbers easily and are capable of generating reasonably unpredictable probabilities of any size on the fly. Every computer has access to an infinite number of monkeys rolling an infinite number of dice.

Why is this useful? Because, as we've discussed, games have a unique capacity for improvisation! Though each scene has the same shape—Link battles a gang of Moblins—each performance is different. So what if, in one performance, one of the Moblins comes from the left instead of the right? Digital games have the capacity to create variations on many subtle details in every play, keeping the experience from becoming stagnant.

The differences don't have to be subtle, either. In Chris Klimas and Joel Haddock's online game, *Where We Remain*,³⁸ for example, the player is a boy searching for a girl on an island patrolled by monsters that are intended to evoke characters from Greek mythology. The layout of the island—what tools are hidden in which caves, what areas which monsters patrol, and in which cave the girl is hidden—is different every time, decided by a random number generator. In effect, this randomness makes the characters and events of the game more archetypal because the emphasis is on the shape of the game—the boy's search for the girl while monsters pursue him—rather than on the details like what treasure is hidden where. Games have lots of room for improvisation, for every play of a game or scene to be unique, and digital games in particular have easy access to a great degree of chance.

Games as Culture

I keep bringing up the profound influence role-playing games like *Dungeons & Dragons* have had on digital games. Both Eastern and Western videogame trends have their roots in *Dungeons & Dragons*, but both experienced and responded to that influence in different ways, much as Chess and Hnefatafl reflect different experiences of and responses to warfare.

Character creation is an important feature of role-playing games: the players literally describe the role they intend to play, both in narrative terms (what is this character's background and personality?) and in mathematical ones (how many times can this character be hit by dragon breath before she collapses?).

The American game *Wizardry*, created in 1981 by Andrew Greenberg and Robert Woodhead, was an attempt to bring the dungeon exploring and monster battling of *Dungeons & Dragons* to the computer. (It has plenty of peers with similar intentions and similar properties, but it's a good example for discussion.) In *Wizardry*, the player controls a team of up to six adventurers, similar to a team of player characters in *D&D*. Each of these adventurers is, fundamentally, a set of statistics (Strength, I.Q., Piety, Vitality, Agility, Luck, Age), a Class (the character's job or specialization), and a list of Spells (tactical magical abilities characters can use to help with combat or exploration). Character creation is held over from tabletop role-playing games, but in *Wizardry* it loses its narrative dimension. Players "roll" statistics (as with dice) and assign character classes and ability sets based on those random "stats," and that is the extent of characterization.

Character creation is present in many Western digital role-playing games, though implemented in different ways. The *Ultima* games, which I've mentioned before, ask the player a series of moral choices that determine which character class she plays as. Games like Michael Toy and Glenn Wichman's *Rogue*, which borrows its combat rules directly from *D&D*, skip character creation but assign the player's character no properties beyond this set of stats.

Yuji Horii's *Dragon Quest*³⁹ is similar to early Western digital role-playing games: character creation is limited to entering a name, which also determines your stats via a hidden algorithm. *Dragon Quest* was widely imitated in Japan, in addition to receiving many of its own sequels, but over time Japanese games broke with the mathematical focus of character creation in Western games. The trend Japanese digital role-playing games tend to follow is to have the player character be increasingly designed by the game designer. In the *Final Fantasy* series, characters wear outfits designed by the authors, answer to names chosen

by the authors (though the player is sometimes given the choice of changing these names), and speak scripted dialogue that the player has little say in.

Compare a 2002 Japanese game like *Kingdom Hearts*, a collaboration between the authors of *Final Fantasy* and the Disney corporation, to a 2008 American game like *Fallout 3*, produced by Bethesda Game Studios. In *Fallout 3*, the player not only names her character, but also designs her face and appearance and decides on her race and gender. In *Kingdom Hearts*, the player plays a character named Sora, who is given clear motivations by the writers and dialogue by voice actor Haley Joel Osment and whose name cannot be changed. Sora's appearance in *Kingdom Hearts* is mostly static: he changes appearance to fit some of the worlds he visits, but the only part of him the player is allowed to change is what his weapon looks like when he's swinging it. His outfits, designed by the game's art staff, are law, as is his race and gender: white and male.

Why has character creation remained such a fixture of American interpretations of digital role-playing games while Japanese role-playing games have phased it out? It could possibly reflect that America is a young country, and a nation that has been capitalist almost since inception. American culture sells the idea of individuality and ego. In Japan, a much older country in which social roles are valued (and connected to uniforms), role-playing might more easily mean playing the role to which you've been assigned. (In Yuji Horii's *Dragon Quest*, the protagonist's sole characterization is that his ancestor is a hero.) There's an ongoing dialogue between Eastern and Western design these days, so none of these trends are exclusive (and they've never ruled all of design, obviously), but there are clear patterns in games that we can trace to the values of the people who created them.

Games tell stories that communicate the values of their creators in a unique way: not just through their explicit content but through the logic of their design, and the systems they choose to model. And if games communicate the values of their creators in a unique way, then it's absolutely essential that there be more creators passing on more values, more perspectives. Games must become more personal.

Footnotes

- [33](#) Sometimes it is: consider the contemporary interest in game theory among non-game designers.
- [34](#) This list obviously omits some rules, like how the game selects the pieces to bring into play next, and at what speed the pieces will fall. Digital games tend to have many more of these very specific rules.
- [35](http://home.comcast.net/~arch.robison/seismic_duck.html) http://home.comcast.net/~arch.robison/seismic_duck.html.
- [36](#) It's worth mentioning, by the way, that labeling traditions of games the predecessors of digital games is not to imply that these traditions are obsolete and gone. I know people with armies of tiny soldiers.
- [37](http://jeepen.org/games/gr) You can contact the author to request a copy at <http://jeepen.org/games/gr>.
- [38](http://twofoldsecret.com/games/whereweremain) Playable online at <http://twofoldsecret.com/games/whereweremain>.
- [39](#) The *Dragon Quest* games appeared in America under the name *Dragon Warrior* until recently, for legal reasons: Horii borrowed the name *Dragon Quest* from a specific *Dungeons & Dragons* supplement.

Chapter Four

Changing the Game

Creating a game from scratch takes time, effort, and tools—tools that haven't started to become widely available until recently. Digital games are capable of presenting video and audio, and creating a game often requires that music be composed, sounds be recorded, art be drawn, or characters be animated. That involves a variety of tools and training and a lot of work. Often it requires a team of people, each with a different specialty, to accomplish a particular design.

And while plenty of hobbyist authors work on that level, the truth is that most digital games are not made from scratch. Most tell their stories by borrowing pictures or sounds or music or code. They use kits that other people have made; they use sound effects that other people have recorded and released for their use.

And many, many game creators, budding or otherwise, piggyback on existing works, taking advantage of existing infrastructure to make their creations. A commercial game from a large studio, for example, requires thousands of 3-D models, sounds, and assets to be loaded onto the player's computer. Even more important, a commercial game comes with game logic: a means of resolving collision, of determining the effects of gravity, of moving objects around a game world. These things are difficult for someone inexperienced to code, but here a group of paid professionals have already done all the work. The player who's bought this game has all of those resources on her computer: why not use them?

In this chapter we're going to talk about what are usually called "hacks" and "mods" (for "modifications"): changing existing games to create new stories. There are a variety of ways in which people change games, and a variety of motivations for doing so. Sometimes changes are akin to crude vandalism, such as redrawing Super Mario so he has a dangling naked cock that flops around as the player walks through the Mushroom Kingdom. Sometimes they're akin to clever, subversive vandalism, such as changing the player character into the Princess and the prisoner who waits at the end into the Plumber. And sometimes they're something

entirely new, almost indistinguishable from the games that gave birth to them.

Sometimes authors want to situate their creations in the context of a particular work. For example, Jesse Petrilla created *Quest for Al-Qa'eda: The Hunt for Bin Laden* in response to the September 11, 2001, attack on the World Trade Center, which the American media attributed to Osama Bin Laden. This game is a modification of *Duke Nukem 3D*, a first-person game about ultraviolent American culture in which the stripper-tipping, alien-gunning protagonist appears to the player only as two attributes: a gun that hovers in front of the camera and a series of prerecorded movie quips. The jockish American attitude of military vigilantism totally fits the play of *Duke Nukem*, a game in which the most logical response to everything the player sees is either to shoot it or to stuff bills between its tits. It makes sense that *Quest for Al-Qa'eda* and Petrilla's 2003 sequel, *Quest for Saddam*, should take the structure of that game.

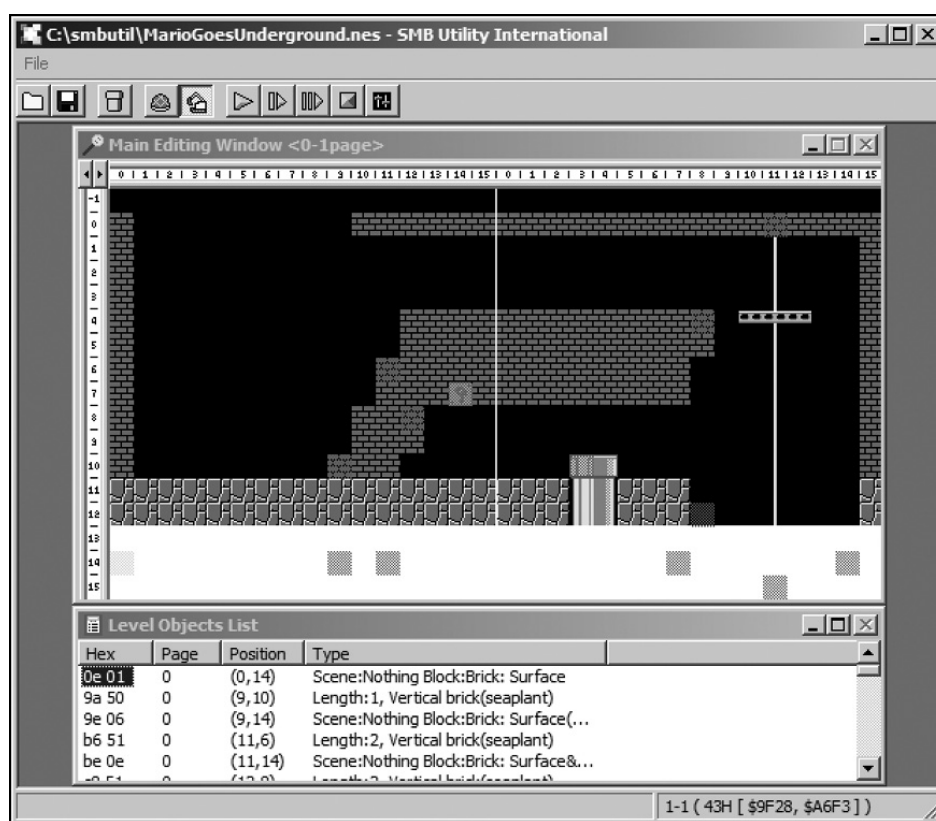
It further makes sense that the Global Islamic Media Front should, in 2006, reinvent *Quest for Saddam* as *Quest for Bush* (or *The Night of Bush Capturing*), a modification of *Quest for Saddam* in which the Saddam Husseins the player ceaselessly guns down have been replaced with George W. Bushes, an icon of American militarism. They're structurally the same game, with the faces swapped and the pictures on the wall changed in the simplest possible inversion of the original game's xenophobic aggression—a blunt “how do *you* like it?”⁴⁰ Hacks and modifications are made to subvert or comment on the original author's intentions, or to simply correct what the modder feels is an oversight on the author's part, or simply because it's an easy existing infrastructure for creating something new. Let's examine the varied motivations for hacking a game by looking at the most-hacked digital game: *Super Mario Bros.*

Another Castle

Super Mario Bros. was published on a cartridge that plugs into the Nintendo Entertainment System. Hardware has existed for a while, however, that allows the contents of a Nintendo cartridge to be “dumped” to a computer and distributed digitally as software. The digital file is called a “ROM,” and “emulator” programs have been written for many

different machines that will run that game software as a Nintendo Entertainment System would. In this way, *Super Mario Bros.* has made the transition from hard media to a purely digital form, available for modification by anyone with the knowledge and tools.

And tools have been readily available for a long time. *Super Mario Bros.* was written in Assembly, the machine language I mentioned in chapter 2, and reprogramming the game requires working knowledge of Assembly. But programmers cracked the code a long time ago, and plenty of tools currently exist for changing the appearance, rules and levels of *Super Mario Bros.* without having to look at machine code. When I made my 2008 *Super Mario Bros.* hack, *Mario Goes Underground*, I used a tool called SMB Utility⁴¹ that made it easy to rearrange *Super Mario Bros.*'s maps.



Super Mario Bros. has enjoyed popularity with ROM hackers for a variety of reasons. For one, the large number of people who've played the game made it an early target for programmers to crack, and tools have existed for a while. It's also widely played enough that most people have a

sense for how the game works: move left, move right, jump, catch a mushroom to grow big. Most players have an idea of what to expect from *Super Mario Bros.*, so they can pick up a hack easily and be surprised effectively when the hack turns out to be different from the original.

Some hacks simply change the game in a superficial way. *Silhouette Bros.*, by Leon Arnott, changes all the characters and architecture to stark black silhouettes on a solid-color background. The hack demonstrates how recognizable the elements of *Super Mario Bros.* are and the choices of color are well suited to the tones of different parts of the game. *Enigmario* by Dr. Floppy replaces the soundtrack of the game with covers of songs by the band Enigma. The somber tones of this new soundtrack give *Super Mario Bros.* a more introspective feel.

Mario vs. Airman is an interesting attempt at intertextuality: the “Airman” Mario fights is a villain from Capcom’s *Mega Man 2*, and Mario is required to navigate a level from this game to defeat Airman. Mega Man, as a player character, moves very differently than Mario. Momentum does not affect his motion, he can “brake” instantly, and his arm gun makes him able to shoot his opponents, rather than requiring him to jump on them to defeat them. Trying to navigate a level designed for Mega Man as Mario is a challenge that emphasizes the importance of the nuances of motion in platform game design.

A hack like *Super Mario Forever* exists to challenge the player’s technical knowledge of *Super Mario Bros.*’s rules. Mario starts mid-fall into a bottomless pit, and must instantly maneuver to land on a tiny platform. The game then requires the player to generate enough momentum on that tiny platform to make a long jump to the next, which involves jumping off of and back onto the platform in order to build speed.

Some hacks change a single rule of a familiar game in order to give the game a new experience, similar to the concept of “house rules” in non-digital games. Normally, the player of *Super Mario Bros.* must complete each stage within a limited time, which resets at the stage’s end. Nanashi’s *900 Time Challenge* gives the player a single time limit in which to complete the entire game: nine hundred ticks of the game’s internal clock. Completing the challenge requires the player to minimize the wasted time in her play, and to exploit shortcuts and warp zones in order to get through the game in as fast a time as possible.

Then there are hacks like *Extra Mario Bros.* (or my own hack, *Mario Goes Underground*), which attempt to tell an entirely new story using the familiar framework of *Super Mario Bros.* Rather than requiring Mario to proceed from the starting point on the left to the castle on the right, *Extra Mario Bros.* contains a big world for the player to explore in multiple directions, using pipes to travel vertically from area to area. It's essentially *Super Mario Bros.* through the lens of contemporary game design, which is why it's interesting as a direct modification of *Super Mario Bros.*: the games that informed *Extra Mario Bros.* are all descended from *Super Mario Bros.*

Then there's *Super Daisy Land*, which changes the star of the Nintendo Game Boy game *Super Mario Land* from Mario to Daisy, a captured princess who normally waits at the end of the game for rescue. But that's not exactly a hack of *Super Mario Bros.*, and there's a richer subject for gender correction in game mods.

Bobs Whose First Name is Betty



Anne-Marie Schleiner, artist and writer, maintains *Mutation*

.fem, a gallery of modifications of first-person shooting games: games where the player's character is personified as the gun she holds in front of

her, like the earlier-mentioned *Duke Nukem 3D*. Each mod changes the gender of the characters in those games. The patches hosted on Mutation.fem⁴² change male characters to female for a variety of reasons: either for the purposes of gender play, to correct gender bias by the games' authors, or to subvert the aggressive masculinity of this school of game creation.

The first mod in Mutation.fem that I played was Lynn Forest's *FemDOOM*. The original *DOOM*, a 1993 release from id Software, was one of the earliest games to popularize the first-person shooter, and has given tropes to that genre that are still imitated today. The game is viewed from a first person perspective, and the protagonist is given no name or background beyond being a space marine assigned to a tour of duty on Mars. Only the tiny, emoting face of a man in the status bar and the occasional baritone grunts when the player is shot gender the protagonist as male, and reinforce the tacit assumption that the player of a game of aggression must be male.

Lynn Forest, a fan and player of *DOOM*, was frustrated with the implication that she was playing a game for men. So she drew femme faces to replace the male faces that appear in the status bar, and put her patch online for other players to download. A later patch, "FEMDMSND.WAD," credited to "Amanda, Ivor B. and Rob Lord," replaces the masculine sound effects with feminine ones of equal intensity.

Gender is even more of an oversight in id Software's later *Quake*, which puts an even greater emphasis on competitive play. In *Quake*, one sees other players not as masked figures, but as hulks with a single male face. Players modded the game to introduce feminine faces and bodies. Schleiner notes⁴³ that later versions of *Quake* acknowledge players' desire for female avatars by including female models. She also hosts "skins" worn by PMS Clan, the Psychotic Men Slayers,⁴⁴ a band of women players who play competitively together in the game *Quake 2*.

Mutation.fem contains patches that paint a mustache on *Tomb Raider*'s protagonist Lara Croft and change Bungie's *Marathon* from a game about firing guns in dark, angular space stations to a game about fighting with egg flippers and dish towels in a checkerboard-tiled kitchen. Loren Petrich's "Tina-Bob" patch replaces the generic jumpsuited men—the

“Bobs”—that solely populate *Marathon*’s space station with identically jumpsuited women.

The Hack as Sampling

The repurposing of commercial game assets can be compared to sampling in music: using part of an existing song as an instrument in your own piece of music. Hip-hop artists often rap their own material over music sampled from another source, using the found music as a background for their own words.

HyperBound was created in 2009 by Michael Iantorno as part of his thesis project at Ryerson University.⁴⁵ Iantorno’s game is a hack of *EarthBound*, a Super Nintendo Entertainment System game developed by APE and the Japanese writer Shigesato Itoi and released in 1994. *EarthBound* is a digital role-playing game informed by Yuji Horii’s *Dragon Quest*, but instead of telling the story of a warrior who battles wolfmen and dragons in a fantasy world, *EarthBound* is set in a world vaguely resembling a Japanese vision of contemporary America. The protagonist is a young boy in a baseball cap who fights aliens and renegade animals using an old baseball bat and his newly discovered psychic abilities.

HyperBound takes its name from “hypertext,” text that’s arranged in a nonlinear structure. (This book is a text: it’s arranged to be read from start to finish, one page to the next. A website, where you might click on a word to “link” to a page about that subject, is hypertext.) What better model for the nonlinear exploration of text than the space of a digital game, where the player moves around the world by moving her character across the screen, encountering characters, and listening to what they have to say? That’s the part of the design of *EarthBound* that *HyperBound* has lifted. What it’s rejected is the fighting. The hack is purely about exploring the world and discovering the text, an original script written by Iantorno and his brother.

In *EarthBound*, the protagonist typically wears a baseball cap, a striped shirt, and a backpack. There’s a brief scene at the beginning of the game, though, where he appears in pajamas with ruffled bed-hair. That means that there were animated sprites of the bed-headed protagonist, seen from

all angles, just waiting in the game for Iantorno to use. *HyperBound* uses those sprites for its protagonist, to give him an identity apart from that of *EarthBound*'s protagonist and to emphasize his confused state: the protagonist is amnesiac, and the information he is trying to recover is knowledge of his own identity.

Throughout the hack, Iantorno repurposes assets from *EarthBound* to fit his new story. The bearded, sunglasses-wearing criminal the player encounters in *EarthBound* becomes the radio DJ whose show the protagonist of *HyperBound* used to call in to before he lost his memory. The boarding school that appears in *EarthBound*, with its classrooms and lockers, becomes the university that *HyperBound*'s protagonist attended, where he meets former teachers and finds valuable information on his previous life. All of *EarthBound*'s graphics are sampled and given new purposes in the hacked game.

The other assets Iantorno samples are less obvious, though just as crucial: the ways in which the player presses controller buttons to move the protagonist around a world, to engage characters who wander around of their own accord in conversation, and to advance their dialogue text on the screen. These are all things that are non-trivial to program, and that have been programmed for Iantorno by APE. To present his nonlinear story, he's taken advantage of an existing infrastructure for allowing the player to navigate a world of characters who can be spoken to.



If you can create a script but you can't create animated characters, scenery, or code to take player input and move a character around a large world that scrolls in eight directions, why not use those existing assets to present your original script? The work has already been done and the resources already exist to be sampled.

Changing the Script

There's another obvious reason to change the script of a game: to translate it to a new language. Publishers often neglect to translate and publish a game in a new language because of the cost: if they think a game won't sell enough in a part of the world to justify paying people to rewrite the script, to manufacture cartridges for different game-playing hardware, and to market the game in another country, they won't do it. But dumped roms, which are purely digital, don't require the expense of physical publishing, and because they're distributed freely, don't depend so heavily on marketing to make up that expense. And thus, people who read languages neglected by videogame publishers and care enough about games from other parts of the world will invest their own time and effort into the unpaid translation of games.

The work of translating a game involves more than just translating the script—which is often a lot of work, given that the dynamic nature of games usually means lots of scenes and incidental text to translate. The new script also needs to be inserted into the original game. In formats where space is expensive, such as Nintendo game cartridges for example, programmers use a lot of clever tricks to store text as efficiently as possible. Cracking those codes can be tricky. And if the original game was written in Japanese, the hacker also has to insert a new font. The way the game displays text might not be suited for English letters, so screen space is usually at a premium, too. Japanese is a compact, ideographic language, and a word that takes up two Japanese characters on screen could be an eight-letter word in English, or an idiom that requires translation as a full sentence. How do you translate the word so it fits? All game text tends to be presented in a limited part of the screen (a window that takes up the bottom half of the screen, for example, so the player can see the characters who are speaking on the top half), so all word choice is informed by just

how much space is available to display those words—space that was chosen based on the structure of an entirely different language!

The Japanese Game Boy Advance game *Mother 3*, a sequel to *EarthBound*, was translated into English over the course of two years by a two-man team who refer to themselves as “Jeffman” and “Tomato.”⁴⁶ The script for the game was written by the Japanese writer and journalist Shigesato Itoi. Nintendo’s American branch decided that translating the game would be too much work, and too expensive a project, given the limited audience they expected the game to have. Nine years before, Nintendo of America put a lot of money into translating *Mother 2* into *EarthBound* and manufacturing copies, only to have their marketing department completely mishandle it. Physical distribution is a big investment, after all: Nintendo of America was unwilling to make the investment a second time. American players who had encountered and liked *EarthBound*, though, were eager to play the sequel and were upset that it wasn’t to be published in English. Tomato, who works as a professional Japanese-to-English translator, played the Japanese game and decided that English-reading players deserved to experience the game. He undertook the translation of the game, and Jeffman did most of the coding to get around the memory limits of the original Japanese game: a two-year project, undertaken for free. The English version of *Mother 3* was released in July 2009. You can go on the official translation site’s forums to read how many players cried during the ending⁴⁷ (which is, incidentally, Stephen Spielberg’s metric for when games will become art,⁴⁸ not that tears are anywhere close to the only metric for judging the value of an experience).



But don't be misled into thinking that the majority of translation work is in translating Japanese games to English. As of this writing, people are working to translate *Mother 3* into Spanish and Latin American Spanish, Portuguese and Brazilian Portuguese, French, German, Italian, Dutch, and Malay. Whenever I visit the ROM hacking database at romhacking.net, I see translations of English games into German, Polish, Russian, Korean. Hackers are engaged in correcting the oversights of profit-oriented thinking and making the games they care about available in their native tongues.

Machinima

The product of “machinima”—from *machine* and *cinema*—isn't games, but the source material is. It's the same kind of sampling I've described: machinima uses the resources and infrastructure of commercial games as the basis for creating animated movies. All the resources are there in most 3-D games: the ability to move a camera through a world full of 3-D objects, and to move and animate those objects. Game hacking provides an immediate avenue to 3-D animation, one that's far more accessible than dedicated 3-D modeling (with a specialized—and costly—modeling program like Maya or 3D Studio Max) because there are already scripts in place to move things around and to operate a camera.

The origins of the community surrounding machinima can be traced to that of the game *Quake*, a fully 3-D first-person shooting game that offers

players the ability to record, distribute, and play back “demo” recordings of their games. Players began to manipulate and position the camera during these playbacks, introducing cinematography to the demos. Soon they were directing story scenes to frame their demos.

Most machinima stars the characters of the games they sample, because that’s what’s available: usually armored soldiers who crack jokes for the benefit of players familiar with the game. The 1996 *Diary of a Camper* by Matthew Van Sickler, Heath Brown, and company,⁴⁹ for example, follows a band of *Quake* characters as they deal with an opponent who is hiding and picking off their comrades one-by-one. (And who is ultimately revealed to be John Romero, one of the founders of id Software.) *Diary of a Camper* is widely remembered as the first game demo interested purely in storytelling, rather than presenting a sample of play footage.

But the subject matter of machinima has become increasingly diverse. The 2003 machinima *Anna*, produced by Katherine Anna Kang’s (wife of *Quake* programmer John Carmack) Fountainhead Entertainment, follows the life of a single flower on a forest floor.⁵⁰ *Anna* bears little resemblance to *Quake* at all, set in the woods with a cast of plants and animals.



As more tools become available, machinima looks less and less like videogames. A few years ago I encountered a movie of Sherwood Anderson’s short story “The Dumb Man” filmed in *Second Life*,⁵¹ a game that gives players the tools to construct anything they like, be it movie sets or actors or other games.

While machinima is not games, it's demonstrative of the ways that game tools can be used to create things radically different than the originals: including new games altogether.

Something Borrowed

Few commercial games, these days, are made entirely of original resources. The creators of the *Quake* and *Unreal* “engines,” the rules and code that drive the games, license their engines to other developers so that they can save time and work by modifying an existing framework rather than building an entirely new one. The *id Tech 3* engine—the engine that runs the third *Quake* game—was used to create games like *American McGee's Alice*, *Call of Duty*, *Star Trek: Voyager—Elite Force*, and *Star Wars Jedi Knight II: Jedi Outcast*.

The publishers of these games paid for licenses allowing them to sell and distribute their modifications, but plenty of players, using their knowledge of how their favorite games work, changed the games into something new and distributed their modifications for free.

Brandon Chung's *Citizen Abel: Gravity Bone*⁵² was built in a modified *Quake 2* engine, but the finished game bears little resemblance to a game about space marines blowing up aliens. *Gravity Bone* is a spy pastiche that opens with the player riding a fancy gated elevator into a masked costume party. Once there she must make contact with a fellow agent. This sets forth a rapidly accelerating chain of events that lead to an unexpected outcome.

What *Gravity Bone* takes from *Quake* is logic for moving a player around a three-dimensional world, looking around with a first-person camera, and collecting and manipulating items. Everything else—the game's appearance and presentation, the locations and scenario, the player's goals and the events they trigger—is original, designed and inserted by Brandon Chung. *Quake*, here, is just a vehicle for the delivery of Chung's original story.

Or look at Robert Yang's *Radiator* series,⁵³ which is built in the *Half-Life 2* engine. The second episode of the series, *Handle With Care*, involves the player visiting a marriage counselor. What *Half-Life 2* does well is allow the player to manipulate and move objects using a

rudimentary physics simulation: in *Handle With Care*, the protagonist, Jason, withdraws to a “Repression Facility” while his husband speaks with the counselor. The Repression Facility looks like the kind of grimy steel structure common to *Half-Life 2*. In the facility, the player carries crates containing repressed memories (if there’s one thing *Half-Life 2* does well, it’s letting the player carry crates around). If they’re put on the facility’s shelves, they’re locked away forever; if they’re broken, Jason revisits the memory he was trying to repress. The therapy session, seen through a monitor, progresses depending on whether Jason confronts or seals away his memories.



Both of these games look significantly different than the games they’re based on. Nevertheless, we can identify rules they’ve borrowed from their mother games. It’s apparent why it’s easier to change existing games into new games rather than creating them from scratch: particularly when it comes to 3-D games, solitary creators often don’t have the resources that the programmers of *Half-Life 2* had. It’s unlikely that either game could’ve existed without such sampling—it’s far less likely that Valve would make a game about the roles individual memories and pasts play in negotiating modern queer relationships than a *Half-Life 2* level involving building a wall of crates to stop drone planes from killing you.

Scripting cameras and collision, as well as creating 3-D architecture and a way of storing it conveniently in computer memory, are all big projects.

The Valve Hammer Editor, which edits *Half-Life 2* levels, gives a creator immediate access to all the design tools a team of paid programmers put into place. It also gives her access to the resources *Half-Life*'s art team created, which would otherwise be another huge investment of time, effort, and skill. Editors like Hammer allow creators to create without having to build an infrastructure to manage their creations.

New Worlds

Many game designers have anticipated their players' desires and created editors to accompany their games. The first level editor I ever encountered was for the Nintendo game *Excitebike*. *Excitebike* is a stunt bike race. The included EDIT MODE allows players to mix and match the obstacles from the game—ramps, bumps, gravel pits, and boost pads—to create new stunt tracks to race on. When I was little, I spent a lot of time with the instruction book open in my lap to the list of pieces, moving my racer through the track and rearranging the obstacles. The appeal of an editor like this to a child who's creative but lacks experience is that everything—the rules, the code, all of the art and sounds—are provided. The only thing I had to worry about, as a kid, was the track itself.

When we were discussing shareware, I mentioned Tim Sweeney, who founded Epic MegaGames—which later became Epic Games, the publisher of the *Unreal* engine used by many commercial and independent designers to build 3-D worlds. The first game Tim Sweeney sold, as “Potomac Computer Systems,” was released in 1991 and is called *ZZT*. (He picked this name so it would always be the very last game listed on alphabetically ordered shareware CDs.) The shareware part of *ZZT* was a series of four self-contained adventures, the first of which came with the game and the other three of which had to be purchased. (They were later released for free.) The free part of *ZZT*, however, was the editor that Sweeney used to make those adventures, and the means to play “worlds” created with it.

ZZT is a text-mode game: it takes advantage of DOS computers' native ability to display text. In addition to letters and numbers, DOS has a special extended sheet of characters for drawing basic pictures: smiley faces, playing card suits, Greek letters, and a variety of borders, lines, and

patterns. *ZZT* co-opts these: the player is a smiley face, Greek characters such as pi and omega are monsters who roam around, the diamond from the playing card suits is a gem that the player can collect and use to buy items. This set of 255 characters comprises everything that any *ZZT* world will ever contain.



And that's incredibly liberating. It means that *ZZT* is self-contained: there's no need to go outside the editor to find another tool to draw and animate graphics to import into the game. Authors get 255 characters in sixteen colors. And creative authors can do a lot with those: in that set there are patterns of varying density and lines of varying thickness, and I've seen some pretty impressive environments and portraits drawn with those text characters.

Sound and music are similarly self-contained: *ZZT* uses the PC speaker, which contains a range of simple notes and instruments. *ZZT* has a simple scripting system for playing PC speaker tunes, which again obviates the need to go outside the world editor for anything.

But oh, yes: let's talk about scripting. *ZZT* has a nice library of game pieces, which appear in Tim Sweeney's shareware *ZZT* worlds. There are pushable boulders, "rotators" that move objects around and can form conveyor belts if a series of them is placed in a line, stock monsters, weapons, and power-ups, and several kinds of walls (invisible walls, secret passages, walls that can be shot and destroyed). But the most interesting game piece is the Object, a programmable creature that can take on any

appearance. Each Object contains a script written in a language called ZYT-OOP (ZYT Object-Oriented Programming), a simple programming language.

Here's an example of ZYT-OOP:

```
/n/n/n/n/n/n/n
#shoot e
Haha, I shot your favorite PRICELESS VASE!
```

That tells an Object to move north (or up) six times, then shoot a bullet east, then display a haughty message on the screen. That's pretty simple, something someone who's never coded before should be able to understand. Though it's simple, ZYT-OOP is also surprisingly robust: each Object has its own script, and can receive messages from the player and from each other. Here's a more complicated example of two Objects interacting:

```
@Priceless Vase
#end
:shot
#send Collector: broken
#die
```

```
@Collector
#end
:broken
You broke my favorite PRICELESS VASE!
#char 1
/i
#char 2
/i
#char 1
/i
#char 2
```

When the vase is shot (the colon indicates a message the Object can receive) it sends a message to the object named Collector (the @ indicates an Object's name), which causes the Collector to display a message and briefly animate (#char is the "change character command"; character 1 is a hollow smiley face and character 2 is a filled smiley face. The direction "i" means "idle"; it tells the object to wait a moment between flipping its character).

Authors have taken advantage of ZYT-OOP to do things author Tim Sweeney never envisioned. One can create a *Super Mario Bros.*-style jumping game by creating button Objects that, when touched by the player, send messages causing a Super Mario Object to move left or right or jump, and that falls when there's nothing beneath it. Recently, someone built a machine that graphs sine waves in ZYT. Creative authors have bent the tools they were provided with to entirely different ends than they were intended for.

What's interesting about ZYT is not only the robustness and versatility of ZYT-OOP, but the simplicity of the game's overall presentation. Because all of the graphics and sounds that a ZYT world can produce are already inside it, it frees authors from having to worry about asset creation. They can just design with the tools they have in front of them. This makes it an

ideal introduction to scripting and design, and I know a handful of game designers who got their start making *ZZT* worlds.

House Rules

There's a concept in non-digital games called "house rules." Because players keep the rules in board and card and physical games, and because all the players have to agree on what rules they'll play by, it's easy for the players to change the rules or invent new rules to suit themselves. Maybe they feel as though their modification will correct an oversight on the part of the designer: part of the game may not quite work as it's presented. Or perhaps they've thought of a variation on the rules that's more interesting to them as veterans of the game. Maybe they need the game to be easier or harder, or to change it so it better suits the circumstances in which they're playing it: a game of Tag where the goals change to suit the space that's available, for example.

In digital games, the rules are kept by the computer and thus are somewhat hidden from the player. Adding house rules to a digital game seems more far-fetched. There are plenty of ways, though, to change games: changing the rules, inventing new rules, inventing new scenarios and whole new games out of the same content. Telling new stories!

What's really valuable about hacking and modifying games is the realization that there are ways of interacting with games other than just playing them: roles beyond consumer. Inventing rules is, after all, inventing games. The knowledge that games can be changed and remixed is the knowledge that games can be created: it's a small conceptual leap from the one to the other. While game creation tends to be more interdisciplinary—using the Valve Hammer Editor allows the creator access to all of the 3-D models, textures, sounds, characters, and such from *Half-Life*, letting her focus on design—there are game creation tools now that are as easy to approach as the typical level editor.

I said that the first level editor I encountered was *Excitebike*'s. I've played with plenty through the years: *ZZT* and its sort-of successor *MegaZeux*, *Dezaemon 3D* (a spaceship shooter kit for the Super Famicom), *Knytt Stories* (a jumping-and-climbing story creator), *Tombs of ASCIIroth* (a text-mode puzzle game kit), *Ragdoll Masters* (rubber dolls fighting

each other in the air), *The Elder Scrolls IV: Oblivion* (a fantasy computer role-playing game). I've used whatever tools I could get my hands on to tell stories.

You can, too. And once you've realized that videogames—even if they're made by big corporations with teams of hundreds and budgets of millions—are mutable, and can be reinvented by a single person, you can start to imagine what's possible when you have the means to create games that are entirely your own.

Footnotes

⁴⁰ More information on the history of all three games can be found at http://www.gameology.org/reviews/quest_for_bush_quest_for_saddam_content_vs_context.

⁴¹ <http://www.zophar.net/utilities/neslevel/smb-utility.html>.

⁴² <http://www.opensorcery.net/mutation>.

⁴³ See “The Female Skin Pac for Quake”: <http://www.opensorcery.net/mutation/patches.html>.

⁴⁴ Newman, James. *Videogames*, 57. The PMS Clan still exists, but has apparently been co-opted by corporate interests and has been rebranded Pandora's Mighty Soldiers.

⁴⁵ <http://hyperbound.net>.

⁴⁶ http://mother3.fobby.net/or_.

⁴⁷ Though you probably shouldn't. Spoilers.

⁴⁸ Lev Grossman, “The Art of the Virtual,” *Time*, November 8, 2004, <http://www.time.com/time/magazine/article/0,9171,995582,00.html>.

⁴⁹ <http://www.youtube.com/watch?v=uSGZOuD3kCU>.

⁵⁰ <http://www.youtube.com/watch?v=bKEr5RRKoO4>.

⁵¹ View it online at <http://vimeo.com/609147>.

⁵² <http://www.blendogames.com>.

⁵³ <http://www.radiator.debacle.us>.

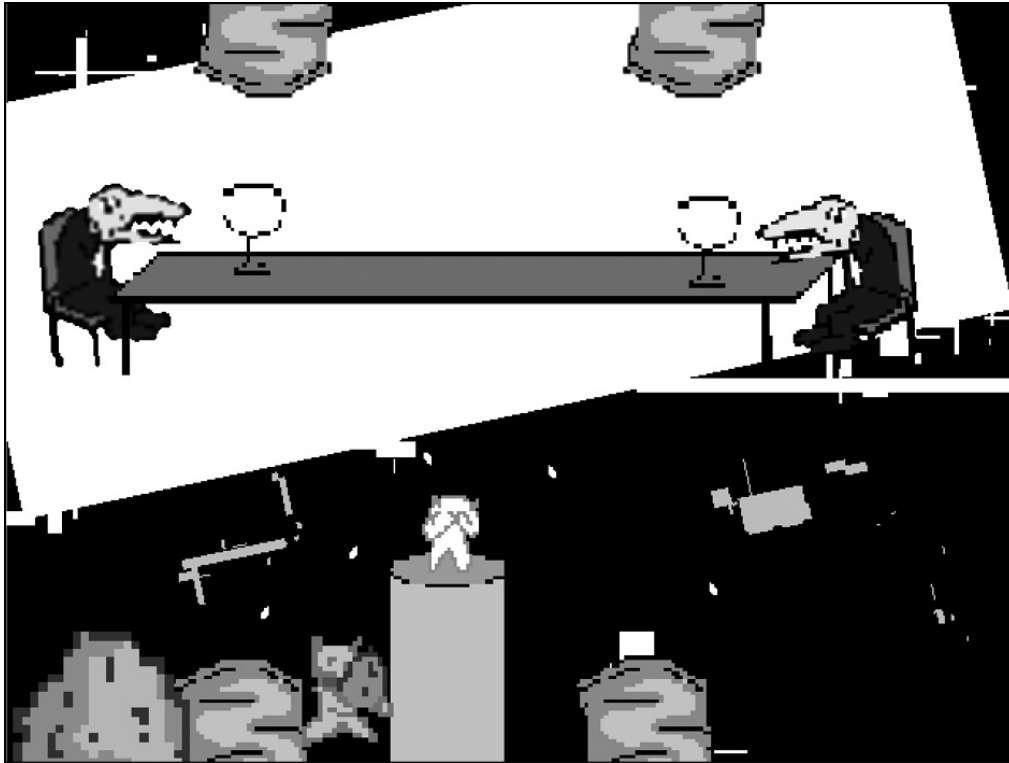
Chapter Five

The New Videogame

The *La La Land* series was created by Matt Aldridge under the alias “The Anemic.” It’s a series of five games, each supposedly based on one of the author’s dreams. The games are populated with recurring scrawl-doodle characters whose features are abstract enough to be sinister. The protagonist, Biggt (pronounced like “bigot”), has a smile that’s Glasgow-wide and a pair of eyes that don’t line up. The music is often seemingly incongruous: the climax of *La La Land 5* happens with the anti-evolution song “I’m No Kin to the Monkey” playing in the background. And the arbitrary goals the games give the player are almost always revealed to be against the protagonist’s best interest.

The first one I stumbled upon was *La La Land 2*. In this game, Biggt is confronted by a towering snaggletooth fish head, baby pink and jittering, which tells him: “it’s ok to steal from the rich cos they havelots [sic] to spare!” A counter—which in most games would be nailed to a fixed point on the screen, but in this game jiggers and jumps from point to point—reads “\$0.00.”

Biggt travels left to a screen where crow-like “nobles” with fish heads sit at opposite corners of a long table. A figure that resembles Biggt is on a pedestal, face in his hands. As he cries, some of the tears pass through pipes and fill the nobles’ wineglasses. At the far left is a hay-colored, shapeless pile that represents gold. Biggt scoops up an armful, making the pile smaller, and carries it back to the initial fish head, adding money to the counter. The fish head says, “Oh thanx biggt i want MORE.”



The player runs back and forth between the fish head and the nobles, redistributing wealth. When the player has stolen enough money, the fish head rises up to reveal a gold medallion hanging from one of its bones. “Look at the shini JEWELS I bought thanks. a lot.” Returning to the nobles, the player finds them dead at their table. “without money how were, the nobles to survive?” The game then closes.

Personal and abstract, The Anemic’s *La La Land* games were the first games of their kind that I encountered, and they would never have been made if not for Game Maker.

I’m not sure when *La La Land 2* was created, but in 2003, when I was seeking out the other games in the series, I discovered a game called *Seiklus* by an admirer of the *La La Land* games. The author, cly5m, had made worlds for Tim Sweeney’s *ZZT* that were as dreamlike as *La La Land*, if a bit larger in scope. (One, *Kudzu*, involves making a collect call to heaven at a melting payphone and visiting a hall filled with imprisoned invisible people.) American by birth, cly5m had visited the country of Estonia on a religious mission, then returned to America and discovered Game Maker. *Seiklus* is the Estonian word for “adventure.”

Seiklus begins with a boy and girl watching the stars on a cliff. A meteor hits and causes the boy to fall into a valley below. The game ends when he has found his way back to the girl. To return, the player has to collect colored tokens, which are found in a giant hollow tree, on a quiet, snow-covered landscape, in a dark land full of bones, and inside the stomach of an enormous creature. All of these scenes were hand-drawn by cly5m himself; the thick line art style is as unmistakably his as the collection of pastoral-alien dream scenes.

To the West

From my current perspective, I can say that what I always wanted to do was make videogames, but as a child, making videogames was something mystical that I never thought I'd be capable of it. So I funneled my creativity into other channels. I was better at writing than drawing, so I was a creative writing major in college.

After playing *Seiklus*, I figured that if cly5m could make the transition from making add-ons for *ZZT* to making self-contained games in Game Maker, then I could do the same.

I had taught myself to program a little, particularly in a language called **Blitz Basic**, in which I had recreated a few games that I knew. I had made a board game-like version of *PAC-MAN* where the player and the ghosts took turns moving rather than moving at the same time. Making objects move and react to each other all at once was something that was beyond me, and objects' placement on the screen had to be hard-coded, making it difficult to experiment with actual design. Game Maker was refreshingly similar to *ZZT* to me: every individual object had its own instructions to tell it what to do and how to react to the other objects. I could draw every object, give it instructions, put it wherever I wanted to on the screen, and let the program play out and resolve interactions based on how I'd told the objects to behave.

My first Game Maker game was called *Jaywalker*. In *Jaywalker*, the player controls a disembodied head named Marjorie with a HotHead Paisan-style crown of orange hair and a stud in her nose (I think I made the game around the time I first got my nose pierced.) Marjorie is fiercely about the rights of pedestrians, and is committed to destroying as many

cars as possible. The player moves her back and forth across a busy intersection, causing cars to swerve around her and crash into each other. Cars will crash into cars that have already been wrecked, creating huge pile-ups and causing Marjorie to stick out her tongue triumphantly. If she's ever hit by a car, the game is over.

I dropped out of college some time after creating *Jaywalker*. When I quit, I was in my fourth year of school with no end in sight. I didn't see myself as a writer, I hated school, and I thought (and think) that "higher education" is bullshit.

I spent another year in New York, where I was born. I made some more games during that time. Terrified I'd be stuck in the same spot for the rest of my life, I decided I needed a reason to move somewhere far away and a way to earn rent, a trade that I could learn while I was there. The thought of earning my living by making games was exciting: why not go to a games college in another part of the country, perfect my craft, and find a way to pay my rent by making videogames?

Specialized videogame schools had been around for a while at this time. I had known of DigiPen, in Seattle, since I was a teen, and had recently heard of Full Sail, a movie school in Florida, but both of those seemed to me like game programs tacked on to visual arts schools. I chose the **Guildhall at Southern Methodist University** because it was the only school I discovered that had a level design focus. (A Game Maker game I made after quitting creative writing school, *Invader*, which was about an alien from the game *Space Invaders* crash-landing on a weird planet and having to find her way off, was part of my portfolio.)

The Guildhall was in the middle of Plano, Texas. Plano, Texas, is brown and not much else. They have a Frito-Lay factory, parking lots, and a videogame school. At the time, I kept a strict vegan diet and didn't drive. There was nothing to eat and nowhere to go.

But the latter didn't matter; when you were at the Guildhall you had no life outside the Guildhall. I remember the first day of orientation, sitting in a lecture hall with my future classmates and the spouses they'd brought with them to this wasted brown land. One of the other level design students had his wife and their year-old child with him. "Give her a kiss and say good-bye," the director of the school told him in front of the assembly. "You're not going to see her for two years."

I was in Plano, Texas, for six months.

You're at school from nine to five. You stay after and do your work with the teams they've assigned you to. Late at night you drag yourself home and do your actual homework. Maybe you get a few hours of sleep. The idea behind the school is that you're always in what the Big Games Industry calls "crunch time": unpaid overtime. Your masters want the game done by Christmas, so you don't leave the office until it's done. This is why people in the industry aren't healthy; this is why they burn out and quit games within a few years. This is why you miss the second year of your daughter's life. This is their scheme: you put up with crunch time all the time while you're in school, so when you work for a big publisher—or, rather, a studio contracted by a big publisher—you won't complain about being told you can't see your daughter until the game's done. The Guildhall boasts an over 90 percent employment rate, and it's true: they will get you a job in the games industry. That's because they will make you into exactly the kind of worker the games industry wants. It's that kind of school.

And it works; that's the horrifying thing. My classmates were all self-identified gamers and game fans and were willing to put up with anything in order to live their dream of making videogames. That's the carrot the industry dangles, and it's what we take away from the industry when we create a form to which anyone can contribute. As long as the industry is allowed to continue acting as the gatekeeper to game creation, people will continue to accept the ways in which the industry tramples the lives and well-being of the creative people who make games, rather than challenging the insane level of control that publishers ask over developers' lives.

Needless to say, I was not at the school long. I butted heads with lecturers too many times, I asked too many questions, and my *Oblivion* mods were too experimental (I didn't include messages that popped up and told the player she'd achieved her goal because I didn't think the player was too stupid to realize that.) Eventually I was pulled into the director's office and asked to leave. "If your unit is marching to Fort Worth, you don't ask 'what if we go to Austin instead.'" That's a paraphrase of what the school's director told me.⁵⁴ I've had a while to think about this, and

I've since decided that if you know there's an army waiting to ambush you at Fort Worth, it's fatally irresponsible not to try to dissuade your comrades from marching there blind.

I said that I was in Texas for six months. Each semester at the Guildhall was just over two months long; they canned me the first day of the third semester. I spent the last month or so getting my stuff moved back to New York and making a game called *Calamity Annie*.

Calamity Strikes

Stranded in Plano, Texas, with everyone I knew occupied with school all of the day and most of the night, working on *Calamity Annie* is what kept me from going stir-crazy until I flew back to New York.



The protagonist and namesake of *Calamity Annie* is a brash young dyke who rolls into the Old West (from the New West, naturally) with her

breasts bound and a pistol in her holster. She tears across the countryside, dropping hombres in one-on-one shootouts. Along the way she meets a lady named Valentine with a different past but the same breed of loneliness. If the player's sharp enough, Annie and her Valentine ride off into the sunset together at the end.

The whole game is played with the mouse, by pointing to aim and clicking to shoot. A duel consists of “holstering” your cursor at the bottom of the screen, waiting for the call of “DRAW!” and quickly aiming and firing before your opponent can do the same. Interactions with Annie's love interest use the same controls. The first time you meet her, Valentine will ask for a light for her cigarette. Aim at the cigarette, click, and PEW! Flirtation has begun.

It's a hard game—the quick-draw contests get quicker and quicker, and eventually the player has just milliseconds to react. But even when the player gets GAME OVER and Annie spills some blood, Valentine patches her up, and their relationship goes on. What's between them outlasts any individual firefight. Every play of the game isn't self-contained but is—in a small way—fitted into the ongoing tale of love and connection, which the gunfights are just a means to developing.



Calamity Annie is a lot of things to me. It's about being an angry young woman in a hostile land (that just so happens to be Texas), trying to prove your worth. I certainly was trying to do that after they kicked me out of the Guildhall. I was making a game, and I was doing it my way: I built every piece of the game myself. I drew all the pictures; I wrote all the music; I scripted all the events in Game Maker. If the Guildhall taught me anything, it was that videogames needed to be saved from the industry, that a creative form deserves better than an assembly line production process.

But the game is also about love, and how passion finds love. I had started seeing a loud-ass submissive named Daphny Drucilla Delight David shortly before starting at the Guildhall, and the workload had kept me away from her except for one week of vacation between each semester. The time we got to spend with each other while I was at school was mostly limited to me calling her and crying from the stress of having to split my

energy between standing up to teachers, finishing my work, and doing good work, when all I wanted to do was make games.



I'm writing this, three years later, from the apartment where I live with her in California. The theme of *Calamity Annie* is that being driven will drive you to love, and that passionate people are attracted to passion. *Annie* was one of the first in a series of games that led to my being able to pay for a home and food by making games, which is what I went to Texas to seek. But it didn't come from the Guildhall. It came from me, from my stubbornness, and the stubborn friends who helped me.

Later games have made me more money—I asked players who wanted to support *Calamity Annie* for a donation of at least a dollar, in exchange for which they got a bunch of secret bad guys to shoot (I made a few hundred total)—but I still think of *Annie* as my most important game. It's everything I realized I wanted games to be. It's personal (Annie has my name, after all) and it has a clear, unmuddled mechanical idea that

translates perfectly to a storytelling idea. I want games to tell stories, and *Calamity Annie* tells my story, or at least the story of my stay in Texas.

Authors

My friend Lamar Williams is working on a documentary about videogame creators called *You Meet the Nicest People Making Videogames*. In one of the project's trailers, he points out that the idea of digital games as the product of teams is a myth: many of our most important games, and the ones that have been the most widely duplicated, are almost entirely the projects of impassioned individuals.⁵⁵ The examples that appear in the trailer are: *Another World*, a game about finding friendship during a struggle for survival in an alien world, created by Eric Chahi (who even painted the cover art for the game himself!); *Karateka*, a kung fu game with strong visual storytelling by *Prince of Persia* creator Jordan Mechner; *Defender*, a seminal arcade game by Eugene Jarvis and Larry DeMar; and *Berzerk*, almost as seminal an arcade game by Alan McNeil. He goes on to show contemporary games by individual authors, some of them made in Game Maker.

There's nothing unnatural about a digital game by an individual creator (or a pair of creators). It is, in fact, much harder to keep the idea behind a game coherent when the designer is managing a team of many people who are each working on one aspect of the game separately. That's part of the reason why contemporary big-budget games have so much clutter and so few strong ideas. The games are all over the place because the creators were all over the place. It's hard to have a strong singular vision when the process of creation is spread so thin.

Digital games contain video, audio, animation, design, and rules. You can parcel out these roles, but the closer they remain to each other, the more cohesive the work you create. If I'm the designer and I'm also drawing the spaceship that appears in the game, I know exactly how I intend to use that spaceship in terms of play, what its place is in the larger story, and what its appearance should express. I have a vision, in other words.

Books are written by single authors or by author-editor teams. Visual art is typically made by an individual artist. It makes sense for creators to be

close to their work and to own their work completely, and that's something that the big teams that big-budget games demand can't have. When an individual or pair is solely responsible for a work you can watch an individual style develop: you can trace themes, both mechanical and otherwise, across a creator's work. (The Anemic's *La La Land* games, though each game is very different from the rest, have a strong singular style that persists throughout the series.)

And being able (or learning to) identify the individual style, and growth, of individual authors leads to better criticism and a critical understanding of games. Not to mention, like I said, more personal games, more relevant games, more games with something to say. I want a world where everyone is capable of sitting down at a computer and making a game by herself. This is not to say that all games need to be made that way, but as a paradigm, I think the individual author has more to offer us than the team, especially at a time when videogames are so seemingly creatively bankrupt.

Death from Overwork

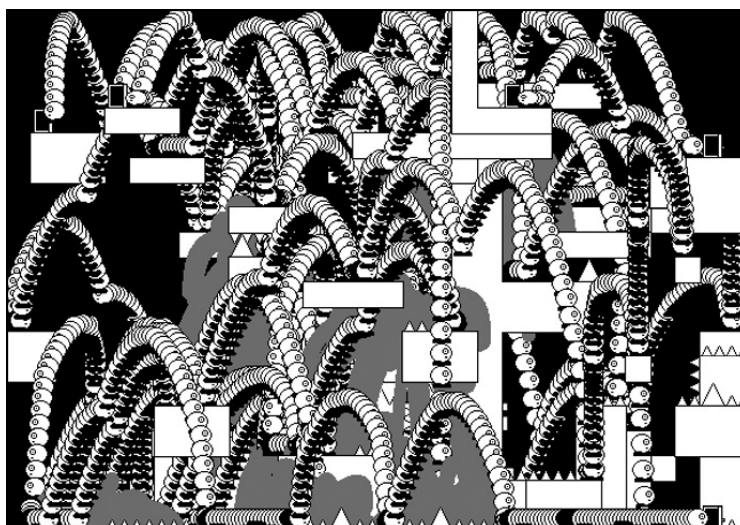
I first became aware of Jesse Venbrux, a game designer from the Netherlands, in 2008 because of his game *Execution*. When the player starts this game, she sees, through the sight of a gun, a prisoner tied to a stake. If the player shoots the prisoner, the prisoner dies, and the player is told, "You lose." If the player starts the game a second time to try to avoid losing, she again sees the same prisoner, still riddled with bullet holes.

Execution (a direct inspiration for *Calamity Annie*) points out something about causality in games: in particular, it points out how, as players, we expect to be allowed to undo and redo our choices at whim. The game takes just a few minutes to make this point: it's the kind of game that could never be a commercial product, but nevertheless has something essential to tell us.

Execution would probably never have been made without Game Maker. Jesse Venbrux might never have dug into game creation were it not for Game Maker. Though he's more recently gotten involved in Flash game creation and released games for the iPhone and its permutations, his website lists Game Maker-created games going as far back as 2004.⁵⁶ In

Venbrux's body of work we can clearly see ongoing themes, both mechanical and contextual. His 2007 *Frozzd*, which was created for a Game Maker theme competition (entrants were asked to design games with a "winter" theme), involves navigating free-floating, irregularly shaped planets that all have their own gravity. That is, the player can walk all the way around the surface of one and arrive where she started, before jumping off and flying to another one. His 2010 game *Maru* features the same kind of irregular planet navigation, as does his most recent game (as of this writing), *They Need To Be Fed* for the iPhone.

His 2008 game *Deaths* remembers every time any player playing the game anywhere in the world meets death. Players will see the corpses of the last fifty players to have played the game, indicating particularly difficult areas and hidden traps. There is one screen in which players must continually die in order to build a bridge of corpses for future players to reach a high plateau (this is where the earlier-mentioned *We the Giants* gets its seed). His 2008 and 2009 diptych, *You Made It* and *You Probably Won't Make It*, also remember the player's death, painting patches of blood wherever she encounters a deadly spike. In the former game, this is especially important because the blood will paint over the visual information from the previous screen, showing what the terrain actually looks like. (In this game, nothing the player sees is ever erased from the screen. It is only overwritten by new information.)



The theme of protagonist death as narrative progress is an important one in Venbrux's work. His *Karoshi* series of games (*Karoshi* can be translated from Japanese as "death from overwork") require the player to guide the protagonist, a suit-and-tied salary man, to his death in a series of increasingly convoluted screens. Sometimes, in a similar vein to *Execution*, the levels of *Karoshi* force the player to make use of something she thought was external to the game in order to proceed. For example, there's a stage in *Karoshi 2.0* called "What's On the Menu Today?" In this stage, the player must quit to the game menu—the screen where the player selects what stage she wants to play. The protagonist follows and falls to his death. Another stage requires the player to put a CD in her computer so that a boom box on the screen can play loudly enough to shunt a safe onto the protagonist.

Sure enough, Venbrux's iPhone game *They Need To Be Fed* requires the player to feed the protagonist to a monster at the end of every stage in order to complete the stage. (His 2008 game *Pazzon* requires the player to lead characters to the dinner tables of other game inhabitants.) It also features some of Venbrux's recurring "characters," such as the guided missile launchers that appear in his 2009 *Focus*, which in turn features the protagonist of *You Made It*.

We can notice and trace details and themes like these from game to game because each of these games is almost entirely the work of a sole, identifiable author. It makes us better, more literate critics of games to be able to see and discuss the progress of Jesse Venbrux as a designer. And it makes games more richly personal if we can play them in the context of the ongoing work and growth of a knowable author. This is a more useful paradigm than viewing games as the work of nebulous teams of hundreds.

The Author in the Games marketplace

At the time that Jesse Venbrux started making games, there was no place for him to market his work. Six years later, he's released a game for sale through the Apple Store. While this book isn't about making games for money—it's about making games for the sake of having more games by more people, and there will always be more hobbyist authors than commercial authors—I think it's worth discussing the changing

videogames marketplace in the context of the changing videogames paradigm. Just as digital games are starting to be thought of as the work of an author rather than a corporation, there is a growing place for solo game authors in the marketplace, and people can make a living doing this (though it's not easy).

I've been making my living in recent years by selling Flash games to "sponsors." Flash is an infrastructure for embedding movies and games in websites, and sponsors will pay for the privilege of including good games in their websites because games attract players who look at ads, and then explore the rest of the site to play more games and look at more ads. The more traffic a website gets, the more valuable that site is to advertisers, so traffic from popular games is important to site owners. Flash is much more complicated to use than something like Game Maker (although that's changing in a small way with tools like Stencyl, Flixel, and FlashPunk), and requires the kind of dedication that someone who just wants to make games in her spare time isn't likely to be able to spare. It's an avenue, though, for solitary authors to make the games they want and to find money for them.

The first Flash game I sold is about a flying pig. I sold it to *Newgrounds*—the "Everything, By Everyone" portal I mentioned earlier—for enough money to help me move from America's East Coast to the West. Later I sold a game called *REDDER* for twice as much, and while working on this book I sold a game called *Lesbian Spider-Queens of Mars* to *Adult Swim* for enough to keep me out of trouble for a while. I like this model because it means that although I get paid for my work, the game remains free to players. That's important, because I want as few barriers between my work and its audience as possible.

Like Jesse Venbrux, I've also released an iPhone game, although mine is totally free. It's also based on a game I made in Game Maker—a friend of mine with greater technical knowledge, Bennett Foddy, programmed the iPhone version. The game's called *Chicanery*,⁵⁷ and it's about the interactions that go on between players outside of the digital components of the game (the stuff that's on the screen). Each player's goal is to keep a finger on the screen as long as possible, and to hinder the other players' abilities to do that. Usually the players do this by punching, shoving, or

tickling each other. A guy I met who worked at WayForward Technologies later showed me the dent that playing the game had made in his iPhone.

When they're not free like *Chicanery*, iPhone games are usually sold for a dollar or two. And Apple has total control over its marketplace and what games can be sold within it, and is willing to wipe out games without a second word, as they did in 2010 with thousands of games they thought were too “sexual.”⁵⁸ But the fact that authors are selling games there, and sometimes making a profit, shows that the digital games marketplace is making room for a new paradigm. Note that the Apple Store sells games exclusively through digital distribution—there's no publishing or manufacturing cost whatsoever.

Similar markets are Android phones, Steam for the computer, and the Xbox Live Marketplace. These are all digital distribution stores tied to specific technologies, and each is run by a corporation that exercises total control over what content is available on which device. It's a situation that leaves creators at the mercy of corporations, but it's a sign, at least, that avenues are appearing for solitary authors to make money by creating games. And, hopefully, there will soon be more avenues, and decentralized ones.

Some authors take it upon themselves to sell and distribute their own games. My friend Edmund McMillen sold a CD with a bunch of his games and drawings on it—his attempt to create a game-as-zine. One of the things I used to do was to release a game that was free to play from start to finish—again, free is important—but to offer a secret password to anyone who donated at least a dollar. The password unlocked additional characters that would show up in the game as neat surprises. I got a few people who donated exactly one dollar, but most people gave five or ten: the invitation to donate was all they needed.

But that's enough about money. What I'm interested in is game creation as a goal in and of itself.

Crap Games

Glorious Trainwrecks is about bringing back the spirit of postcardware, circa 1993. It's about throwing a bunch of random crap into your game and keeping whatever sticks. About bringing back a time when you didn't care so much about “production values,” as

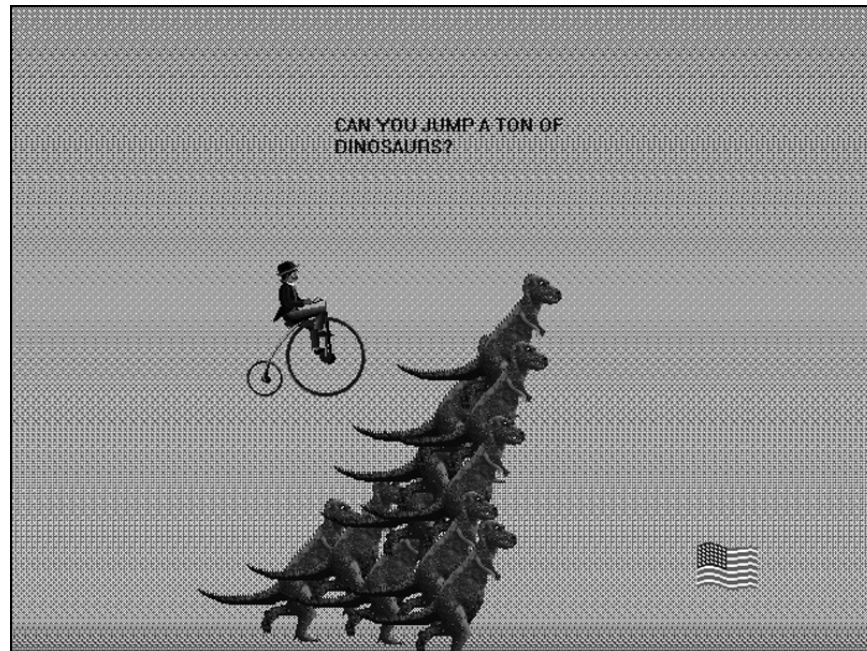
much as ripping sound samples from your favorite television shows to use in your game, or animating pictures of yourself making goofy faces on your webcam. Where every ridiculous idea you had, you would just sit down and code. When you would make up a “company name” to legitimize dorking around on the computer with your friends. . . .

Together, you and I will bring the true spirit of indie gaming back. Yes, you! For this site is about nothing, if it is not about getting off your ass and *creating*. Wikipedia claims that they used to stage trainwrecks (with empty trains, of course) for the amusement of the general population. Would the world not be a better place if we brought this tradition back?

So states Jeremy Penner on his website, *Glorious Trainwrecks* (at gloriotrainwrecks.com), founded in 2007. The site is interested in a videogame application of the “Crap Art” philosophy⁵⁹: that professionals aren’t the only ones who should make art, and that creation is a meaningful goal in and of itself. Games are more wonderful, more creative, and more inventive when they’re thrown together around an idea with little regard for production values or painstaking creative choices. There was no place to explore the videogame as an act of raw creation, so *Glorious Trainwrecks* was built to provide one.

On the third Saturday of every month, *Glorious Trainwrecks* holds an event called **Klik of the Month Klub, after Klik & Play**, the simple, ugly game creation system (designed for children and people who have never made games before) that most of the participants prefer. But there are no enforced rules about what tools people can use, what they should make, or how much time they should take, though it’s in the spirit of the thing to finish the game, from start to end, in two hours.

The experience forces participants to get past their egos and their meticulous plans for future epic games, to stop focusing on details and CREATE. Klik of the Month is about doing, not planning. Having to finish a game in two hours keeps authors from getting hung up on lesser decisions about their games. This is why Klik & Play is so well suited for the exercise: **it’s full of clip art, stock sound effects, and design shortcuts. All the resources are already there: just get your hands dirty.**



The results are usually sloppy and haphazard. That's what the glory is: having made something raw and improvised. During different Klikes of the Month, I made a version of *Pong* where touching the ball makes you lose points (*Dodgeball*), a game about escorting vulvas back to their home planet (*Box Pushing Game*), a game about pushing sumo wrestlers into basketball nets with your belly (*Sumo Dunk*), and a pile more. In 2007, Glorious Trainwrecks ran a two-day event (the 100-in-1 Klik & Play Pirate Kart) that produced one hundred games. We did it again in 2010 and produced 529 games by over a hundred people. People have the desire and creativity to make games; they're only stopped by the roadblocks that get put in the way. Tear down the roadblocks and we'll have so many more games—and more important, more authors.

Game Sketches

What will help to tear down those roadblocks and allow games to be more creative—in the sense of, “Let's create something!”—are tools that allow more spontaneity. Klik & Play comes with a bunch of clip art that the author can easily appropriate toward other ends. Game Maker allows the author to draw sprites right in the program before giving them rules and actions to perform. Stencyl allows instant access to an online database of resources other creators have already made. That's the solution, I think: to

put as little distance as possible between the idea and a playable game. Certainly, many authors will spend a lot of time adding details to their games and developing their ideas. But there's value in ideas, especially in a form so young, and those ideas need to be put out there.

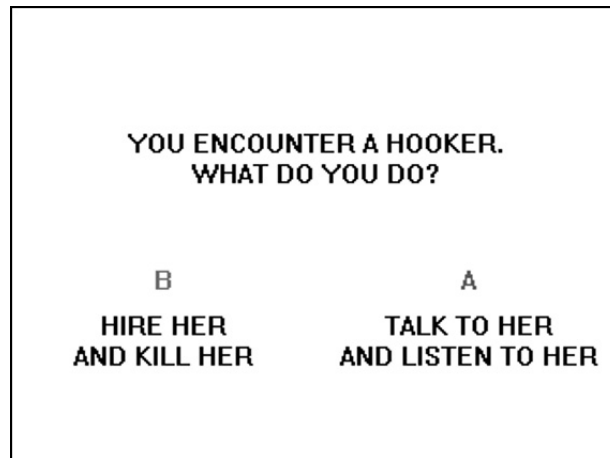
There's no way to sketch in games. In half an hour, I can sit with a pencil and paper and draw some dumb little comic strip. Videogames have no easy equivalent to that. And yet dumb little games are important because they enrich our vocabulary of ideas. The ideal game-making tool, I think, would look like a sketchpad: I could draw a landscape with a character in it, then give that character rules about how she could move across that landscape.

The editor that Fred Wood made for his game, *Love*,⁶⁰ comes close. *Love* is a simple little game about a stick figure who navigates a world by running and jumping; if she touches anything dangerous, she dies. To create a *Love* level, all an author has to do is draw it. The game will figure out how the stick figure interacts with the level. What stops the editor from being perfect is the fact that the author has to split the drawing into a few different images: a layer for things the player can touch and a layer for things that will kill the player on touch. But this is the right direction: if level design can be as simple as drawing a sketch, then game making can be as quick as putting an idea onto the screen.

And someone that doesn't know how to script or code probably knows how to draw—at least, to sketch. All of the most successful game-making tools for non-professionals I've seen have looked like this. *Warioware: D.I.Y.* is probably the most mainstream commercial game-making software I've encountered, in that it was developed by a big publisher (Nintendo) for sale on store shelves. And while Nintendo makes it hard for people to actually distribute games they've made with the software (if the tool had a YouTube-like gallery where people could browse and play other peoples' creations, it would have changed a generation), the process of creating a game is brilliant.

Warioware is for the Nintendo DS, which has a touch screen and plastic stylus. Player-authors can draw sprites on the screen like they're drawing with a marker. Then they assign the sprites simple rules to tell them what to do during the game. But there are also a number of constraints that keep

the author from being too ambitious: each game is only seconds long, the author only has a handful of objects to work with, and there's an equally limited number of sprites. This is a tool that's designed specifically to create very small games, and that's good, because small games are exactly what we need more of.



The proliferation of small games also serves another goal: it'll make games more personal. Ian Bogost, writing for *Gamasutra*, discussed the potential of games by way of talking about snapshots. The introduction of cheap portable cameras like the Kodak Brownie in the 1960s allowed people who were not professional photographers with access to expensive cameras and lenses to document anything, including things that held little or no value for anyone other than the taker:

[Consider] *You're Invited to Go to Heaven*, a simple quiz game. . . . *You're Invited* is a rudimentary example of Christian evangelism.

The game poses just a single question, "Who is the Lord of your life," and offers four answers: Chris Brown, Orlando Bloom, Zac Efron, and Jesus Christ. The "correct" choice is obvious, and it's tempting to write off this game as trite, even worthless.

Its single question would seem barely to qualify it as a quiz game, a genre itself on the very fringes of the medium.

But there is something deliberate and honest about its simplicity: this is not a game meant to inspire conversion or even head-scratching; it's just a little touchstone in someone's day for reinforcing what's really important to the believer.⁶¹

It might be hard to see the value in a thousand versions of *You're Invited to Go to Heaven*, but the real value of people producing digital games as quickly and easily as photographs is more subtle:

The Brownie teaches us that snapshots aren't just good pictures created easily thanks to simple tools. They are also good pictures—or games—created for different purposes. The future of video game snapshots will require platform creators to show their potential users how to incorporate games into their individual lives.

The result could be very important. The snapshot didn't just popularize photography as chaff, it also helped more ordinary people appreciate photography as craft. The successful game creation platform will be the one we can say the same of, someday.⁶²

Making games more pervasive—not just games, but game CREATION—will help us to better appreciate games and think about the craft and design of them. It'll also, I think, demystify game creation. We won't think of games so narrowly because we'll understand that they're capable of telling many different stories: stories about dyke cowgirls getting their girlfriends off booze, for example, or a record of a dream.

The ideal game-making tool, the game sketchpad, isn't here yet, but there are many tools that are close—or at least much closer than what engineers had to work with back in the day. But even given that, how do you, with little to no game making experience, get your feet wet and your hands dirty? How do you make a game?

Footnotes

⁵⁴ Paraphrase only because I don't remember which two actual cities he named.

⁵⁵ You can watch the trailer online at: <http://www.youtube.com/watch?v=Cvd0GbVCoYw>.

⁵⁶ <http://venbrux.com>.

⁵⁷ <http://itunes.apple.com/us/app/chicanery/id395775628?mt=8>.

⁵⁸ Jason Kincaid, "The New App Store Rules: No Swimsuits, No Skin, and No Innuendo," *TechCrunch*, February 20, 2010, <http://techcrunch.com/2010/02/20/app-store-rules-sexy>.

⁵⁹ See the principles of Crap Art explicated at: <http://crapart.spacebar.org>.

⁶⁰ <http://www.love-the-game.com>.

⁶¹ Ian Bogost, "Persuasive Games: Video Game Snapshots," *Gamasutra*, September 11, 2008, http://www.gamasutra.com/view/feature/3784/persuasive_games_video_game_.php.

⁶² Ibid.

Chapter Six

Making the Games

How do you become a game author? By making a game! Get an idea, find some tools, and make something you can play. You can improve your craft later. At the start, just getting some experience putting together a game and making creative decisions about it will be incredibly valuable. I'm going to describe a few of the smaller, simpler games I've made and the process by which I made them.

Title, Interaction, Resolution: *Gay Sniper*

I made *Gay Sniper* in May of 2009 during a Klik of the Month Klub, the two-hour game design contest I described in the last chapter. I think I made it in about an hour. It was inspired by a video made by West Virginia 4 Marriage,⁶³ an organization that absolutely does not want queers to gain the right to legally marry in West Virginia. One scene in the video particularly grabbed me: The ideal heterosexual family—one man, one woman, and two kids—sits on a stoop blowing bubbles. (The man is white and the woman brown, so the filmmakers can position themselves as an anti-racist organization.) The narrator: “Marriage between a man and a woman is held up as the ideal in all of civilized society because of its profound stabilizing influence on our culture, as well as the important economic benefits of strong and intact families. But today that ideal is under an unrelenting attack, and same-sex marriage in Western Virginia is a closer reality than you may think.” As he says this, a crosshair appears on the screen with the het couple in its sights, personifying the homo who wants to get married as a family-assassinating sniper.

The shot of the couple through the rifle sight, at least to me, was reminiscent of a videogame—a sniper game like *Silent Scope*, for example. I decided it *should* be a game: about a gay sniper who assassinates long-held American values.

I made *Gay Sniper* in Game Maker. Here's how the game plays: The game starts with a title card. Then the player sees the ideal family from

the video through a crosshair. The crosshair is smaller than the screen, and the player can move the mouse to pan the crosshair over the screen. (The player can only see what's inside the crosshair. Anything else is blacked out.) The voice-over from the video—"marriage between a man and a woman is held up as the ideal in all of civilized society"—plays. When the player left-clicks the mouse over the family—to pull the trigger—a gunshot is heard, and the game changes to a screen that displays the text "America is destroyed" over a picture of the contiguous United States cracking in half. After a couple of seconds, the game ends.

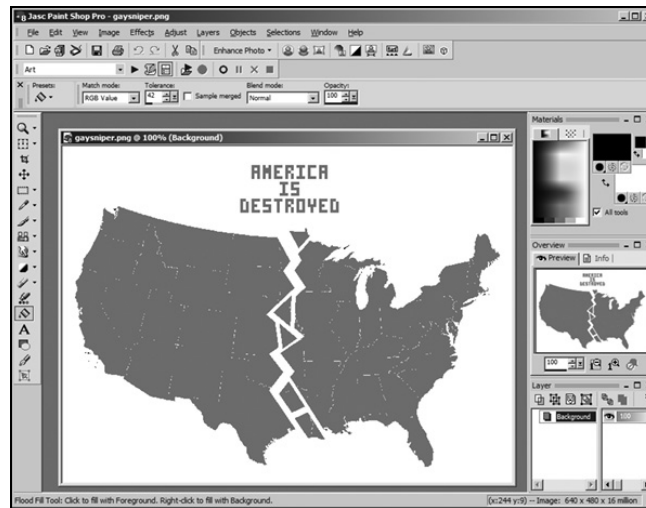


It's a simple game, much like *You're Invited to Go to Heaven*. How exactly did I make it?

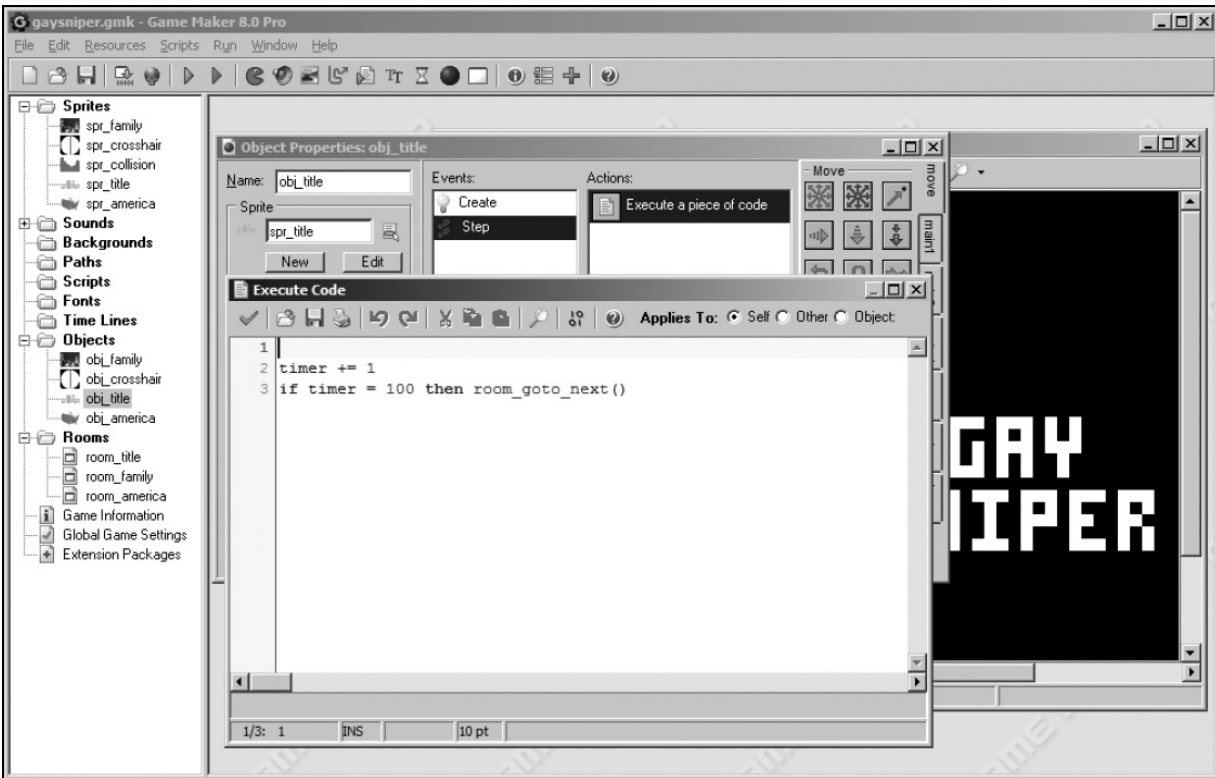
I'm going to avoid going into too many details in explaining how I made the game work. This isn't intended to be a guide to working with Game Maker: I'm interested in writing a more universal approach to thinking about game design and in making the decisions that guide an idea into a game. If you want more specific details on how to build a game in Game Maker you ought to be able to find them easily, by asking questions of other Game Maker users or by consulting a tutorial on the Internet.⁶⁴

First, let's consider the shape of the game. It's made of three distinct screens: a Title screen that introduces the game's theme, the Crosshair screen where the player takes action, and a screen that displays the Resolution of the player's action (the destruction of America). This is the

basic narrative shape. In Game Maker, I could easily use “rooms” to arrange these individual screens.



The Title and Resolution screens are the easiest. Each of these screens needs to do just a few things: show a picture, wait a few seconds, and advance the game (or end the game, in the case of the Resolution screen). To draw the pictures I used a program called PaintShop Pro, though Game Maker has a built-in drawing program that I could’ve just as easily used. A free drawing program like GIMP could have done the same thing. Here’s a list of the pictures I drew: one that says “Gay Sniper” in red text on white, and one that says “America is destroyed” over a picture of a cracked United States. For the latter, I downloaded a map from the Internet, dyed it red, and drew a big crack along the middle. Then I used Game Maker’s built-in timer system, called an “alarm clock,” to make those images stay on the screen for a few seconds so the player can read them before going to the next “room.” (Though making a manual timer is just as easy as adding one to a counter on each frame, and then doing something when the counter reaches the right number.)



That's all these two screens do. The Title screen also plays the same little melody that opens the West Virginia 4 Marriage video (a light twinkly tune that helps us prepare to appreciate heterosexual marriage). I recorded that directly from the video using a free program called Audacity⁶⁵ and imported it into Game Maker. The Title and Resolution screens are easy, as there's no interaction. They just frame the action that happens in the Crosshair screen. Interaction is the tricky part, and also the most important part.

The Crosshair screen requires a few things: the narration from the video and the sound of a gunshot, for starters. The former I recorded off the video with Audacity, like I did with the melody on the Title screen, and the latter I made with a free sound effects program called SFXR.⁶⁶ Next, the family in the background. I took two screenshots of the family, one in which they're about to blow bubbles and one in which bubbles are blown, and I used the timer to make the image switch every few seconds to give them a little bit of life. And then there's the crosshair itself: the interaction. There are two parts to the interaction: the player moves the crosshair by moving the mouse, and the player fires the gun by clicking.

For the first part, I just needed a crosshair that moved to wherever the player's mouse cursor might be. I drew a round crosshair in Game Maker's drawing program and made the inside of the circle transparent and the outside black. Now I needed to have it follow the cursor, so I used Game Maker's scripting language to check the mouse's position at every "frame" of the game, then set the crosshair's position to `mouse_x`, `mouse_y` (`x` being the horizontal position of the mouse on the screen, `y` being the vertical).

But I only wanted what's inside the crosshair to be visible. So every frame, after I moved the crosshair, I drew four big black boxes, large enough to cover the screen: one to the top of the crosshair, one to the left, one to the right, and one below. Since we've told the game to check the crosshair's location at every tick of the game's timer, we can easily use that information to plan out where to put the boxes on the screen. An even easier solution would have been to just make the crosshair image twice the size of the screen, with black all around.

The second part of the interaction is left-clicking to fire the gun. Game Maker includes predefined "events" to check player input, such as pressing a key on the keyboard or clicking the mouse. When the player left-clicks, the simple solution would be to tell the "left-click" event to stop the narration, play the gunshot sound effect, and advance the game to the Resolution screen. I wanted something a little more ambitious, though: I wanted the player to have to shoot the family. Clicking away from them should fire the gun, but not destroy America.

What I did was take one of the pictures I had made of the family and trace over it in two colors, one of which I set to transparent, indicating where the gun would miss, and the other, which outlined the family, indicating where the gun would hit. (This is called a "collision mask" in Game Maker.) When the player left-clicks, the game checks where the hidden outline of the family is on the screen, where the crosshair is, and if the two are touching. If they are, the "event" stops the narration, plays the gunshot, and goes on to the Resolution screen. Otherwise, it just plays the gunshot.^{[67](#)}

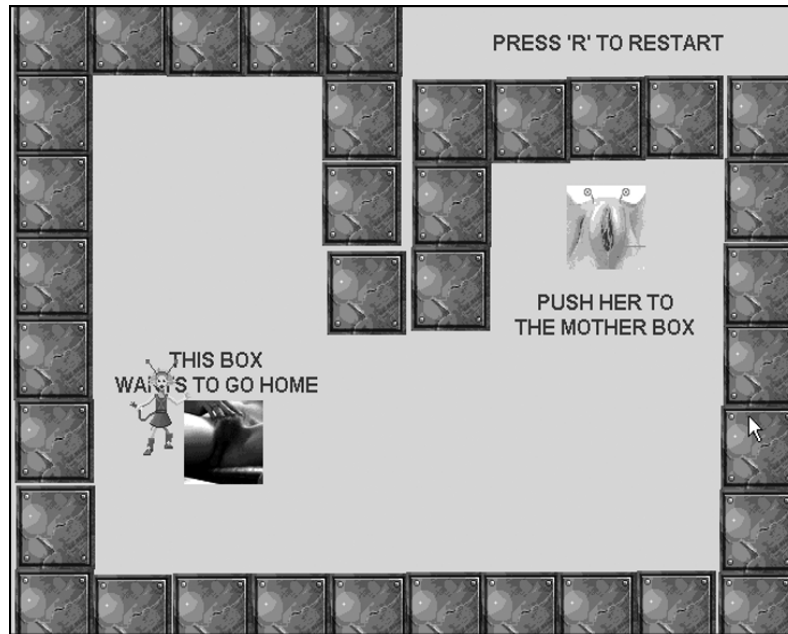
So that's the game: we have a screen that establishes the scenario ("Gay Sniper"), one in which the player takes action (moving the crosshair and

firing the gun, inevitably shooting the idealized hetero family), and a final screen that resolves that action (with the inevitable destruction of America). What makes it a game is the Crosshair screen. It's the player who does what the video implies; she fills the role of Gay Sniper and fires the gun. This is why I want the player to miss if her crosshair isn't over the family: I want the player to have to consciously aim and shoot, to make her more complicit in the fantasy that the West Virginia 4 Marriage video suggests, and that I am forcing the player to act out. By forcing the player to inhabit a political ideology—because that's what games, uniquely, can do—I am pointing out how absurd it is. It's not a complicated game or a long one, but one I thought was worthy of bringing into the world.

Imperfection: Box Pushing Game

In November of 2009 I made a game called *Box Pushing Game* for Klik of the Month. That was a particularly exciting Klik of the Month because some friends of mine were in town and we all got together in the same room to make games, and to play and give criticism of each others' games.

Box Pushing Game was intended as a parody of games like *Sokoban* where the player's goal is to push and arrange a number of crates in a constrained area. (*Sokoban* translates from Japanese as “warehouse manager.”) In *Sokoban* the boxes move along a grid, and the conflict is that pushing the crates around will put them in the way of other crates, forcing the player to plan and manage them carefully.



In my game, the boxes aren't crates but vulvas. (*Box* is old slang for “vagina” and I’m not sure anyone knows why.) It was an idea I thought up in less than a minute during a conversation, and it was the perfect level of dumb for Klik of the Month.

So the ideal version of my game would be one in which the player pushes pictures of vulvas (downloaded from the amateur porn site *I Shot Myself*) neatly along a grid, managing them carefully to ensure that they all reach their destinations. But in the interest of getting the game done in the two hours of Klik of the Month, I was using Klik & Play to make it. And that meant making a lot of compromises. But I could convey the concept of my game—*Sokoban* with hoo-hoos—without all the particulars having to be exact.

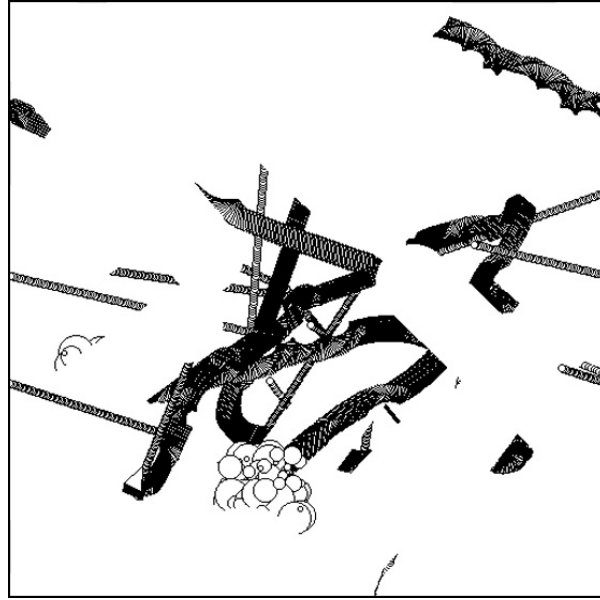
Instead of pushing the boxes along a grid, whenever the player touches a box it just shunts a bit in the direction the player was moving. Instead of being stopped by walls, it just bounces in the opposite direction with a CLANG when it touches one. Instead of having to fill a number of marked spaces with boxes, each level includes “Mother Boxes” (bright pink vulvas with eyestalks), each of which vanishes when a box comes into contact with it, taking the box with it. When all of the Mother Boxes are gone, the player goes on to the next level. And instead of checking whether one box is blocking another—which would take time to figure out how to do

properly—when two boxes overlap, they resolve the collision by wiggling in random directions until they aren't touching anymore.

It's a really clumsy solution to the problem of keeping things from occupying the same space, but it doesn't distract from the idea of the game: push a box with a vulva on it from point A to point B. If anything, it gives the game more character and identity: another Klik of the Month participant described the rule as, "When my box touches another box it goes crazy."

Accidents are creative. At a more recent Klik of the Month that we held on the night of a thunderstorm, my friend Loren Schmidt (hereafter called "Sparky") was working on a quick implementation of Atari's old *Asteroids* game. (The player maneuvers to destroy asteroids by rotating a ship left or right and firing an engine for thrust.) When Sparky called me over to look at it, he hadn't yet implemented a screen refresh—that is, everything that was drawn to the screen stayed there instead of going away. So the player's ship, when it moved across the screen, left a long snake-like trail behind it. Sparky really liked the effect and wanted to keep it.

I told him that he should make the asteroids white, the same color as the background, so that they'd be hidden until they passed over the trails that the player (or her ship's bullets) left on the screen, giving her an incentive to move around and cover as much of the screen as possible (while using bullets to gauge the presence of asteroids in her path). The "accident" of not implementing a screen refresh led to the author finding an entirely unique identity for the game.

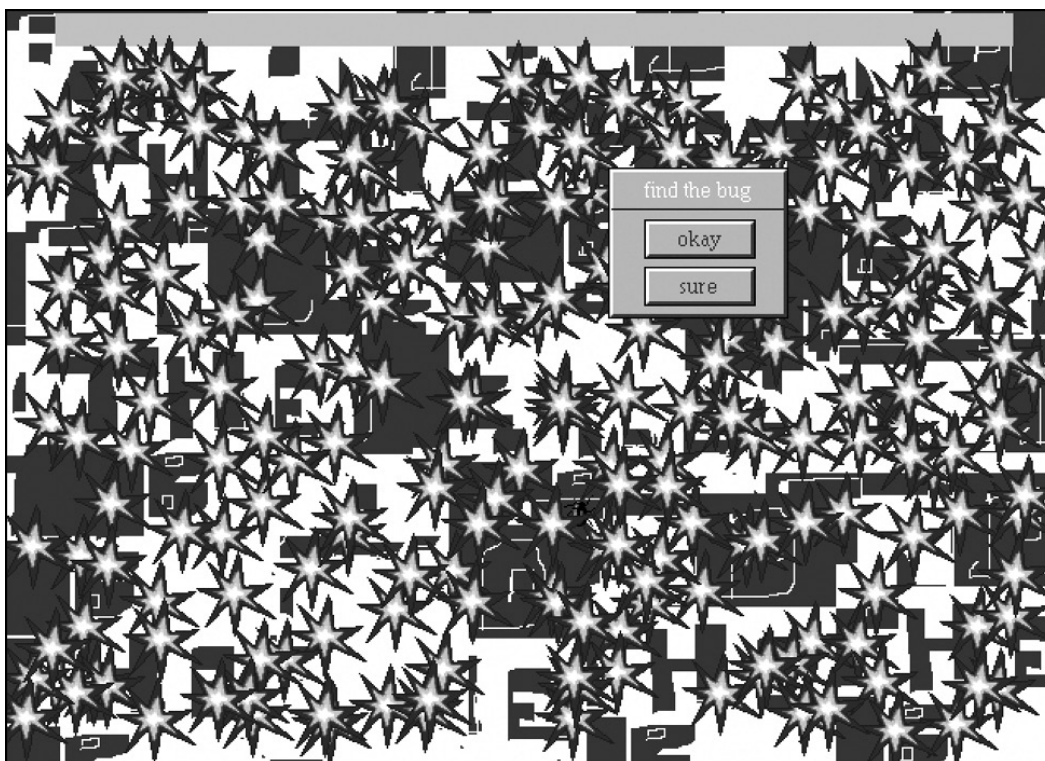


Perfection isn't a useful goal; if anything, it keeps amateurs from getting their feet wet and authors from finishing their works. It's an ideal that hinders more than it serves. Imperfections, creative accidents, and compromises, on the other hand, give a game personality and individuality. Tolstoy's *Anna Karenina* begins with the line: "Happy families are all alike; every unhappy family is unhappy in its own way."⁶⁸ Difference is valuable, and creative accidents and jury-rigs help us achieve it.

Imperfection is an invaluable tool when making games, particularly when making your first games. Think about the ways you can approximate an idea with the tools you have and the things you know you're capable of. Don't worry about minutiae: you can express the idea of your game to a player without everything being exactly as you've envisioned it.

Use What's on Hand: Find Shit

My submissive's first videogame was a Klik of the Month creation called *FIND SHIT*. She had no prior experience with coding or with game making other than looking over my shoulder. How did she make her game? She was using Klik & Play, so she had access to all of the clip art that comes with it. She put a tiny dancing red bug on the screen. Then she covered the rest of the screen with huge red starbursts. The player's goal is to find the bug hidden in the loud, flashing, abrasive screen before the timer runs out.



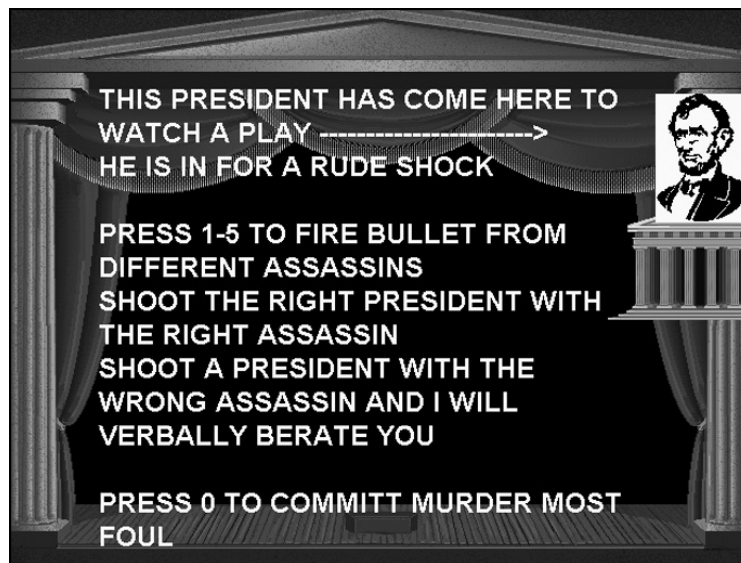
[69](#)

She minimized the amount of scripting she had to do (what a lazy pig) by making clever creative decisions about what she had available to her. The only things she had to script were the timer, clicking the bug to advance to the next level, and the message at the beginning of the level that tells the player to find the bug and asks her to click on a button to start. The buttons are a kind of object available in Klik & Play to allow the author to ask yes/no-type questions of the player. After putting the premade button objects in her game, though, my pig discovered that they're broken: every choice calls the same response. So she made every choice an increasingly apathetic variation on "YES." (For example, the first screen asks the player to "FIND THE BUG," and lets her respond with "okay" or "sure.")

Klik & Play supports WAV audio files, so she put a bunch in. Playing three of them on top of each other, she found, makes Klik & Play totally freak out and begin playing glitchy machine noises. The shrill nightmare sounds perfectly fit the tone of the game, which is already visually abrasive.

Limitations and Creativity: Shoot! Win!

In February of 2010, a friend's fiancé made a game called *Shoot! Win!* This game is a shooting gallery of American presidents: images of Abraham Lincoln, William McKinley, James Garfield (represented by a picture of Garfield the cat), and other presidents bounce around the screen, while images of their assassins (John Wilkes Booth is a phone booth) pose along the bottom.



The player presses keys to make assassins fire at the bouncing presidents, trying to shoot a president using the historically accurate assassin.

When a president is shot, a voiceover explains why that assassin did not historically assassinate that president and chides you for your incorrect choice. (“John Wilkes Booth did NOT shoot James Garfield.”) There’s a unique voiceover for every combination of victim and assassin (with five assassins and five victims, that’s twenty-five vocal samples). These explanations make up the brunt of the game’s content; they’re what make the game interesting and entertaining to play.

All it took to create the vocal samples was a microphone, the historical background, and the willingness to record them. Again, the author keeps scripting to a minimum by making the focus of the game something he can easily create. And because the game is built around components that the author had the means to create, it’s again unique, different from anything that an author who had the time, resources, and ability to perfectly mimic

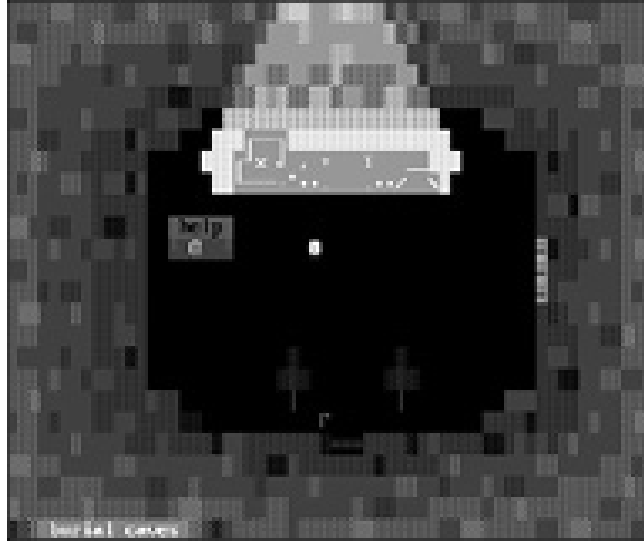
any existing game might have produced. Limitations, both self-imposed and otherwise, guide our creativity.



Working Within Limitations: “Flimsytown” and “Pestilence”

Tim Sweeney’s *ZZT*, created in 1991, is a game-making tool that uses ANSI, the sixteen-color text mode that came preinstalled on PCs of the time, to display all its graphics. This limits *ZZT* authors to a choice of 256 characters—half of which are the letters of the alphabet written in lower and upper cases and using different combinations of diacritical marks—rendered in sixteen possible colors. (Each character can contain two colors, a foreground and a background color, the former for the character itself and the latter for the negative space that surrounds it. Eight of the sixteen colors can only be chosen as foreground colors.) This may seem constrictive, and it is, but authors took the limitations imposed by the program as a creative challenge.

How do you animate a spinning turnstile, for example, in sixteen colors and 256 mostly alphabetical characters? You might have a character that alternates between an X and a cross. A twinkling star could be a dot that blinks as a +. A club (the playing card symbol) could be colored green and made to stand for a bush or tree. The symbol for pi could be a table. Three different “pattern” characters from the extended ASCII character set are similar to different weights of crosshatching, and when combined with a creative mind and an artful use of color can texture a scene impressively.



In fact, the scenes are more impressive for being drawn within the limitations of 256 characters and sixteen colors. Working within those limitations teaches the author to be clever and creative with what she's given in order to make an impression. The limitations of *ZZT* also change the terms of the challenge. An author isn't trying to produce something visually beautiful in comparison to the entire body of contemporary videogames, those produced by small armies of artists and those consumed by millions of mainstream gamers, but rather to produce something visually effective within a small set of limitations and for a much smaller body of peers. This narrowing of the set of possibilities helps the author stay focused and avoid the pitfalls of over-ambition.

The same applies to scripting. Creating a variant of a game like *Lemmings* might not be technically impressive in most programming environments because it's already been done many times, but in *ZZT-OOP*—*ZZT*'s scripting language, which is far less powerful than most programming languages—it's a somewhat astonishing feat.⁷⁰ (That still doesn't mean creating a *ZZT* version of *Lemmings* is necessarily valuable; to do so would be an accomplishment on the order of creating an intricate copy of the statue *Rape of the Sabine Women* in LEGOs. Limitations force you to be inventive, but you can also fall into the trap of pursuing arbitrary technical goals rather than growing as an author and storyteller.) When a small and isolated community consolidates around a game-making tool, creators start to create more and more for each other (because who else is

playing?). Their target audience becomes other creators who possess a detailed mechanical understanding of the game they're working with. As a result of authors creating for one another (and challenging one another), their creations become harder and require players to have more mechanical knowledge in order to play a game at all, until most experiences are too hard for most players.

This is true of a lot of levels made for *Knytt Stories* (Nicklas "Nifflas" Nygren's expandable game about running, jumping, and climbing.) Many of the levels that circulate on Nifflas's online forums⁷¹ are about really tricky, difficult maneuvering. So much of the technical ground of what is possible with the tool has already been explored and so many creators who work with *Knytt Stories* are familiar with the basic and advanced mechanics of the game that creators are naturally drawn toward designing for this group and taking as their starting point something far beyond what most novices using the tool would be capable of easily appreciating.

But this tendency to design for a select group can also push authors to further unexplored conceptual ground. Recently, I played a *Knytt Stories* level called "Pestilence."⁷² In this story, the author has taken the terrain tiles that come with the tool and arranged them seemingly at random to create a broken-looking world. Power-ups that the player would normally expect to collect are locked inside walls, out of reach. The characters who populate the shattered landscape bemoan the hopelessness of their situation as though they were in glitchy videogame hell. (They are, in fact, trapped—the editor for *Knytt Stories* doesn't provide characters with any means of moving around the world or leaving the place in which the author set them.) Though the world seems to have little order, a player who explores will eventually be able to find one of several ways out.

This is the sort of experiment with form that rarely happens in a less restrictive format. With the exception of its music, "Pestilence" is made entirely from parts included with *Knytt Stories*; it's the way in which those parts are arranged that is unique. It's easy to arrange tiles in an unconventional order, but to do so with intention, to build a functioning, solvable world out of them? This is the sort of conceptual ground that, I believe, is usually only reached when possibility-space is finite, and when a lot of that space has already been thoroughly mapped. Think of Los

Angeles versus New York City: one city, with a great deal of continental terrain, expands outward; the other, built on an island, expands upward.

ZZT, unsurprisingly, has an equivalent level to “Pestilence.” It’s usually called “Flimsytown.” “Town of *ZZT*” is the official level packaged with the *ZZT* editor and program. “Flimsytown” starts identically to “Town of *ZZT*,” but quickly diverges, presenting a jarring and confusing experience that requires the player to actually exploit the bugs and glitches inadvertently built into *ZZT* in order to progress. For a while, supposedly, the website *Z2*, the most well-known online archive of *ZZT* levels,⁷³ included “Flimsytown” in its *ZZT* package in place of “Town of *ZZT*.”

Be Derivative: Mighty Jill Off

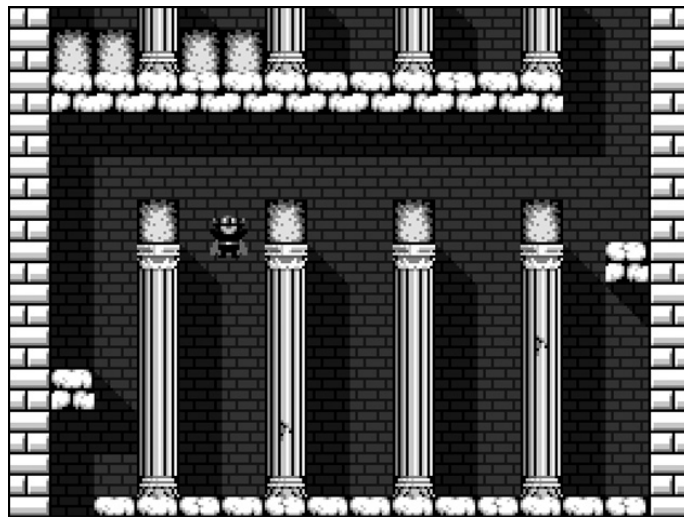
Hi, I’m Der Rivative. Come to see my work? I’ve got this medical paraphernalia here, which I’ve made into little objects. Butt-plug stool—some people say it’s like Nayland Blake. No, it’s mine, Mr. Der Rivative. I made it. Here you have scaffolded video representing my face in six monitors. Some people say Nam June Paik, some people say Bruce Nauman. No, I made it—Der Rivative. I like to lie in bed, as long as I can, let people in the gallery come and see me. Nothing like the Chris Burden on the shelf thing, it’s a Der Rivative extravaganza. I invented it. I made it. People come to the museum and say, “Hey, there’s a Der Rivative.”⁷⁴

—Bob Flanagan

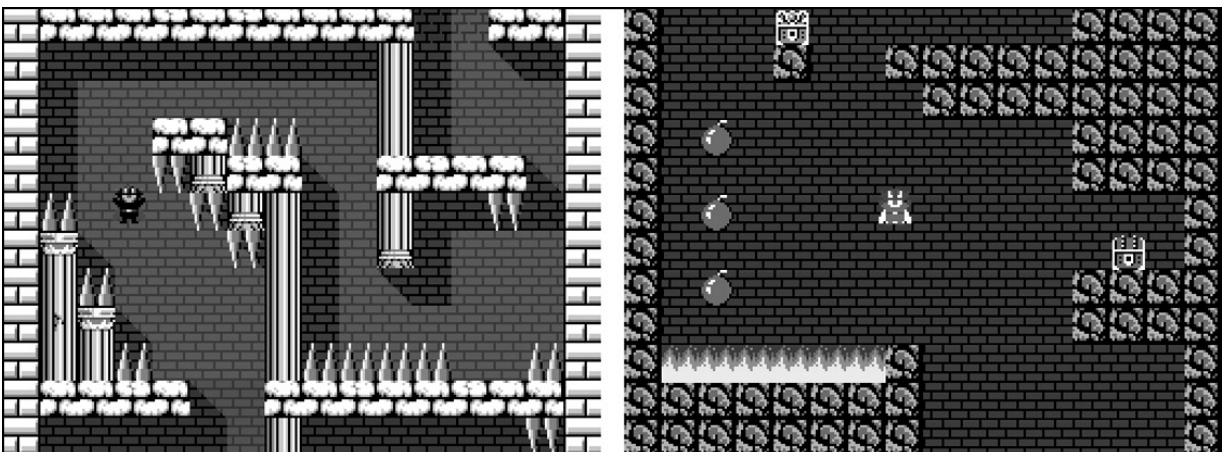
Nothing is original. You’ve probably heard the old quote about there only being seven basic stories (or three, or fifteen; the number always changes) a thousand times. Quantification aside, the purpose of the quote is to make the listener realize that what’s interesting about the story is not the story itself but the way it’s told. There are a lot of novel ways to tell stories with games. We’ve found very few of them so far.

We’re going to find many of them, probably, by accident. Don’t worry about making an original game—just worry about making it yours. Many authors start by trying to replicate games they’ve already seen: *Super Mario Bros.*, *Mega Man*, and *Final Fantasy* are popular. And many of those games end up being very different from their antecedents because the authors didn’t have the technical or artistic skill to mimic those antecedents exactly, and instead had either to approximate the parts of the game they couldn’t replicate or to improvise something entirely new. Or to

incorporate a single new idea that changed something significant about the basic model. Or simply to invest the game with their own personality.



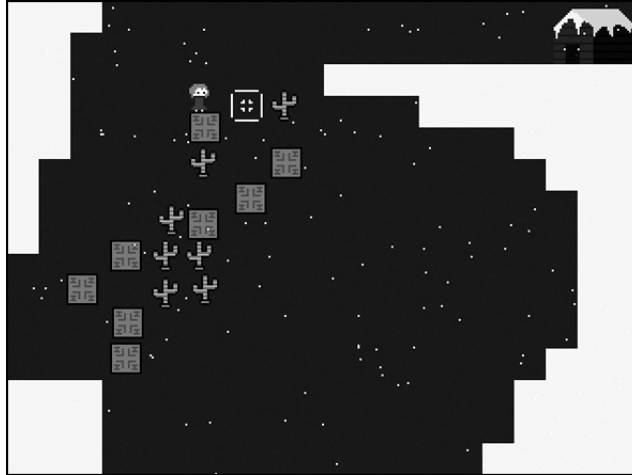
One of my most successful games (in terms of press, not money; the game was released for free on the Internet) is *Mighty Jill Off*. It's based on a Nintendo game called *Mighty Bomb Jack*, published by Tecmo in 1986. *Mighty Bomb Jack*'s predecessor game, *Bomb Jack*, predated *Super Mario Bros.* and presents a very different vision of the way a videogame about jumping can work. While the emphasis in *Mario* is on the planning of a jump—the momentum Mario gains on the runway before the jump, the limited English he can spin on his jump while he's actually in the air—the character Bomb Jack is highly maneuverable while in the air. He can jump the entire height of the screen, reverse a jump on a dime, and flap his cape to keep himself from descending too quickly.



I made *Mighty Jill Off* during the first semester I spent at the Guildhall because I was bored to shit with their lectures and I needed an actual design task to challenge me. For *Jill Off*, I lifted the entire vocabulary of *Bomb Jack*—the high jumps, the reversals, the cape flap—and put it in the service of a story about a submissive masochist climbing a tower to prove herself to her Queen. As a pervert who missed her own submissive (who waited for me in California while I toiled at school in Texas), it was a subject that was both on my mind and relevant to the way I experience games: games about challenge are about the relationship between the player and the designer, the former trying to prove her capacity and will to the latter, the latter trying to continuously challenge the former while maintaining her trust.

Borrow whatever you can. If you were to try to complete a jigsaw puzzle with pieces collected from five different puzzle sets, there'd be blanks that the pieces you've borrowed wouldn't fill in. You'll fill in the blanks with your own personality. You'll smudge the corners in a way that is identifiably yours. And you'll create something new: maybe by design, probably—and preferably—by accident. The modern novel started when Cervantes took the stock idea of “a knight who has adventures” and put him into his own contemporary Spain. Modern music started when bluesmen took traditional folk songs and began to bend the notes. Modern sculpture began when Duchamp took a space in the gallery reserved for artwork and put a urinal in it. Setting out to make something utterly new can be a trap to a new author: you'll spend forever planning and no time doing. If you get busy without worrying about being original, you're liable to stumble across many more interesting ideas.

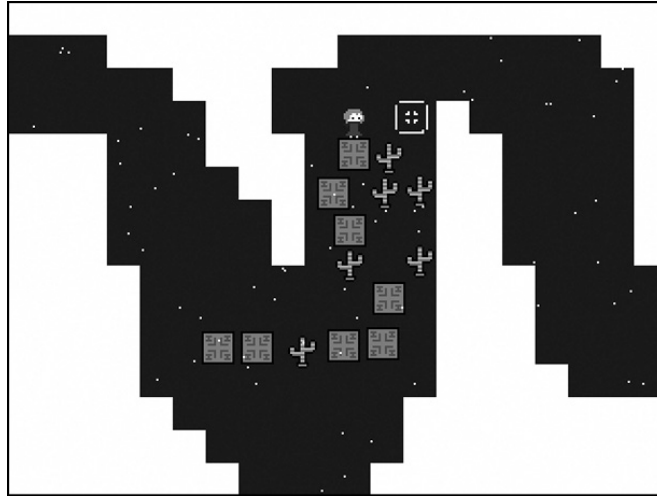
Make Weird Shit: Cactus Block



Put weird shit into your game. Make unusual creative decisions. You're not beholden to anyone, so why be conventional? Whenever you see an opportunity to make something a little more distinctive, take it. These are the parts of the game, after all, that are most uniquely you.

Glorious Trainwrecks user chuchino made a game called *Cactus Block*⁷⁵ in February 2010. This is a simple game about a huddled figure trudging through the snow from the left side of the screen to her house on the right side of the screen. She can jump, but her house is located atop ledges and cliffs that are far too high to jump to the top of. Fortunately, the player has the power to create additional platforms—blocks—by moving the mouse and clicking on areas of open space. The goal, then, is to place the blocks appropriately in order to climb over the obstacles.

This is the point at which the author made a Weird Decision. About half the time, when the player clicks the mouse, a deadly cactus will appear instead of a block that's safe to stand on. (If the player touches the cactus, she must start the screen over.) That's a pretty silly idea, but this is the rule that defines the game.



Because of the unpredictable way that this introduces chance to the player's actions, the player has to plan carefully to accommodate for potentialities. Maybe I click in a less optimal platform location because there's a fifty-fifty chance of my click producing a cactus, and I don't want to block that prime location. Do I take the risk of trying to move forward quickly, depending on a lucky streak of blocks, or do I work slowly, making sure I always have a back-up plan? Do I try the last resort of jumping into the air and clicking immediately under the protagonist, with an equal chance of producing solid ground or instant death? These are all interesting, unique player choices, and they spring from a single unorthodox creator choice.

As a Klik of the Month game, *Cactus Block* was made in two hours. It's not the same case as a big-budget commercial game, where lots of money and hours have been invested in a project, and where a publisher expects a return on its investment. If you do something wacky in a two-hour game and it doesn't quite work out, you've gambled and lost just two hours of your time. And I use the word *lost* reluctantly, because I don't consider any game a total loss, even if the results don't work very well. A "loss" here means that you've made something unique and that you have that much more experience putting a game together.

Beyond the potential for interesting design generated by making something weird, any opportunity to do something weird is also an opportunity to put some of yourself in a game. Most of my games star perverts, even if all the perverts are doing is jumping up a tower. Some

hobbyists try to emulate what they think of as “professional” game development and divorce all aspects of themselves from their games. That’s ridiculous! Your game comes from you; your game should contain you. A game you make should have your signature on it. It should reflect something of your personality, your humor, your values. After all, you’re only beholden to yourself. Make something that’s your own.

What to Make a Game About?

Your dog, your cat, your child, your boyfriend, your girlfriend, your mother, your father, your grandmother, your friends, your imaginary friends, your summer vacation, your winter in the mountains, your childhood home, your current home, your future home, your first job, your worst job, the job you wish you had.

Your first date, your first kiss, your first fuck, your first true love, your second true love, your relationship, your kinks, your deepest secrets, your fantasies, your guilty pleasures, your guiltless pleasures, your break-up, your make-up, your undying love, your dying love.

Your hopes, your dreams, your fears, your secrets, the dream you had last night, the thing you were afraid of when you were little, the thing you’re afraid of now, the secret you think will come back and bite you, the secret you were planning to take to your grave, your hope for a better world, your hope for a better you, your hope for a better day.

The passage of time, the passage of memory, the experience of forgetting, the experience of remembering, the experience of meeting a close friend from long ago on the street and not recognizing her face, the experience of meeting a close friend from long ago and not being recognized, the experience of aging, the experience of becoming more dependent on the people who love you, the experience of becoming less dependent on the people you hate.

The experience of opening a business, the experience of opening the garage, the experience of opening your heart, the experience of opening someone else’s heart via risky surgery, the experience of opening the window, the experience of opening for a famous band at a concert when nobody in the audience knows who you are, the experience of opening your mind, the experience of taking drugs, the experience of your worst

trip, the experience of meditation, the experience of learning a language, the experience of writing a book.

A silent moment at a pond, a noisy moment in the heart of a city, a moment that caught you unprepared, a moment you spent a long time preparing for, a moment of revelation, a moment of realization, a moment when you realized the universe was not out to get you, a moment when you realized the universe was out to get you, a moment when you were totally unaware of what was going on, a moment of action, a moment of inaction, a moment of regret, a moment of victory, a slow moment, a long moment, a moment you spent in the branches of a tree.

The cruelty of children, the brashness of youth, the wisdom of age, the stupidity of age, a fairy tale you heard as a child, a fairy tale you heard as an adult, the lifestyle of an imaginary creature, the lifestyle of yourself, the subtle ways in which we admit authority into our lives, the subtle ways in which we overcome authority, the subtle ways in which we become a little stronger or a little weaker each day.

A trip on a boat, a trip on a plane, a trip down a vanishing path through a forest, waking up in a darkened room, waking up in a friend's room and not knowing how you got there, waking up in a friend's bed and not knowing how you got there, waking up after twenty years of sleep, a sunset, a sunrise, a lingering smile, a heartfelt greeting, a bittersweet goodbye.

Your past lives, your future lives, lies that you've told, lies you plan to tell, lies, truths, grim visions, prophecy, wishes, wants, loves, hates, premonitions, warnings, fables, adages, myths, legends, stories, diary entries.

Jumping over a pit, jumping into a pool, jumping into the sky and never coming down.

Anything. Everything.

Get Your Hands Dirty

So get to it. Make a game. Your first game will be rough and derivative. It may not change the world, but it will be yours. The next chapter will be an attempt to walk you through your first game step by step. It's intended to be as abstract as possible so that you can apply it to whichever game-

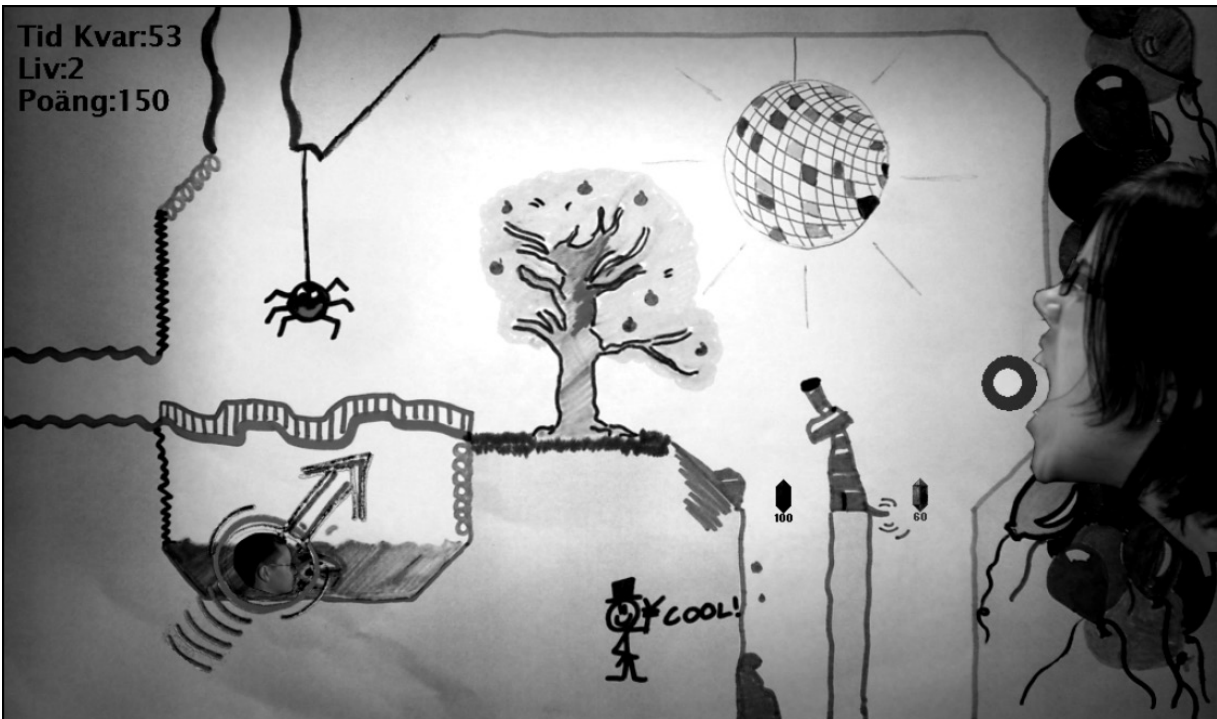
making tool suits your needs best. You can find out the details of making that tool do what you want through reading the documentation, asking questions of the community surrounding that tool, or best of all, fiddling with the tool to see how it works.

As for what tool to choose: Appendix A lists a bunch of different game-making tools that are available, describing what they're good for and what the experience of working with them is like. It's by no means a complete overview of all the game-making software out there—there are far more tools than anyone could document—but I've included those that I think are the most useful and approachable. There will also be tools that are created after this book is published. Nothing that I'd consider the ideal game-making tool exists yet, but I'm confident we'll keep getting closer.

It's possible that your interest in digital game creation is purely academic and doesn't extend to becoming an author. In that case, I hope that what you take away from this book is that the videogame isn't the creation of a corporation, but of an author, that this form is important, and that people are using it to do exciting things.

In 2008, Nicklas Nygren (a.k.a. Nifflas, author of *Knytt Stories*) attended a festival called EDGE in Umeå, Sweden, with a laptop, a printer, and a digital camera. He asked other festivalgoers to contribute to a game he was working on. They drew pictures, photographed themselves, and sang into a microphone. Over the course of the day, he created a game called *Det Officiella Edge-Dataspelet*, or *The Official Edge Videogame*.⁷⁶ Every image and sound in the game was created by someone Nifflas had pulled from the crowd: the protagonist is a photo of someone's head in profile, the stages are drawn in crayon, the music and sound effects were made by strangers' mouths.

This is what game creation is becoming: small, personal, and made by people's own hands and mouths instead of by corporations with teams, managers, and software renderers. What we call a videogame is not a product. It's the creation of an author and her accomplice, the player; it is handmade by the former and personally distributed to the latter. The videogame is a zine.



Footnotes

⁶³ You can watch the video online at <http://www.youtube.com/watch?v=5vQm0hJ8Dd0>.

⁶⁴ Although the concepts I'll talk about can be implemented using a wide variety of tools, I'll use Game Maker's names for the concepts where appropriate in this section.

⁶⁵ <http://audacity.sourceforge.net>.

⁶⁶ http://www.drpetter.se/project_sfxr.html.

⁶⁷ Collision detection, telling whether two things are touching—in this case, the mouse and the family's collision mask—is pretty tricky, because it involves comparing a lot of values, yet it's a basic part of most game interactions. GameMaker lets you accomplish it really easily. Every game-making tool you use will allow you to check (or at least improvise a way of checking) whether two things are touching, but some tools will make it harder than others.

⁶⁸ I totally stole the idea of repurposing this quote from Jared Diamond.

⁶⁹ There's a video of *FIND SHIT* being played at http://www.youtube.com/watch?v=zPssxEE_qnE.

⁷⁰ Lemmings for ZZT exists, naturally. See John D. Moore's *Zem!* games.

⁷¹ <http://nifflas.lpchip.nl/index.php?PHPSESSID=ebf1b8ac1a8a3fc1f5109d79a1813537&board=39.0>.

⁷² It can be found on Nifflas's forum at <http://nifflas.lpchip.nl/index.php?topic=1942.0>.

⁷³ <http://zzt.org>.

⁷⁴ From *Sick: The Life and Death of Bob Flanagan, Supermasochist*, DVD, directed by Kirby Dick (Santa Monica, CA: Lions Gate, 1997).

⁷⁵ <http://www.glorioustrainwrecks.com/node/451#comment-3591>.

⁷⁶ <http://nifflas.ni2.se/?page=Miscellaneous>.

Chapter Seven

By Your Bootstraps

This chapter is here to help you through your first game. It's not intended as an instruction manual, as an encyclopedic reference, or even as a step-by-step guide. Rather, it's a series of tasks designed to stimulate you creatively, to get you thinking about game design decisions, and to actively involve you in working on a game—which is the most important part. Lots of would-be game creators spend such a long time planning dream games that their ambitions are far too large for them to achieve, or even to know where to start. For others, the whole notion of making a videogame is alien and it's difficult for them to know how to even approach the work from a technical perspective. The steps of this chapter attempt to introduce concepts of game making in a less intimidating way.

The tasks are purposefully left abstract so that they can be applied to any tool you choose to work with. This means that I'm not providing specific instructions for how to make a character move around in Game Maker, for example. Figuring that out is up to you. Fiddle around, read the documentation, ask questions online. By working it out yourself, you'll start to learn your way around the tool of your choice.

Some tools are very different than others. Twine and Inform, for example, are for making games presented largely through text alone. You'll need to be creative in adapting the tasks below to fit the kind of game you can make with those tools. Remember that these tasks aren't instructions: they're just here to point you in the right direction. Diverge from them as often as possible; do them out of order; ignore the ones that don't fit your vision; leave them behind once you've found your own way. This is just the start of the path. The end is everywhere.

Task #1: Choose a Tool

The first step is to pick a tool—either one that sounds like it would be useful for the game you have in mind, or one that seems interesting to you

and that starts you thinking about what kind of games you might like to make, or that suggests possibilities that your game might explore.

To make the kind of games most people intuitively think of when they think of videogames, The Games Factory (for Windows) or Game Maker (for Windows or Mac) are strongly recommended for being fairly robust. They're capable of crafting a great variety of games that look and play differently without much difficulty. Game Maker is probably the best-documented tool of its kind, but the current version (rebranded "GameMaker" by the soulless corporation that currently controls the intellectual property) makes it difficult and unrewarding to self-publish. But starting with an older version of Game Maker that publishes games for free is a good idea.^{[77](#)}

If Game Maker and The Games Factory are too intimidating, try Scratch. Scratch provides a good introduction to thinking about games in terms of rules, an essential habit for more advanced game making. Every rule in the Scratch editor is essentially a sentence made up of smaller phrases, which helps you understand how rules create the actions in a digital game. There's also a tool called Stencyl that uses the same concept as Scratch, but produces Flash games for online distribution.

I'm also partial to Twine for games that are closer to written fiction. Though you can create projects that include images and other graphical elements if you have knowledge of HTML and CSS (in which Twine is based), Twine is essentially a tool for creating text-only games. It's intended for making branching stories in the vein of Choose Your Own Adventure books, and has a clean visual presentation that shows how passages in the story are connected. Other text game creators like Ren'Py and Inform 7, described more fully later, provide other simple options for creating this kind of game.

In the next chapter you'll find a fuller list of the tools that I think will make the most sense to people who have no programming or game making experience. I've tried to describe what each tool is good for and what the experience of working with each tool is like. Try more than one. You can always learn to use a new tool. Each one will teach you something different about design.

Task #2: Introduce a Character

A character could be abstract or concrete. It could be a person, an animal, a symbol. It could be represented by a photo—one of yourself or of a loved one or of a stranger that you’ve grabbed from Google’s image search. Mold a character out of clay and take a picture. Open up MS Paint and doodle a cartoon. Find a picture of an American president and draw horns on it. Or just use an image file that comes with whatever tool you’re using. Klik & Play includes lots of clip art, and Stencyl lets you grab animated characters from an online database.

Take this image of your character and find a way to start a game and display the character on screen.

Once you have that, imagine what this character might be capable of doing, and how that will characterize her. It can help to think of this in terms of a question: what job does this character have? Think about what kind of conflict the character might have: what makes it difficult for her to do that job? I once made a game about a robot gardener: her conflict was that coming in contact with water would cause her to short circuit.

Let’s say your character starts from a picture of one of your friends, grabbed from Facebook. In the photo your friend is wearing a sombrero. Why would this character you’re creating wear a sombrero? Perhaps she’s a MATADOR. What forces come into conflict with a matador? Perhaps she’ll have to face some RAGING BULLS. A story starts to develop from a single picture that you’ve taught your program to display on the game screen. (Or the story starts to develop and you change the picture to fit the new story.)

If you’re working with something text heavy like Twine or Inform or Ren’Py, you can, instead of using an image, write an opening passage that characterizes the protagonist, what she’s capable of doing, and what her relationship is with the world. In either case—text or image—this is the first piece of information the player will see when she loads your game. So you should choose something that tells her about the game and its world.

Task #3: Teach Your Character to Do Something

Specifically, teach your character to do something in response to the player. This means teaching the game to TAKE INPUT. The player should

be able to press the spacebar, or click the mouse, or type the letter A, or *something*, and your character should respond to that input somehow. Maybe your character turns invisible while the SHIFT key is held down. Maybe your character flies up the screen when the player hits the UP arrow key. Maybe when the player clicks somewhere on the screen with the mouse, the character teleports there.

As you do this, think about how what you're teaching the character to do characterizes her. How does being able to teleport anywhere on the screen fit her character? What does that ability tell the player about her? Maybe she's not just any matador, but a MATADOR FROM THE FUTURE. Maybe the raging bulls she comes into conflict with are charging from one side of the screen to the other, and she has to teleport to stay out of their way.

This is your game's first rule. When the player does X, the protagonist does Q. This is the kind of rule I like to call a "verb," because that's the part of speech this rule would correspond to in a sentence describing your game, one that governs the relationship between the player's character and the other characters in your game. In other words: FUTURE MATADOR (subject) TELEPORTS (verb) to avoid RAGING BULLS (object.)

If you're working on a text game, write a second passage that describes a location, be it a physical place or a state of mind, so long as it's different from the one the game starts in. Maybe the player starts in a desert, but she can see an oasis in the distance. This second location can be the oasis. Now give the player some means of getting her character to the new location.

Task #4: Introduce a Second Character

This is the object of the previous sentence. This character should have a relationship to the first character, who we'll start calling the player for short. Maybe the second character represents something the player wants to avoid (raging bulls) or something she wants to acquire (a bag of cash). (Or maybe she wants to avoid the corrupting influence of the bag of cash and touch the confused raging bull, gently, in order to calm it.)

The relationship between the characters should have something to do with the player's verb. For example, if the player can FLY UPWARD and

the object is THE SUN, which the player wants to touch, then writing the game's rules such that the player can fly upward high enough to reach the sun will change the experience of the game. If the player can fly, but never high enough to reach the sun (or perhaps the player can only fly straight upward, and the sun is a little to the left or right), you'll have made a game about unattainable dreams, ones that remain just out of reach. If the protagonist can reach the sun but it burns her wings off, causing her to fall back to the earth and the game to end, you'll have made a cautionary fable. If she can reach the sun, put it in her pocket, and earn one hundred points for it, you'll have made a power fantasy.

Rules change not only the meaning but also the feeling of a game. In our future matador game, maybe you'll decide that a raging bull always moves left and right across the screen, or instead, that it always moves toward the protagonist. The latter bull is probably more difficult to avoid than the former. Once you have your rule in mind, experiment with different ways of implementing it to find a feeling and balance that you like.

It's most likely that you want this character to abide by a rule that's internal to the game—that is, one kept by the computer, not controlled by the player. You could, of course, experiment with giving the player two characters to control in different ways (maybe one of them is a mirror image that moves left when the other moves right), or with making two different characters that are controlled by two different human players.

If you're writing a text game, introduce an object to the story, one whose relationship with the player introduces a conflict. Maybe when the player reaches into the oasis for a drink, she discovers what appears to be an old, waterlogged map.

Task #5: Make Some Noise

Put a sound effect in your game. Think about which interactions in your game would be clarified by the playing of a corresponding sound. For example, if our matador makes a sci-fi BZZIP sound when she teleports, it would definitely characterize her as a matador that uses future technology. And it would act as punctuation to the teleport, helping the player to understand that clicking to move the character isn't a mistake, but what she should focus on. The bull striking the matador might be accompanied

by a drawn-out scream to let the player know that it's an undesirable outcome (or a choir of angels singing if the future matador secretly seeks a glorious death in the ring).

How do you find sounds to put in your game? Open the sound recording program that comes with your computer (or download a free one, like Audacity⁷⁸) and record yourself making sounds into your computer's microphone. Try shifting the pitch or the tempo, reversing the sound, adding a thousand phase or distortion effects. Andrew Plotkin made all the sound effects for his 1994 shareware game, *System's Twilight*, by recording himself making silly noises into a cheap mic. You can even download his sounds if you want; he's made them available.⁷⁹ You can find plenty of sounds online, at sites like Freesound.org. And many tools, in particular Game Maker and Klik & Play, come with many of sounds for you to use. There are programs that will let you generate your own sounds directly, without a mic: a free program called SFXR⁸⁰ lets you easily generate Nintendo-like sounds.

In a typical videogame, the screen communicates a lot of visual information. In a busy game, it can be hard to follow absolutely all of what's happening on the screen. Sound communicates on a different channel, giving you a separate and sometimes more immediate layer of information. Use sound to underline the important relationships in your game.

If you're making a text game, the tool you're using may not have an easy way to play sound, or playing a sound effect might be jarring in a game that's all read. Your story might not need sound at all. You might try fiddling with some kind of background ambiance, like a loop of birds chirping or water running that only plays at the oasis, to differentiate that location from the desert. Or maybe a quick sound like a SPLOOSH of water or a CHIME of discovery upon pulling the map out of the water.

Task #6: Round Out the Player's Vocabulary

Give the player's character more verbs; give the player more ways to communicate with the game. Try to think of new rules that will interact with the existing ones in interesting ways. For example, what if you make your future matador into a future cowboy? In addition to being able to

teleport, she can also drop a ZAPPO LASSO, which will catch bulls that run across it and return them to the future. But maybe being able to get rid of the bulls makes the game feel too easy, so maybe she can only drop the lasso where she's standing. This means that in order to trap a bull, the player will have to use her teleport to put her in the path of the bull, set the trap, and teleport away again. The conflict escalates.

In the case of the Icarus character who flies UP to reach the sun, maybe she can move LEFT or RIGHT as well. If she moves off the left or right side of the screen, she'll travel around to the other side of the planet, where it's night and where she can land safely on the moon instead. (The transition can be as simple as changing the background from a sky blue to a night black, and the sun to the moon.)

Think about other rules you can add. Rules don't only have to be verbs; rules can also be *adverbs*. Your future cowboy might be able to teleport SLOWLY. In practice, this means that it takes a few fractions of a second between the player clicking the button and the cowboy arriving at her destination, which forces the player to plan more carefully and react more quickly. Maybe Icarus is affected by gravity, and whenever she's not flying up, she's falling slowly down. You could give her the ability to move up, left, and right when the player presses buttons, but she has to depend on gravity to move down.

Try to ensure that all the rules in the game have relationships with each other. If you think of them as a vocabulary—nouns, verbs, and adverbs—try to construct a vocabulary with which you can tell engaging stories.

In your text game, give the player a way to interact with the object she's encountered. Maybe if she tries to read it, she's told it's too waterlogged to make sense of. Maybe she has to take it back to the desert to dry it off. The way she interacts with the object should advance the story. Now that the player's returned to the desert, maybe the dried map leads to a treasure hidden somewhere in the sands.

Task #7: Design a Level

I'm using the term *level* to mean a sequence of events the player has to negotiate using her vocabulary of verbs. In other words, a level is a story: the tension that rises before the climax of the game resolves it.

A story doesn't have to be complicated. Levels for *Future Cowboy* could just be an escalation. Every time our cowboy catches a bull, it enrages two new bulls who charge onto the screen, causing the bull population to gradually increase with the player's performance, and making the bulls harder for the player to manage. This is rising tension too, and each new bull on the screen is a new chapter, ultimately telling a story about human performance against overwhelming odds (or whatever you like).

However, some stories you want to tell might demand more complicated levels. What if the sun our Icarus wants to reach is blocked by storm clouds that will electrocute her if she touches them? The storm clouds make the sun a more valuable goal—even though, in the version of the game where Icarus burns up when she touches the sun, it's a negative goal—because they require Icarus to work harder in order to reach it. They also force the player to recognize Icarus's ability to move left and right, making it more likely that she'll decide to fly Icarus off the edge of the screen, and thus more likely that she'll realize that the moon might also be a valuable goal. You can push this possibility further by leaving clues: plant a trail of storm clouds along the side of the screen, leading to the edge, or—to be even more obvious—make the side of the screen a wall of clouds with an Icarus-sized hole in it.

Once you've gotten the player on the night side of the planet, you might give her a trickier maze of clouds to navigate, cluing her in that the moon goal is more desirable. Now you're starting to tell a really interactive story: the player can take the easier and more obvious route of flying to the sun and being burned, or she can take the more difficult and oblique route to discover this newer goal. (Notice that I haven't said anything here about telling the story explicitly, having Icarus shout out a text bubble saying, "I MUST REACH THE GLORIOUS MOON!"—if done carefully, letting the player discover the moon on her own by accident or by following clues, the player will get the idea, and consider it that much more interesting and personally relevant because she discovered it herself). The most difficult part of the story then becomes the climax: now that the player's goal has changed, she uses all her skill and patience to reach it.

If navigating the storm cloud maze around the moon is the climax of the game's tension—then maybe on reaching the moon Icarus arrives at a

third, easier scene that can serve as the game's denouement. On this screen the player just has to guide Icarus to a Greek-columned house sitting at the bottom of the screen, moving left and right to avoid obstacles (maybe they can be hot stars instead of storm clouds) while gravity pulls her downward. This adds some variety and makes the scene distinct. What literary critics call the "falling action" of the story could be Icarus literally falling toward the surface of the moon and the game's conclusion.

Notice how this story was entirely told using our basic verb set: all Icarus is doing is flying, but we've given her interesting goals that cause her to use her verb set in interesting ways. Although the *Future Cowboy* story is maybe less complex, it's still a story entirely told through the player's two major verbs—TELEPORT and LASSO—and the relationship they put her in with the objects in her world. Plan a story that will develop the player's relationship to the rules of your game, especially her verb set.

If you're writing a text game, give the player somewhere to go. Maybe the map says something like, "Go west until you reach a mushroom-shaped rock, then travel north until you find the skull of a giant wizard, then head east to the secret cave." Maybe there are things for the player to do at these places, maybe not. But they should be at least interesting to visit: make them distinctive, write them descriptions that are evocative or silly or weird.

Task #8: Finish the Story

The beginning and resolution of a game do a lot to tell the player how to think about the experience in between. Think about *Gay Sniper*, which I described in the previous chapter. The opening screen—with the words "Gay Sniper" and the opening tinkle of a designed-to-tug-at-your-hetero-heartstrings public service video—and the closing screen—"America is destroyed"—do a lot to characterize the absurdity of the political message they bracket.



Not every game needs a title screen, but a title screen is very good at giving the player a context with which to understand the experience that follows. Think about what the first thing the player sees tells her about the game she's about to play. The title *Future Cowboy* goes a long way toward characterizing the rules of that game: along with whatever art you choose, the title gives a context both to the teleportation and to the bull-trapping in that game. If the title screen of *Icarus* shows a picture of Icarus on the ground, reaching toward the sun, that effectively conveys to the player that the sun should be Icarus's goal—and also makes the player more excited when she realizes that the title screen is not being entirely straightforward with her about what the game's true goal might be.⁸¹

The way your game ends is the last thing the player will see, and will similarly have a great effect on how she interprets the experience as a whole. The ending could resolve the story or subvert it. *Icarus* could freeze to death away from the sun's heat, or might live happily ever after soaring in the lower gravity of the moon. If the game program closes, it lets the player know she's reached the resolution of the story and there's no farther to go. But what about the other game-ending scenario in the *Icarus* game, when Icarus touches a storm cloud and is electrocuted? When this happens, the game could restart rather than close, telling the player she hasn't found the resolution of the story yet. Getting burned by the sun could lead to the game restarting as well, to clue the player in that there's another, more desirable resolution.

In the text adventure cave, maybe the player finds something valuable or maybe something useless. Maybe, after traveling so far from the oasis, she

frees a genie who offers her “the greatest gift one can give in the desert: water!” Maybe to enter the darkness of the cave, she has to set the map on fire and use it as a torch, destroying her means of finding her way back. Whatever she discovers in the cave, the text should make it clear to the player that this is the story’s resolution: she’s found the ending.

Think about how the beginning of your game, what the player experiences during your game, and the ending(s) of your game form a story. What story are you telling, and how will the beginning and ending help provide the context you want for the game?

Task #9: Have Someone Play It, Then Change It

You should be playing and replaying your game with every change that you make to see if it works in the way you expect. But you should be having other people play it, too. Because of your perfect knowledge of how the game works and what it contains, you can’t get an accurate sense of how a player will experience your game from your own plays alone. You’ll get the most information about what the player’s experience actually is by having people who are unfamiliar with the game play it.

Get as many people as you can to play. Try to have an idea of how experienced these players are with videogames to give you some idea of how players with different backgrounds will react to your game. I usually try to have the widest variety of players as I can, and to have people play the game both while it’s being constructed and after I consider it finished. Often I’ll have different people play the game at different stages, so that I can ensure that players playing the “finished” game are seeing it for the first time.

If you’re still putting the game together, give your player an idea of where you want to take the game so that she can give you ideas for how to take it there. If it’s finished, don’t tell her anything. Watch her play the game and don’t make any comments or assist her unless she asks. Make notes on what she does first, where she gets hung up, whether she reacts as you expected, at what points she gets confused. Also make notes on anything that doesn’t work as it should—and if any interesting consequences result when things don’t work as they should. Someone less familiar with the game than you is more likely to stumble upon

mechanical errors because she doesn't know the range of possibilities in the game. What if the player flies Icarus off the top of the screen and disappears? Nothing prevents the player from doing so because you never expected her to: you knew exactly what the goal was, so in all your plays of the game you've never flown off the top of the screen.

When you sit someone down to play the game, keep in mind specific parts of the game that you're uncertain about. Watch how she deals with them, then after she's done, ask her what she thought and whether those parts of the game worked or not. She may have some suggestions of how they or the game as a whole can work better.

Now that you have a better idea of how the player experiences your game, change it! Fix errors, smooth snags, add whole levels if you need to, implement some of the ideas your players suggested, and see if they improve the game. Have them play the game again. Have someone new play the changed game. Change everything, have the original player play again, and see which version they prefer.

Task #10: Distribute Your Game

What's the use of a game that no one can play?

Before broadband Internet, small game authors either had to find a publisher to pay the cost of manufacturing the games and putting them on store shelves, or they had to come up with clever solutions—shareware, which we talked about in chapter 2, being a popular one—for distributing their games. You, however, can take advantage of the speed and accessibility of the Internet to digitally distribute your game.

Upload your finished game to a free file sharing website like MediaFire.com. Some tools, such as Stencyl, have official websites where authors can make their games available. Start a free blog to keep track of all your games; put them on your ancient Myspace page; ask your significant other to upload them to a secret directory of the web server at her job. All you need is some web space for the files and an address you can send people a link to.

Once you have that, spread the link around! Put it on your Facebook page, on your e-mail signature, on a postcard you staple to local telephone poles, everywhere. E-mail it to IndieGames.com. A good place to start:

most tools I talk about in this book have communities with attached forums where you can show your game to peers. Post your link there, then read any feedback or criticism you receive (even though most of it will be dumb). Keep it in mind when you make your next game.

Just because digital distribution makes things easier, however, doesn't mean you shouldn't be creative about passing out your game. A friend of mine who puts out his games under the label "Amon26" used the online publishing site Lulu.com to sell CDs containing two games of his that had previously been available on the Internet, along with scans of concept art and MP3 files of the soundtrack.⁸² Rob Fearon, a British author and father, asks a one-time donation in the amount of the buyer's choosing for a collection of all of his games.⁸³ Burn your game to CD and sneak it onto a store shelf, leave a basket full of free copies at a local coffeehouse.

Task #11: Make Another Game

Do all of this again, using what you've learned from your first game. Make an entirely different game, use an entirely different tool, use completely different verbs than you used before. Or don't: make the same game over again, but slightly better: *Icarus 2*, with day, night, and twilight levels.

You're a zinester, after all. Whatever you're doing is right because you're doing it, and that's valuable. Don't worry about being brilliant or original—just make sure you're creative.

Footnotes

⁷⁷ You can download this older version at <http://www.auntiepixelante.com/?p=1240>.

⁷⁸ <http://audacity.sourceforge.net>.

⁷⁹ <http://eblong.com/zarf/twilight.html>.

⁸⁰ http://www.drpetter.se/project_sfvr.html.

⁸¹ You might also be clever: if the design and presentation of the game *Icarus* is exactly as described above, but the title screen shows a picture of Samuel Gompers and bears the title *The Story of Organized Labor in America 1886–1894*, the entire experience essentially changes. A little of this goes a very long way.

⁸² <http://www.lulu.com/product/cd/and-you-will-be-fit-for-more-than-ashes/6053570>.

⁸³ <http://bagfullofwrong.co.uk/bagfullofwords/whats-in-bundle-of-wrong>.

Chapter Eight

Growing Up

When I was little I was enchanted by videogames. Kids will doodle and sketch, and like a lot of kids, I sketched videogames. I would draw potential videogame levels, imagine my own versions of *Super Mario Bros.* where Mario had to collect a magic sea anemone instead of a magic flower to shoot fire underwater. I would draw my own videogame characters, cut them out (or, when I was really young, have my mother cut them out), and then move them through my levels and imagine what my own videogame would be like to play.

There never seemed to be a way to take it further than that, though. I remember the excitement when my parents bought our first family computer: I knew, somehow, that people used computers to make videogames. I was never able to unlock the secret, though. I learned some QBasic, but I couldn't imagine how to get from writing text on a screen to the games of my imagination.

I shelved my childhood ambition of making videogames. I tried to write fiction, because that was something creative, but I sputtered out, got sick of it, and dropped out of college. Though not before discovering Game Maker.

I was lucky: I left school just as a whole generation of new game-making tools for people like me—people who didn't program, people who hadn't worked in the games industry—were becoming available. But the really lucky ones are the generation who are growing up with access to tools like Scratch and Stencyl. For these kids, game creation won't be the mystical, inaccessible thing it was for me. It'll be something they can actually touch. They'll be able to doodle playable games as easily as doodling a comic or writing a simple story. Imagine what this generation will do with videogames once they've grown up.

What videogames need right now is to grow up. The videogame industry has spent millions upon millions of dollars to develop more visually impressive ways for a space marine to kill a monster. What they've invested almost nothing in is finding better ways to tell a story, and in

exploring different stories to tell. That's for us to do: the people who don't have to sell thousands of copies of a game to break even, who aren't obliged to fill their games with eighty hours of content, who are beholden to no one, who are free to be silly and weird and creative and personal. Hobbyists and zinesters. You and me.

Every game that you and I make right now—every five-minute story, every weird experiment, every dinky little game about the experience of putting down your dog⁸⁴—makes the boundaries of our art form (and it is ours) larger. Every new game is a voice in the darkness.

And new voices are important in an art form that has been dominated for so long by a single perspective. Engineering students and venture capitalists have given us valuable pieces of culture, but there's more to the human experience than orcs, elves, and wish-fulfillment power fantasies. If people don't take videogames seriously, it's because, as an art form, they tell us very little about ourselves, so far. But authors outside of the mainstream—those who haven't spent fortunes to bring their works into the mainstream—have revealed much more. They have shown us a new perspective through their unconventionality, their creativity. They have shown us new ways for games to use rules, new ways of giving the players liberty to act and play within those rules, new ways to create experiences that are unique to games.

Digital games—through their ability to keep complex systems of rules, their ability to present audio and visual information, and their reproduceability—have enormous potential for telling stories. So let's tell some stories! I want to hear about your hopes, your dreams, your fears. I want to hear about what it was like to put down your dog or to fall in love or to realize that something you thought controlled you holds no real sway over your life. I want creative people to take a creative form and do amazing things with it.

I want zinesters to find new, inventive ways to distribute games. Fuck Steam and the App Store! Let's invent new networks for letting people sell their games online. Let's bring back CD compilations of games, available at your local co-op or coffee shop or by mail. And let's take advantage of what our predecessors, the shareware authors, never had: a fast, centralized Internet to distribute our games for free.

There's nothing to stop us from making our voices heard now. And there will be plenty of voices. Among those voices, there will be plenty of mediocrity, and plenty of games that have no meaning to anyone outside the author and maybe her friends. But we'll find new ways to sort that shit. And imagine what we'll gain: real diversity, a plethora of voices and experiences, and a new avenue for human beings to tell their stories and connect with other human beings.

Are you excited? I'm excited just to be alive at this time. You and I are going to see some amazing things in our lifetime. Are you going to be a part of them?

Publishers claimed game creation as their private territory. And for a while, they convinced a lot of people that their claim was legitimate. But that's over now. This is the age of the videogame zinester.

Footnotes

⁸⁴ See the *selectbutton* forums at: <http://forums.selectbutton.net/viewtopic.php?t=33106>.

Appendix A

What to Use

There are many different ways to make digital games, and different people are most comfortable with different tools. Here I've tried to identify the tools I think are the most useful to new authors, based on a few criteria. First, that they require as little prior knowledge of game making as possible. And second, that they're reasonably self-contained. What I mean by this: There are a lot of different tasks required in making a game. For many of my games, I use one program to draw images, another to edit and resize them, another program to record and edit audio, others to generate sound effects, and a final program to put them all together and script the game rules. Here, I've tried to recommend tools that let you produce and edit images and animations within the tools themselves, or tools that don't require you to bring in outside assets at all.

Working with each of these tools will be a different experience, though there are commonalities between them, and the basic skills you learn in one can often be applied to any game-design project, using any tool. In discussing each of the tools below, I try to describe what the experience of working with the tool is like, as well as what it can teach you about game making that you can take with you to a different tool. Remember, you're not bound to a single tool, and I encourage you to experiment with as many as possible.

You'll eventually figure out what works best for you. In the meantime, try to figure out what you can make with what you can get your hands on. The more you do, the more you'll be capable of doing.

Klick & Play and the Games Factory



Klik & Play was originally designed as an educational tool for use in schools: a way for kids with little computer experience to make games. It proved popular enough that Clickteam, the creators, have continued to update the program since its initial 1994 release.

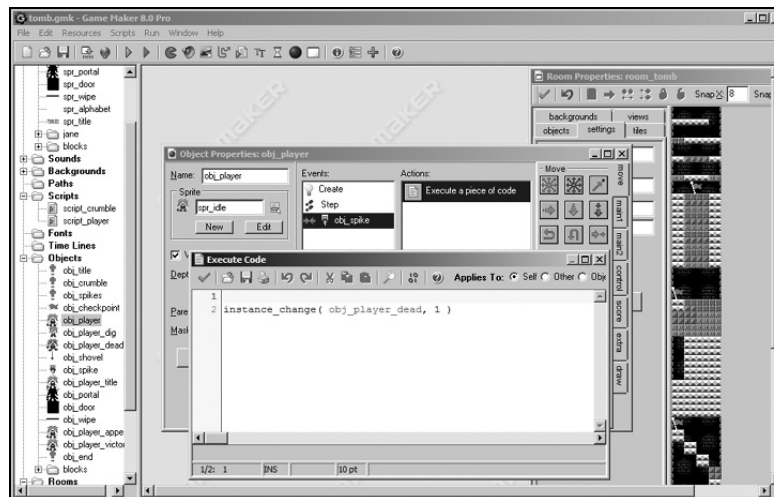
I think the original 1994 Klik & Play is almost the ideal place for an inexperienced game creator to start, with its simple, rules-based structure and its vast library of clip art. Unfortunately, it won't run on most contemporary versions of Windows. But in 2010, Clickteam released a free *Newgrounds* edition of The Games Factory, the third incarnation of Klik & Play. It has less clip art than the original Klik, but it publishes games directly to the *Newgrounds* website as Flash games—meaning that most people with a web browser will be able to play them (although *Newgrounds* users, a cantankerous lot, have the power to vote down and remove any game they dislike from the site). It will only run in Windows.

To use any Klik program, simply drag a character to wherever you want it on the screen. The character image can be one you found on the Internet, one you drew yourself using the program's art functions, or one of the pieces of clip art included with the program. Once your character is in the right place, you can give it some properties—how it moves, whether the player controls its movement, etc. Then, in an “event editor” screen, you

can set specific rules for that screen: “every time the character touches a penguin, give the player a hundred points,” for example, or “every thirty seconds, add another penguin to the screen.”

Above all, Klik & Play/The Games Factory will help you think about and plan your game in terms of rules, which is, after all, what games *are*. It will always be an aid to you to be able to envision the experience you want the player to have and then imagine which rules will create that experience, and Klik makes it easy to establish rules and think primarily about their effects and the interactions between them, rather than forcing you to focus on the details of lower-level scripting, displaying art, updating the position of objects on the screen, and other potential obstacles to novice creators. (Construct⁸⁵ is a newer and more complicated tool designed along the same basic lines as Klik & Play and The Games Factory, and might be a good place to go after experimenting with these older tools and learning some of the basics of this style of design.)
<http://www.newgrounds.com/wiki/creator-resources/game-dev-resources/the-games-factory-2>

Gamemaker



Game Maker is probably the most popular of these tools because it has the easiest metaphor for managing all the pieces of a game, one that's open enough to support a really diverse collection of graphical games. Every scene in a game is a “room”; the characters in those rooms are “objects”;

the images that you see are “sprites.” The concept of “rooms” makes it easy to mentally separate your game into a series of scenes and to then assemble them one by one. It’s particularly useful for laying out levels: just open up a room and stick objects wherever you want them to appear on the screen.

Initially, you’ll script the rules of your games with icons that each mean something: “move in this direction at this speed,” “wait for this long,” “play this sound.” As your games get more complex, though, using the icons will become cluttered and intractable, and you’ll want to transition to Game Maker’s scripting language, GML. GML looks like a programming language, but it’s easy to make the leap to GML from the icon-based vocabulary because most functions will correspond to the icons you’ve learned to use, and the basic logic of making the game is the same. From there, it’s easier to make the leap to a more traditional, restrictive programming language, or to similar scripting languages in other tools.

Regretfully, the corporation that distributes Game Maker, YoYo Games, has decided that in order to make a profit from their product, they have to spin it as a tool for professionals and small studios. Thus, they’ve hiked up the price to 40 dollars for the “standard” edition and 100 dollars for a version that creates web-based games. YoYo Games has always distributed both a “Pro” version of the program (costing 25 dollars) and a “Lite” version (costing nothing), with the only difference being that the “Lite” version was missing only a few of the flashier features of the “Pro” version. But the price hike coincided with a new version of the “Lite” program: one that stamps a watermark on every single screen of a finished game.

I recommend the Lite version of Game Maker 8.0^{[86](#)}—the last version before the watermark—to zinesters and newcomers. This version displays a “Made in Game Maker” banner on the loading screen for your game, and that’s it. You can make self-contained executable files that you can zip up and put on a file-sharing site like MediaFire.com, upload them to your website, anything.

Version 8.0 of Game Maker and all the earlier versions are made for Windows, but YoYo recently published a version for Mac.^{[87](#)} According to the company’s website, though, only the paid version of Game Maker for

Mac is able to produce distributable games.
<http://www.auntiepixelante.com/?p=1240>

Scratch

Scratch was developed by the Lifelong Kindergarten Group at MIT Media Lab to allow people of any age to create small games and animations. It's probably the simplest and fastest of the graphical game-making tools: there's almost no time between adding a rule to the game and seeing how it affects the play. Scratch is free for Windows, Mac, and Linux.

Scratch uses a very simple metaphor for rule construction: every rule is literally constructed, made up of a collection of jigsaw pieces that each represent part of a full sentence. Snapping a "when up arrow is pressed" piece onto a "move ten steps" piece, for example, tells a character to move forward when the player holds the up arrow key. Each rule is a single, adjustable object that sits on the "desktop" of the game editor and flashes whenever the game applies it. It's really easy to visually keep track of the rules of the game.

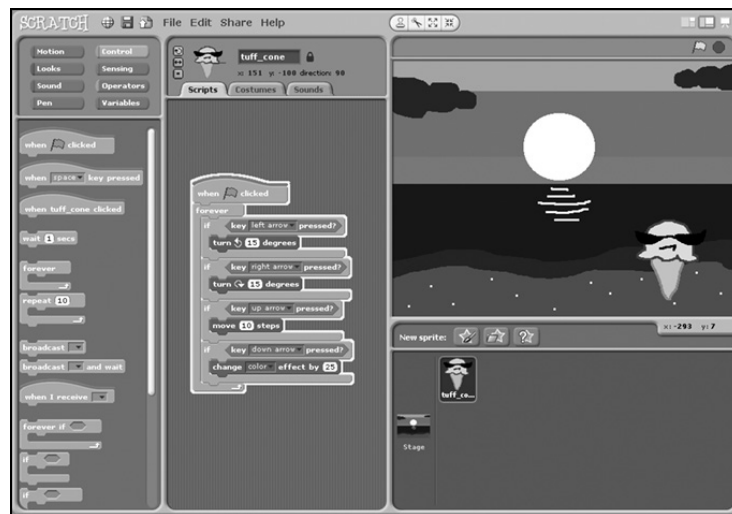
It's also designed to be as quick and dirty as possible, which is great when you're a first-time game maker without ambitious plans. There's a bunch of clip art, an option to pick a piece of clip art at random, and a "play a musical note" jigsaw piece that allows you to string together melodies really easily.

You can also share your games directly to the Scratch website. What's even better, though, is that you can download any game you play on the Scratch site, open it up, and see exactly how it works. See something you want in your game? Peek inside and see exactly how the author did it.

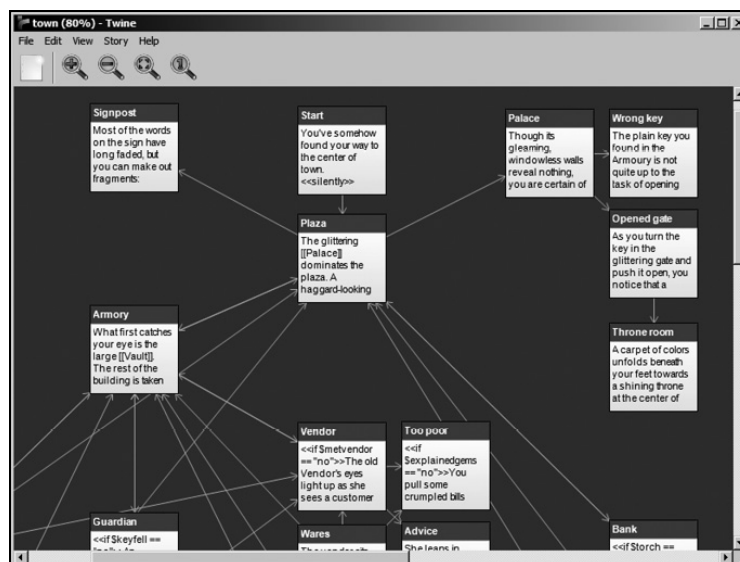
Scratch is a good starting place if Game Maker or The Games Factory are too intimidating. It's an excellent introduction to thinking about games as combinations of rules, and to thinking about the ways in which rules interact. It's also a good way just to get some experience making stuff, experimenting with flinging rules together and seeing what results you get.

After you've played with Scratch, you might consider trying Stencyl,^{[88](#)} which uses the concept of Scratch's jigsaw pieces as the basis for its own scripting. Stencyl creates Flash games, which you can publish to Stencyl's

website or to any other website. It lets you tinker with some fun, silly physics for your games. The program is still in its infancy, though, and I feel like it's a little too spread out right now; doing things that should be quick and simple take too much work. But it's likely that Stencyl will become more concise and usable as its creators update it.
<http://scratch.mit.edu>



Twine



Twine was created by Chris Klimas to make branching text stories that can be published online. It's a really clean and simple way to make Choose

Your Own Adventure–like story games. It doesn't require much from you other than the ability to write, and it runs on Windows, Mac, and Linux.

Twine keeps stories organized by drawing a map of all the passages you've written, with lines showing which scenes connect to which.

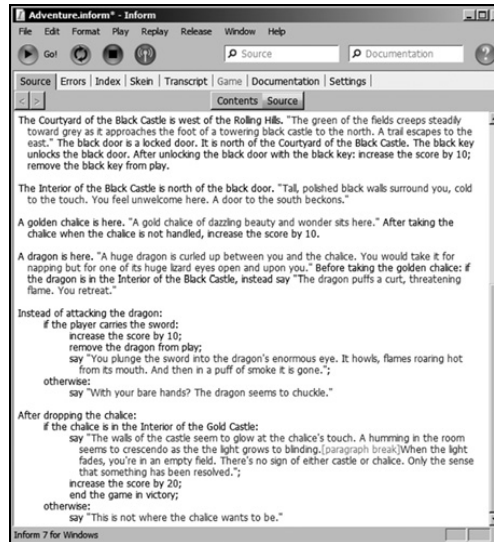
Passages are just blocks of text, but you can put brackets around a phrase to create a link to another passage. That's all you need to write a story in Twine, though there are other simple grammatical structures that will let you keep track of different things the player does within the story: which passages the player has seen, for example, or how many magic beans she's carrying around. And if you know CSS, the standard method of designing web pages, you can build features on top of the basic Twine structure to do even wackier things with your finished stories.

The program outputs stories as web pages—a full story being a single HTML file—meaning it's really easy to upload your text games to the Internet, or to pass them to other players. It is, in fact, the best distribution model I've seen in any tool: you can run an HTML game on any computer that has both the Internet and a contemporary web browser.

What you'll take away from Twine is an understanding of how to track the flow of a story with flags and variables: “Has the player seen this yet?” or “How many times has the player done this?” This is critical for any game involving a long and somewhat specific narrative. You'll also gain, in working with pure text, a sense of how to direct the player's interest with simple cues—if you're describing a gigantic castle hallway filled with suits of armor, tapestries, and ancient candles burning in iron fixtures, how do you get the player to notice that there's a loose brick in the wall?—which is a valuable skill in both text and graphical games.

If you want to try a similar tool that lets you accompany your prose with pictures and music, fiddle with Ren'Py,^{[89](#)} a tool for creating “visual novels”; essentially, Choose Your Own Adventures with pictures. <http://gimcrackd.com/etc/src>

Inform 7



Inform, created by the British poet and mathematician Graham Nelson, is another tool for making text-only games, but unlike in Twine—in which the player clicks on links within the text to navigate the game—Inform games require the player to type full sentences that the game interprets or responds to. Naturally this makes Inform (which is for Windows, Mac, and Linux) more complex than Twine, but not as complex to work in as one might think.

Inform 7 games are written in what its authors call “natural language” code. That means that the code you write (or other people’s code that you consult for reference) looks and reads like English sentences. For example, here’s a piece of text from the code of Emily Short’s *Bronze*,^{[90](#)} a retelling of *Beauty and the Beast*, that describes the rules for a helmet that the player can discover, pick up, and wear:

The helmet is a wearable thing in the Scarlet Tower. “On the windowsill, [a helmet] waits, for the use of the sentry.” Understand “very old helmet” or “helm” as the helmet. The description of the helmet is “A very old helmet that you have seen the Beast wear.”

It's still programming, and it still uses a very particular syntax that you'll have to learn. But it's a very simplified language—compare it, for example, with the Assembly code in chapter 2, or even a random sample of Game Maker script—because the tool itself does the work to reconcile the things you write with the way the system works. The Inform syntax is well documented and, as I've said, example code isn't hard to read. With a printout of the (free) Inform 7 handbook open, I made my first Inform 7 game in a weekend.

Inform is intended for people with no programming experience—that's the philosophy behind the “natural language” code. But the natural language can, in a way, be deceptive, and there's a lot more trial-and-error than you might expect while you're trying to work out the syntax that Inform expects before it'll do what you want. Because Inform games involve a text parser for player input rather than just linking the player to different predetermined routes, they can allow for a much richer level of player input: for example, the player in a Twine game might click on a link saying “loose brick” to connect with a passage of text indicating what happens when she pushes the loose brick, while in an Inform game she might type “push loose brick,” “jiggle loose brick,” or “ignore loose brick,” and these three possibilities might be scripted to have entirely different outcomes. And because the raw materials of Inform are language, it can be an easy way to experiment with managing, controlling, and guiding player input without having to worry as much about actually scripting the outcomes of different verbs—instead of scripting them, you just write them! But the tradeoff is a much steeper learning curve, and so if Inform is too intimidating at first, it might be easier to deal with after getting a handle on Twine.

Like Twine, Inform also lets you output your finished game as an HTML file which you can then easily share online with anyone who has a web browser. <http://inform7.com>

Warioware: D.I.Y.



Warioware D.I.Y. is a product for the Nintendo DS (which you'll need to use it), meaning that this is the most strictly commercial of the tools listed here. That's its strength, though: a team of people were paid a salary to come up with the most widely accessible game-making tool they could, and *Warioware* is one of the simplest tools described here.

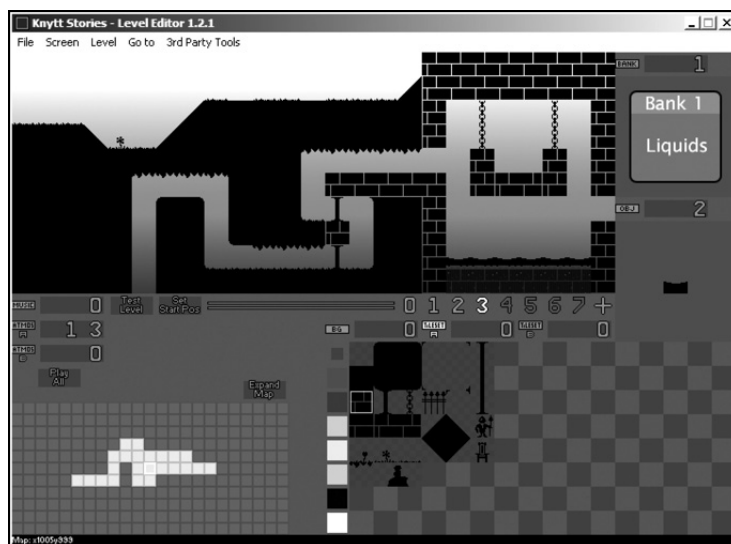
The DS uses a touch screen and stylus, so making the art for a game is just a matter of drawing it on the screen. Everyone knows how to draw, or at least doodle. The game provides a lot of similar shortcuts to creativity that I like: If you can't write music, you can hum into the microphone. Or you can grab a piece of music or a piece of game art from any of the games that come with the package.

That's another big deal: *Warioware* comes with ninety games made by the authors of the tool, and you can open each of them in the editor and see exactly how they work. You'll learn more about the scripting system from this than anything else.

You script objects (up to fifteen per game) by picking rules like "move a certain way" or "play this sound" from a menu, and applying them to characters on the screen. The rules are simple, but can be combined in surprising and clever ways. Here, though, is what I like the most about *Warioware*: each game is only seconds long, includes up to fifteen objects with five rules each, contains just a handful of art, and must be controlled entirely by tapping things with the stylus. In other words, working craftily within limitations is an invaluable skill you'll take away from *Warioware*.

The big failing of *Warioware*—and of any creative tool whose distribution infrastructure is entirely owned and operated by a corporation—is that it’s very hard to transmit your games to anyone. You need an individual code from every person you want to digitally share a game with (from their copy of *Warioware*), and you can only make two games available for download at any given time. The only way to get widespread exposure is to enter a game into one of Nintendo’s semimonthly themed game making competitions, the prize for which is publication and distribution, or—the avenue most authors take—just to record a video of the game and put it on YouTube. <http://www.wariowarediy.com>

Knytt Stories



Knytt Stories was created by Nicklas “Nifflas” Nygren as an expansion on his earlier Windows game, *Knytt*, that allows players to create their own stories using *Knytt*’s built-in verb set: running, jumping, and climbing around a world. (*Knytt Stories* is also Windows-only.) It’s a level-making tool, not a game-making tool, but I include it because there’s so little scripting involved and because there’s zero variance in the game’s basic verb set, making it a good place to start learning about level design.

Scripting is mostly limited to placing markers (visible or invisible) that move the player to different scenes, allowing for interesting narrative progression. The bulk of making a *Knytt Story* is placing tiles on a grid to

create a landscape for the player to navigate with the protagonist's running, jumping, and climbing abilities. The seven background, midground, and foreground layers on which tiles are placed can be hard to keep track of, but this is pretty close to pure level design, with minimal barriers between the creator's ideas and their implementation. (For something even simpler, try *Love Custom*,⁹¹ which lets you draw levels as simple images, bypassing the need to arrange tiles.)

Though the rules of *Knytt* are immutable, you can add your own pictures, tiles, and sounds to your stories. You can also use the music, sounds, and hundreds of background images and tile sets that come with *Knytt Stories*, which let you make a complete level without ever going outside the program. When you're done, *Knytt Stories* will compress your level and all the graphics and sound it uses into a package file that you can upload either to a file-sharing site or to Nifflas's own forums for exchanging *Knytt Stories*.⁹²

Because scripting exists almost exclusively to handle transitions, what you'll gain from *Knytt Stories* is a chance to experiment with and gain a sense for level design: how to arrange the elements of a game in order to create an engaging, interesting, fair experience, as well as how to tell a story. <http://nifflas.ni2.se/?page=Knytt+Stories>

ZZT



ZZT was created by Tim Sweeney in 1991 as a shareware game with a powerful "World Editor." The game is defunct in a lot of ways: it's a DOS

game, an operating system modern computers no longer support, meaning you have to use a DOSBox emulator⁹³ to run it (which ironically makes it equally usable on Windows, Mac, and Linux.) I include it because it's an amazing introduction to scripting and because it requires no resources outside of itself—in fact, it allows for no resources outside of itself.

ZZT is a text-mode game, meaning that every graphic that appears in the game will be one of 256 text characters in one of sixteen colors. All the sounds in the game are generated directly by the PC speaker—there are commands in *ZZT*'s scripting that will allow you to write PC speaker music for the game to play. Thus, *ZZT* is entirely self-contained. Not only do the limitations force you to be clever as a game maker, but the self-contained nature of the tool means you'll never have to leave the editor to create your own art or sound. Everything you need is immediately available to you.

The other wonderful thing about *ZZT* is its surprisingly robust scripting language, *ZZT-OOP* (for “Object-Oriented Programming”). You'll script objects with commands that have a simple structure like “#if contact #endgame” (or “if this object is touching the player, end the game”). The objects can send and receive messages, and thus interact with each other, which is where the possibility for complexity arises.

ZZT is an old tool—one of the first of its level of sophistication—and thus doesn't have exactly the distribution potential of the above tools, although there is a large community website and archive called Z2.⁹⁴ But despite its age, *ZZT* has the potential to teach you a lot about game logic and scripting. It's a really good and deceptively powerful introduction to thinking about how game elements behave in particular ways and communicate with one another, all the more so because its entirely self-contained nature leaves you free to focus on scripting and design. It's a weird old game, but not a bad starting point at all.
<http://zzt.belsambar.net/versions/>

Footnotes

⁸⁵ <http://www.scirra.com>.

⁸⁶ Again, <http://www.auntiepixelante.com/?p=1240>.

⁸⁷ <http://www.yoyogames.com/gamemaker/mac>.

⁸⁸ <http://www.stencyl.com>.

⁸⁹ <http://www.renpy.org>.

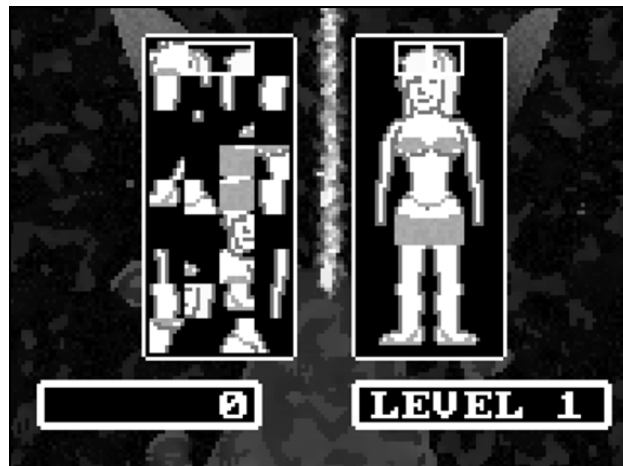
- 90 You can read more about *Bronze* and download the game at <http://inform7.com/learn/eg/bronze/index.html>.
- 91 <http://love-the-game.com/custom.html>.
- 92 Nifflas's forums can be found at: <http://nifflas.lpchip.nl/index.php?board=39.0>.
- 93 Find out more about DOSBox at: <http://www.dosbox.com>.
- 94 <http://zzt.org>.

Appendix B

Zinester Games

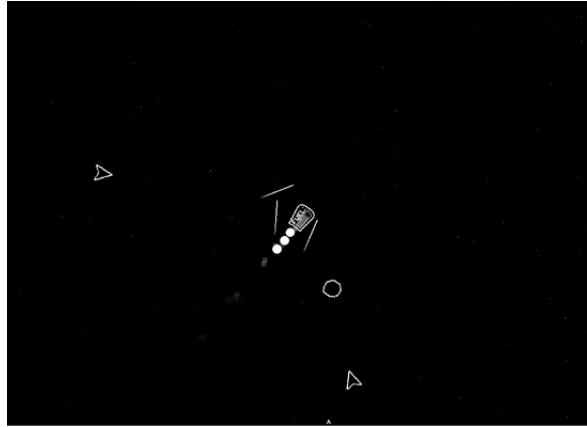
Each of the games below was created by just one or two people. Each is valuable, and I encourage you to play them if you can. Many of them, having been made in Game Maker, are playable in Windows only. (Borrow a friend's computer if you need to!) I've avoided including games I mention elsewhere in the book: consider this "further reading." I've tried to pick games that are simple enough that, in playing them, you should be able to see how you could make them or something like them. Each of them also shows off a different idea about the way digital games can be and what they can give us.

Striptease



Stephen “increpare” Lavelle is a prolific queer game creator from Ireland. *Striptease*, one of his most affecting works, uses the simple rules of an abstract sliding tile puzzle to achieve a narrative that explores themes of objectification, alienation, and the way we internalize and externalize our bodies. <http://www.increpare.com/2009/05/striptease>

Reset



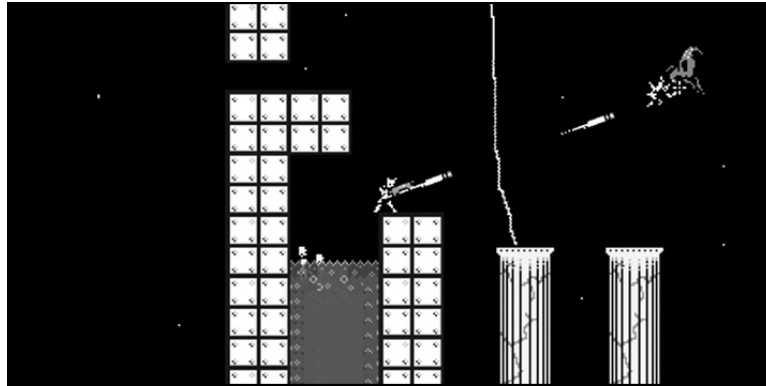
British author Robin Burkinshaw's *Reset* is a game set to a single piece of music: "Rest to Reset" by Trash80. Every event in the game responds to that music, and the player's fuel supply only lasts as long as the song. The game, which is neither won nor lost, is designed as a way of experiencing the song. It was made in Game Maker. <http://www.roburky.co.uk/?p=13>

All the Better to See You



Bento Smile works in the games industry as an artist and creates small games in her own time. *All the Better to See You* is a Ren'Py game she created for Valentine's Day of 2010. It uses the simple mechanical conceit of diminishing player options to characterize a relationship that becomes increasingly dominated by one party. <http://bentosmile.com/mini-games/all-the-better-to-see-you/>

All of Our Friends are Dead



Amon26 is a visual artist, musician, and game author. *All of Our Friends Are Dead*, a running, jumping, and shooting game inspired largely by id Software's 1993 game *DOOM*, was made in Game Maker using an “engine” that someone else had programmed. Amon's lack of complete control over his own creation serves the game's disjointed, hostile tone. http://amon26.site11.com/1_5_Games.html

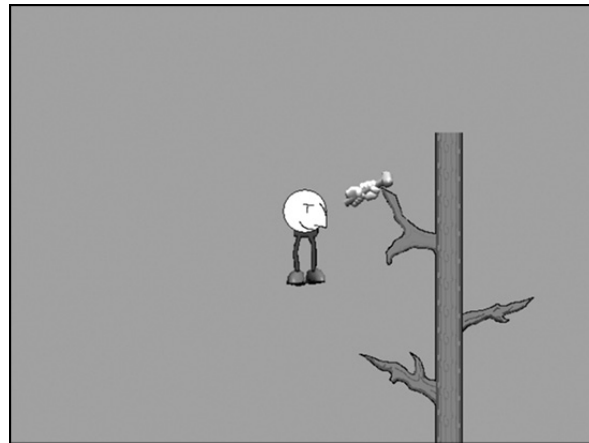
The Lake



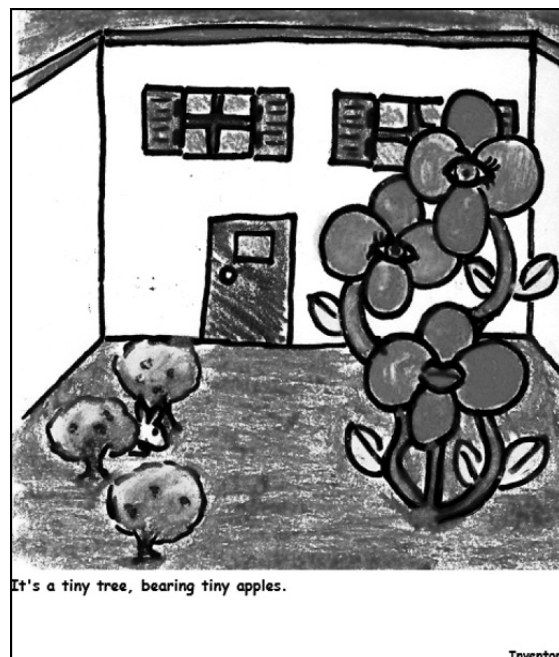
The author “agj” is interested in the expressive capabilities of digital games, and his works tend to have short, simple stories. *The Lake*, made in Construct, is based on a story fragment from H.P. Lovecraft's journal. The game uses only two buttons—the left and right oars of a small boat—to

achieve a subtle and unsettling story.
<http://forums.tigsource.com/index.php?action=printpage;topic=3750.0>

Sexy Hiking



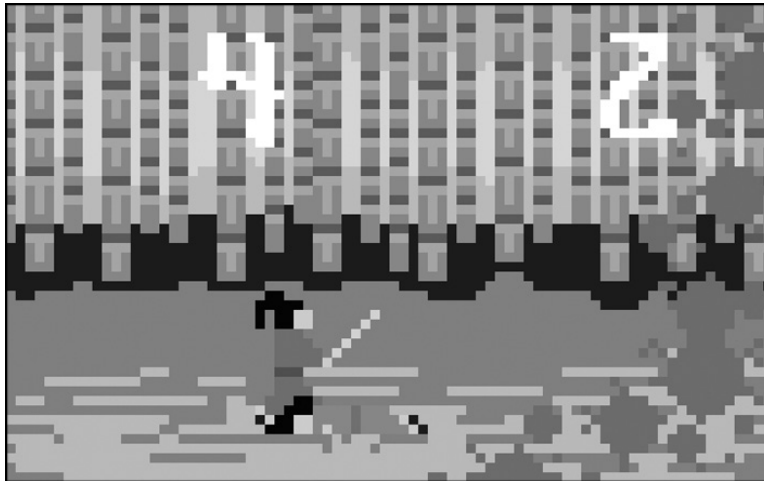
Author Jazzuo likely hit on the rules of *Sexy Hiking* by accident, then designed an intricate and challenging game around them. Jazzuo uses ugliness of presentation—the game is assembled from crude drawings and animation and stolen songs and clip art—as an avenue to unbounded creativity of design. *Sexy Hiking* was put together in Game Maker.
http://www.jazzuo.com/games/sexyhands/sexy_hiking/sexyhiking.htm



The Strange and Somewhat Sinister Tale of the House at Desert Bridge

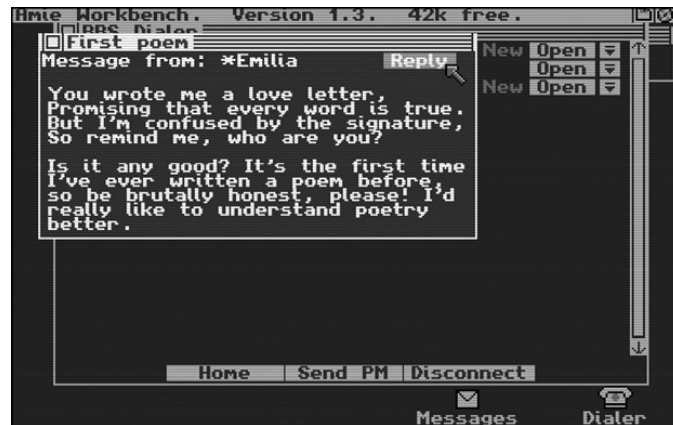
Verena and Jonas Kyratzes are a filmmaker and writer couple living in Germany. *The House at Desert Bridge*, a story about the importance of whimsy as a tool against oppression, is implemented very simply—play consists almost entirely of clicking on crayon illustrations, reading text, and typing information when prompted. <http://www.jonas-kyratzes.net/games/the-strange-and-somewhat-sinister-tale-of-the-house-at-desert-bridge/>

Bushido Edge



Bushido Edge, a Games Factory game by author “Pizza Time,” is inspired by the 1997 PlayStation game *Bushido Blade*. It’s a distillation of what’s most interesting about *Bushido Blade* or any two-player fighting game: the importance of reading and anticipating your opponent. It’s for two players only. <http://badmspaint.livejournal.com/195733.html>

Digital: A Love Story



Christine Love is a queer Canadian writer of fiction and digital stories. Her storytelling primarily concerns young love, particularly the kind that is mediated by technology. *Digital: A Love Story* is a “period piece,” modeled after the BBS networks of the 1980s and the culture they propagated. It was made in Ren’Py. (See also the semi-sequel, *Don’t Take It Personally, Babe, It Just Ain’t Your Story*, which is a meditation on our changing ideas of privacy.⁹⁵) <http://www.scoutshonour.com/digital/>

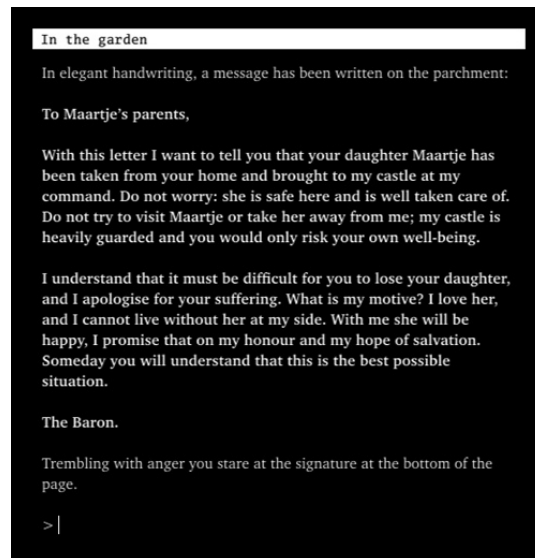
Super Green Shell Bros.

Leon “L” Arnott pays attention to the smaller details of game experiences. *Super Green Shell Bros.* is a takeoff of *Super Mario Bros.*, or rather a specific part of the experience of *Super Mario Bros.*: kicking a shell and chasing it as it mows over enemies.



A short game, it has a perfect pace that builds to a brilliant climax. Almost all of the graphics in this game are sampled from other games. <http://l.j-factor.com/gamemaker/#>

The Baron



Victor Gijssbers is a Dutch philosophy professor at Leiden University. His text game, *The Baron*—made in Inform—beckons the player into a difficult moral allegory, then asks her to decide how a heinous situation is resolved, or whether it even can be. *The Baron* explores themes that a commercial game publisher would be terrified to touch. <http://ifdb.tads.org/viewgame?id=weac28l51hiqfzxz>

Sonic 2 XL

Sonic 2 XL, by “Captain Bozo” and “Ranger,” is a hack of *Sonic the Hedgehog 2* for the SEGA Genesis, and will require a Genesis emulator to play. In this hack, the gold rings Sonic collects are replaced with fried onion rings that make him fatter and slower. It’s interesting both on a thematic level—as commentary on game-related obesity and consumerist collection goals in games—and on a play level: navigating the game without collecting so many rings as to become fat is a challenging

variation on the original game. <http://forums.sonicretro.org/index.php?showtopic=21739>



Footnotes

95 <http://scoutshonour.com/donttakeitpersonallybabeitjustaintyourstory>.



About the Author

ANNA ANTHROPY is a prolific game developer and critic. She is the creator of games such as *Calamity Annie*, *Mighty Jill Off*, and *Lesbian Spider-Queens of Mars* among many others. She lives in the East Bay area, California with her pet girlmonster and two lovely cats. She wants you to stop reading this and GO MAKE A GAME.

About Seven Stories Press

Seven Stories Press is an independent book publisher based in New York City. We publish works of the imagination by such writers as Nelson Algren, Russell Banks, Octavia E. Butler, Ani DiFranco, Assia Djebar, Ariel Dorfman, Coco Fusco, Barry Gifford, Hwang Sok-yong, Lee Stringer, and Kurt Vonnegut, to name a few, together with political titles by voices of conscience, including the Boston Women's Health Collective, Noam Chomsky, Angela Y. Davis, Human Rights Watch, Derrick Jensen, Ralph Nader, Loretta Napoleoni, Gary Null, Project Censored, Barbara Seaman, Alice Walker, Gary Webb, and Howard Zinn, among many others. Seven Stories Press believes publishers have a special responsibility to defend free speech and human rights, and to celebrate the gifts of the human imagination, wherever we can. For additional information, visit www.sevenstories.com.